Профиль кредитора

Листинг программного кода

Версия 1.0 · ноябрь 2026

Файлов: 144 · строк кода и документации: 5935

Состав: backend (Python, FastAPI), frontend (React, TypeScript), автономная версия для Windows, инфраструктура (Docker, Nginx), тесты. Не включены: зависимости (node_modules), lock-файлы, изображения.

Содержание

Корень проекта и инфраструктура

README.md 117 строк
.env.example 58 строк
.gitignore 15 строк
docker-compose.yml 58 строк
docker-compose.prod.yml 67 строк
nginx/nginx.conf 20 строк
nginx/nginx.prod.conf 39 строк
scripts/backup.sh 9 строк
scripts/deploy.sh 18 строк
scripts/restore.sh 13 строк
.github/workflows/windows-installer.yml 62 строк

Автономная версия для Windows (desktop)

desktop/creditor.spec 19 строк
desktop/installer.iss 57 строк
desktop/launcher.py 125 строк
desktop/requirements-desktop.txt 3 строк

Backend: приложение (app)

backend/app/__init__.py 1 строк
backend/app/api/__init__.py 1 строк
backend/app/api/deps.py 41 строк
backend/app/api/health.py 46 строк
backend/app/api/v1/__init__.py 8 строк
backend/app/api/v1/analysis.py 139 строк
backend/app/api/v1/api_keys.py 45 строк
backend/app/api/v1/auth.py 77 строк
backend/app/api/v1/cases.py 52 строк
backend/app/api/v1/documents.py 113 строк
backend/app/api/v1/users.py 44 строк
backend/app/api/v1/webhooks.py 42 строк
backend/app/core/__init__.py 1 строк
backend/app/core/cache.py 59 строк
backend/app/core/config.py 62 строк
backend/app/core/exceptions.py 20 строк
backend/app/core/logging.py 41 строк
backend/app/core/middleware/__init__.py 1 строк
backend/app/core/middleware/cors.py 11 строк
backend/app/core/middleware/errors.py 35 строк
backend/app/core/middleware/logging.py 17 строк
backend/app/core/middleware/rate_limit.py 43 строк
backend/app/core/middleware/request_id.py 17 строк
backend/app/core/security.py 33 строк
backend/app/db/__init__.py 1 строк
backend/app/db/base.py 16 строк
backend/app/db/models/__init__.py 8 строк
backend/app/db/models/audit.py 51 строк
backend/app/db/models/cases.py 54 строк
backend/app/db/models/creditors.py 43 строк

backend/app/db/models/documents.py	30 строк
backend/app/db/models/forecasts.py	52 строк
backend/app/db/models/reference.py	81 строк
backend/app/db/models/registry.py	49 строк
backend/app/db/models/users.py	75 строк
backend/app/db/repositories/__init__.py	1 строк
backend/app/db/seed.py	17 строк
backend/app/db/session.py	14 строк
backend/app/main.py	71 строк
backend/app/services/__init__.py	1 строк
backend/app/services/analysis.py	335 строк
backend/app/services/analytics.py	10 строк
backend/app/services/calculator.py	305 строк
backend/app/services/document_processor.py	129 строк
backend/app/services/email/__init__.py	15 строк
backend/app/services/email/base.py	6 строк
backend/app/services/email/mock.py	12 строк
backend/app/services/email/smtp.py	8 строк
backend/app/services/formulas.py	53 строк
backend/app/services/level_detector.py	24 строк
backend/app/services/mode_detector.py	24 строк
backend/app/services/modules/__init__.py	1 строк
backend/app/services/modules/m0_filter.py	12 строк
backend/app/services/modules/m1_queue.py	13 строк
backend/app/services/modules/m2_1_subordination.py	25 строк
backend/app/services/modules/m2_2_challenging.py	28 строк
backend/app/services/modules/m3_economics.py	26 строк
backend/app/services/modules/m4_practice.py	85 строк
backend/app/services/modules/m5_calc.py	32 строк
backend/app/services/parsers/__init__.py	36 строк
backend/app/services/parsers/efrsb.py	56 строк
backend/app/services/parsers/kad.py	48 строк
backend/app/services/parsers/mock.py	57 строк
backend/app/services/parsers/resilience.py	44 строк
backend/app/services/payments/__init__.py	8 строк
backend/app/services/payments/base.py	12 строк
backend/app/services/payments/mock.py	23 строк
backend/app/services/payments/yookassa.py	14 строк
backend/app/tasks/__init__.py	1 строк
backend/app/tasks/celery_app.py	6 строк
backend/app/tasks/parse_case.py	10 строк
backend/app/tasks/recalculate.py	18 строк
backend/app/tasks/send_email.py	8 строк
backend/app/tasks/update_stats.py	15 строк

Backend: миграции, тесты, конфигурация

backend/alembic/env.py	32 строк
backend/alembic/script.py.mako	21 строк
backend/alembic/versions/0001_initial.py	25 строк
backend/tests/conftest.py	26 строк
backend/tests/test_api.py	202 строк
backend/tests/test_calculator.py	79 строк
backend/tests/test_formulas.py	112 строк

backend/tests/test_infra.py	107 строк
backend/tests/test_modules.py	130 строк
backend/.dockerignore	5 строк
backend/.env.example	58 строк
backend/Dockerfile	12 строк
backend/alembic.ini	19 строк
backend/pytest.ini	4 строк
backend/requirements-dev.txt	4 строк
backend/requirements.txt	23 строк

Frontend (React + TypeScript)

frontend/src/App.tsx	46 строк
frontend/src/components/AnalysisShell.tsx	25 строк
frontend/src/components/CookieBanner.tsx	14 строк
frontend/src/components/Disclaimer.tsx	19 строк
frontend/src/components/Layout.tsx	84 строк
frontend/src/components/Traffic.tsx	27 строк
frontend/src/components/ui.tsx	53 строк
frontend/src/hooks/useAuth.ts	6 строк
frontend/src/hooks/useTheme.ts	13 строк
frontend/src/main.tsx	20 строк
frontend/src/pages/Audit.tsx	67 строк
frontend/src/pages/Cases.tsx	30 строк
frontend/src/pages/Details.tsx	113 строк
frontend/src/pages/Documents.tsx	85 строк
frontend/src/pages/Home.tsx	98 строк
frontend/src/pages/Legal.tsx	19 строк
frontend/src/pages/Login.tsx	40 строк
frontend/src/pages/Notifications.tsx	11 строк
frontend/src/pages/Pricing.tsx	29 строк
frontend/src/pages/Profile.tsx	43 строк
frontend/src/pages/Register.tsx	32 строк
frontend/src/pages/Registry.tsx	40 строк
frontend/src/pages/Summary.tsx	83 строк
frontend/src/services/api.ts	104 строк
frontend/src/services/format.ts	18 строк
frontend/src/styles/index.css	20 строк
frontend/src/styles/tokens.css	51 строк
frontend/.dockerignore	3 строк
frontend/Dockerfile	19 строк
frontend/e2e/smoke.spec.ts	37 строк
frontend/index.html	15 строк
frontend/package.json	35 строк
frontend/playwright.config.ts	16 строк
frontend/postcss.config.js	1 строк
frontend/public/icons/icon.svg	1 строк
frontend/public/manifest.json	14 строк
frontend/tailwind.config.js	20 строк
frontend/tsconfig.json	8 строк
frontend/vite.config.ts	25 строк

Корень проекта и инфраструктура

README.md (117 строк)

```
# Профиль кредитора

Веб-приложение для кредиторов в процедурах банкротства: по ИНН должника или номеру дела считает вероятностный прогноз выплаты, риски оспаривания и субординации, состояние конкурсной массы и выдаёт **решения** (что делать, почему, сколько стоит), а не сырые вероятности.

Реализация — по документам `ТЕХНИЧЕСКОЕ ЗАДАНИЕ v1.0` и `ТЕОРИЯ МЕТОДИКИ «ПРОФИЛЬ КРЕДИТОРА»`.
Сервис носит информационно-аналитический характер, не является юридической услугой.

## Быстрый старт (версия 1.0, localhost)

```bash
docker compose up --build
```

Что	Адрес
Приложение	http://localhost:5173
API / Swagger	http://localhost:8000/docs
Health / Ready / Metrics	http://localhost:8000/health · `/ready` · `/metrics`
Единая точка входа (опц.)	`docker compose --profile proxy up` → http://localhost:8080

Вход: **admin@local / admin**. `.env` не обязателен; для настройки — `cp .env.example .env`.

> По умолчанию `PARSERS_MODE=mock`: приложение работает на **демонстрационных данных**, детерминированно зависящих от ИНН (интерфейс помечает такой расчёт). Попробуйте `7701234567`, `7702234567`; ИНН, оканчивающийся на `000` → режим P1 (банкротства нет), на `00` → P2.
> Для реальных источников задайте `PARSERS_MODE=live` (см. «Ограничения»).

Запуск без Docker

```bash
# backend (SQLite, без Redis — кэш и rate-limit работают в памяти)
cd backend && pip install -r requirements-dev.txt
DATABASE_URL=sqlite+aiosqlite:///./dev.db ENVIRONMENT=local uvicorn app.main:app --reload
# frontend
cd frontend && npm install && npm run dev        # http://localhost:5173, проксирует /api на :8000
```

Установщик для Windows (.exe)

Автономная версия без Docker, Postgres и Redis: один `.exe` открывается **в собственном окне** (pywebview / Edge WebView2), без консоли; SQLite и лог `creditor.log` — в `%LOCALAPPDATA%\CreditorProfile`. Вход выполняется автоматически, без пароля (`LOCAL_AUTOLOGIN`, только с этого же компьютера; в Docker/публичном режиме выключено). Закрытие окна останавливает сервер. Окно строится на Edge WebView2 (встроен в Windows 11 и обновлённую Windows 10); установщик содержит его полный офлайн-пакет и ставит при отсутствии — интернет не нужен (поэтому установщик ~250–300 МБ; только 64-битная Windows 10/11). Если окно всё же не открылось — запасной вариант: браузер. Для отладки `CP_HEADLESS=1` запускает только сервер.

Сборка: GitHub → Actions → **Windows installer** → Run workflow; готовый `ProfilKreditora-Setup-1.0.0.exe` — в Artifacts. Либо вручную на Windows: `npm run build` во `frontend`, затем `pip install -r desktop/requirements-desktop.txt` и `pyinstaller desktop/creditor.spec` и `iscc desktop/installer.iss`.
Режим данных в автономной версии — демо (`PARSERS_MODE=mock`) + ручной ввод. OCR сканов требует отдельной установки Tesseract; Playwright (live-КАД) в exe не входит.

Тесты

```bash
cd backend && python -m pytest --cov=app                          # 93 теста, SQLite
TEST_DATABASE_URL=postgresql+asyncpg://postgres:postgres@localhost:5432/creditor_test python -m pytest  # на PostgreSQL
cd frontend && npm run build && npm run preview &                 # E2E: нужен запущенный backend на :8000
```
```

````
cd frontend && npx playwright test --trace off                  # при необходимости PW_CHROMIUM=/путь/к/chromium
```

## Устройство

```
backend/app/services/formulas.py        Ф1–Ф6 (чистые функции)
backend/app/services/modules/           М0 фильтр Бевзенко · М1 очередь · М2.1 субординация · М2.2 оспаривание
                                        М3 экономика · М4 практика судьи/АУ + балл независимости АУ · М5 расчёт
backend/app/services/calculator.py      оркестратор: режим → модули → сценарии (Ф5/Ф6) → приоритеты П0–П5 → стоимость действий
backend/app/services/mode_detector.py   режимы Р0–Р6;  level_detector.py — уровни полноты 0–5 по модулям
backend/app/services/parsers/           ЕФРСБ (httpx), КАД (Playwright), retry + circuit breaker, mock-данные
backend/app/services/document_processor.py   PDF / DOCX / OCR → факты → ручное подтверждение
backend/app/db/models/                  31 таблица, 8 слоёв;  backend/alembic/ — миграции
frontend/src/pages/                     Главная, Вход, Регистрация, Сводка, Детали, Аудит, Реестр, Документы, Профиль, Тарифы, /terms /privacy /offer
```

Три уровня погружения: **Сводка** (`/analysis/:id`) → **Детали** (сценарии, профиль АУ, стоимость, чек-лист) → **Аудит** (формулы, ДИ, источники, экспорт JSON/PDF).

## Feature flags

Версия 2.0 отличается от 1.0 **только** значениями ENV (полный список с комментариями — `.env.example`):

| Флаг | v1.0 | v2.0 |
|---|---|---|
| `PUBLIC_MODE` | false | true |
| `BILLING_ENABLED` | false | true |
| `PUBLIC_SIGNUP` | false | true |
| `EMAIL_SENDING` | false | true |
| `SSL_ENABLED` | false | true |

При `BILLING_ENABLED=false` всем присваивается `tier="admin"`, `/pricing` показывает «Оплата скоро»; при `PUBLIC_SIGNUP=false` `/auth/register` → 403 «Регистрация временно недоступна»; при `EMAIL_SENDING=false` письма только логируются.

## Переход на версию 2.0

1. Домен и VPS, Docker + docker compose.
2. `cp .env.example .env`, задать: `PUBLIC_MODE=true BILLING_ENABLED=true PUBLIC_SIGNUP=true EMAIL_SENDING=true SSL_ENABLED=true DOMAIN=…`, `ENVIRONMENT=prod`, сильные `SECRET_KEY`, `ADMIN_PASSWORD`, `POSTGRES_PASSWORD`, `PARSERS_MODE=live`.
3. Получить сертификат certbot (команда — в шапке `nginx/nginx.prod.conf`), положить `fullchain.pem` и `privkey.pem` в `nginx/ssl/`.
4. `./scripts/deploy.sh` (проверяет `.env`, поднимает `docker-compose.prod.yml`, применяет миграции).
5. Войти как `admin@local`, сменить пароль (Профиль → Смена пароля).
6. Реализовать `services/payments/yookassa.py` (интерфейс готов), `PAYMENT_PROVIDER=yookassa`, `YOOKASSA_SHOP_ID`, `YOOKASSA_SECRET_KEY`.
7. Реализовать `services/email/smtp.py`, `EMAIL_PROVIDER=smtp`, `SMTP_HOST`, `SMTP_USER`, `SMTP_PASSWORD`.
8. Проверить `/health`, `/docs`, `/pricing`.

Резервные копии: `scripts/backup.sh`, `scripts/restore.sh`; в prod-compose работает сервис `backup` (ежедневно, хранение 14 дней).

## Ручной ввод данных дела

Реальные ЕФРСБ/КАД из среды разработки недоступны (хосты закрыты), поэтому ручной ввод — полноценный путь: на главной «Ввести данные дела вручную» (или поле `manual` в `POST /api/v1/analyze`): дата заявления, активы, расходы, Σ3, статистика судьи, залог. Эти данные перекрывают данные парсеров, сохраняются в запросе (не теряются при смене режима и подтверждении фактов) и помечаются в сводке как «введено вручную».

## Допущения расчёта (где ТЗ неоднозначно)
````
````

```
- «Сумма требования» в форме — **неоплаченный остаток**; уже полученное исполнение указывается отдельно (окна оспаривания М2.2). Выплата считается в общей сумме `остаток + полученное`.
- Оспаривание: `X` возвращается в массу, требование восстанавливается как реестровое: `R_3(S+X, М3+X, Σ3+X)`. Пример главы 14 методики считает этот шаг иначе (внутренне непоследовательно), поэтому цифры не совпадают с ним буквально.
- Если реестр неизвестен: `Σ3 = 10 × требование`; если неизвестно имущество — отраслевой `p_zero` (43% юрлица / 94% граждане). Оба допущения выводятся в «Допущениях расчёта».
- Диапазон: ±20% от прогноза, при данных уровня 0–1 — ±50% (аналог «40–60% → 25–75%»).
- Вероятность успеха защиты сделки для оценки «прироста» принята 50% (`DEFENSE_SUCCESS` в `calculator.py`); действие рекомендуется, если прирост > издержек и ≥ 500 тыс. ₽.
- Модификатор судьи: `0,6 × отклонение`, ограничение ±0,15, при >15 п.п. — не слабее ±0,10, при незначимой выборке ×0,5.

## Ограничения и что не проверено

- **`PARSERS_MODE=live` не проверялся против реальных сайтов.** Эндпоинты ЕФРСБ взяты из ТЗ как есть (публичного API с такими путями может не существовать — понадобится ключ/договор), селекторы КАД Арбитр — ориентировочные и зависят от вёрстки сайта (у КАД есть антибот-защита). При сбое API отдаёт `status: "partial"` + `manual_input_required` и использует кэш. Судебная статистика (судья, АУ) в live-режиме **не собирается** — модуль М4 тогда работает без профилей (честно сообщая об этом).
- Docker-образы в среде разработки не собирались (нет Docker-демона): проверены лишь синтаксис compose-файлов (`docker compose config`) и скрипты (`bash -n`). Backend проверен на SQLite и PostgreSQL 16, миграция Alembic — на PostgreSQL 16.
- Критерий «расчёт совпадает с Excel на 10 делах (<5%)» не проверен: эталонных файлов нет. Формулы Ф1–Ф6 покрыты юнит-тестами, включая числа из методики (`D(10,100,120)=8,33`, `R_sec` и т. д.).
- Критерий «ИНН → профиль < 30 с» выполнен только для mock-режима.
- Извлечение фактов из документов — regex-эвристики; OCR требует `tesseract-ocr-rus` (есть в Docker-образе backend). Факты всегда подтверждаются вручную.
- Celery настроен в режиме `ALWAYS_EAGER`; фоновые задачи (`tasks/`) — каркас, автообновление по триггерам (глава 11 методики) не реализовано: есть только правила (`is_critical`, мёртвая зона, `delta_new`).
- Калибровка модели (глава 8, Brier/ECE) не реализована — ТЗ v1.0 её не требует.
- Миграция `0001_initial` строит схему из метаданных моделей; дальнейшие изменения — обычными `alembic revision --autogenerate`.
```

`.env.example` (58 строк)

```
# ============================================================
# «Профиль кредитора» — конфигурация. Скопируйте в .env.
# Версия 1.0 → 2.0 отличается ТОЛЬКО значениями этих переменных.
# ============================================================

# --- Режим -----------------------------------------------
PUBLIC_MODE=false          # true — публичный доступ (v2.0)
ENVIRONMENT=local          # local | prod. При local схема БД создаётся автоматически; в prod — alembic upgrade head

# --- Доступ ----------------------------------------------
PUBLIC_SIGNUP=false        # true — открыть /api/v1/auth/register
BILLING_ENABLED=false      # true — включить тарифы; false — всем tier="admin", /pricing показывает «Оплата скоро»
EMAIL_SENDING=false        # true — реально отправлять письма (нужен EMAIL_PROVIDER=smtp)
SSL_ENABLED=false          # true — HTTPS (nginx + certbot, см. README)

# --- Функции ---------------------------------------------
ANALYTICS_ENABLED=true     # внутренняя аналитика (таблица events)
METRICS_ENABLED=true       # Prometheus /metrics
RATE_LIMITING_ENABLED=true # лимит 100 запросов/мин на IP
RATE_LIMIT_PER_MINUTE=100
CIRCUIT_BREAKER_ENABLED=true
SHOW_COOKIE_BANNER=false   # показывать cookie-баннер (v2.0)

# --- Внешние сервисы -------------------------------------
PARSERS_MODE=mock          # mock — демо-данные без сети; live — реальные ЕФРСБ и КАД Арбитр
EFRSB_API_URL=https://fedresurs.ru/api/v1
EFRSB_API_KEY=
KAD_BASE_URL=https://kad.arbitr.ru
PAYMENT_PROVIDER=mock      # mock | yookassa
EMAIL_PROVIDER=mock        # mock | smtp
```

```
# YOOKASSA_SHOP_ID=
# YOOKASSA_SECRET_KEY=
# SMTP_HOST=
# SMTP_USER=
# SMTP_PASSWORD=

# --- База данных и кэш ------------------------------------
POSTGRES_PASSWORD=postgres # пароль PostgreSQL (docker-compose); в prod задайте свой
DATABASE_URL=postgresql+asyncpg://postgres:postgres@db:5432/creditor
REDIS_URL=redis://redis:6379/0

# --- Файлы ------------------------------------------------
STORAGE_TYPE=local
STORAGE_PATH=/app/storage
MAX_FILE_SIZE_MB=20

# --- Безопасность -----------------------------------------
SECRET_KEY=change-me-in-production     # python -c "import secrets; print(secrets.token_urlsafe(48))"
ADMIN_PASSWORD=admin                    # пароль admin@local при первом создании; в v2.0 задайте свой
JWT_ALGORITHM=HS256
JWT_EXPIRE_MINUTES=1440
CORS_ORIGINS=http://localhost:5173,http://localhost:8080

# --- Публичный доступ (v2.0) ------------------------------
DOMAIN=localhost

# --- Celery -----------------------------------------------
CELERY_TASK_ALWAYS_EAGER=true          # v1.0: фоновые задачи выполняются синхронно
```

.gitignore (15 строк)

```
.env
__pycache__/
*.pyc
.pytest_cache/
.coverage
*.db
node_modules/
dist/
test-results/
playwright-report/
backups/
backend/storage/
nginx/ssl/*.pem
nginx/certbot-www/
desktop/redist/
```

docker-compose.yml (58 строк)

```
# Версия 1.0 (локальный запуск): docker compose up --build
services:
  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: creditor
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-postgres}
    volumes: [pgdata:/var/lib/postgresql/data]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d creditor"]
      interval: 5s
      timeout: 3s
      retries: 10

  redis:
    image: redis:7-alpine
```

```
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 10

  backend:
    build: ./backend
    env_file: [{path: .env, required: false}]
    environment:
      DATABASE_URL: postgresql+asyncpg://postgres:${POSTGRES_PASSWORD:-postgres}@db:5432/creditor
      REDIS_URL: redis://redis:6379/0
    depends_on:
      db: {condition: service_healthy}
      redis: {condition: service_healthy}
    volumes: [storage:/app/storage]
    ports: ["8000:8000"]
    healthcheck:
      test: ["CMD", "curl", "-fs", "http://localhost:8000/ready"]
      interval: 10s
      timeout: 3s
      retries: 10

  frontend:
    build: {context: ./frontend, target: dev}
    depends_on: [backend]
    ports: ["5173:5173"]
    volumes:
      - ./frontend/src:/app/src
      - ./frontend/public:/app/public

  nginx:   # опционально: единая точка входа http://localhost:8080
    image: nginx:1.27-alpine
    profiles: ["proxy"]
    depends_on: [backend, frontend]
    volumes: [./nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro]
    ports: ["8080:80"]

volumes:
  pgdata:
  storage:
```

docker-compose.prod.yml (67 строк)

```
# Версия 2.0 (публичный деплой): ./scripts/deploy.sh  (или docker compose -f docker-compose.prod.yml up -d --build)
# Требует .env с PUBLIC_MODE=true, SSL_ENABLED=true, DOMAIN, SECRET_KEY, ADMIN_PASSWORD, POSTGRES_PASSWORD
services:
  db:
    image: postgres:15-alpine
    restart: always
    environment:
      POSTGRES_DB: creditor
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:?задайте POSTGRES_PASSWORD}
    volumes: [pgdata:/var/lib/postgresql/data, ./backups:/backups]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d creditor"]
      interval: 10s
      timeout: 3s
      retries: 10

  redis:
    image: redis:7-alpine
    restart: always

```

```
  backend:
    build: ./backend
    restart: always
    env_file: [{path: .env, required: false}]
    environment:
      ENVIRONMENT: prod
      DATABASE_URL: postgresql+asyncpg://postgres:${POSTGRES_PASSWORD}@db:5432/creditor
      REDIS_URL: redis://redis:6379/0
    depends_on:
      db: {condition: service_healthy}
      redis: {condition: service_started}
    volumes: [storage:/app/storage]

  frontend:
    build: {context: ./frontend, target: prod}
    restart: always

  nginx:
    image: nginx:1.27-alpine
    restart: always
    depends_on: [backend, frontend]
    volumes:
      - ./nginx/nginx.prod.conf:/etc/nginx/templates/default.conf.template:ro
      - ./nginx/ssl:/etc/nginx/ssl:ro
      - ./nginx/certbot-www:/var/www/certbot
    environment: {DOMAIN: "${DOMAIN}"}
    ports: ["80:80", "443:443"]

  backup:   # ежедневный pg_dump в ./backups (cron внутри контейнера)
    image: postgres:15-alpine
    restart: always
    depends_on: [db]
    environment: {PGPASSWORD: "${POSTGRES_PASSWORD}"}
    volumes: [./backups:/backups]
    entrypoint: ["/bin/sh", "-c"]
    command:
      - |
        while true; do
          pg_dump -h db -U postgres creditor | gzip > /backups/creditor_$$(date +%F).sql.gz
          find /backups -name 'creditor_*.sql.gz' -mtime +14 -delete
          sleep 86400
        done

volumes:
  pgdata:
  storage:
```

nginx/nginx.conf (20 строк)

```
# Локальный reverse proxy (v1.0): /api → backend, / → frontend (Vite dev, с WebSocket для HMR).
# Включается профилем:  docker compose --profile proxy up   → http://localhost:8080
server {
    listen 80;
    client_max_body_size 25m;

    location /api/    { proxy_pass http://backend:8000; proxy_set_header Host $host; proxy_set_header X-Real-IP $remote_addr; proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for; }
    location = /health  { proxy_pass http://backend:8000; }
    location = /ready   { proxy_pass http://backend:8000; }
    location /docs      { proxy_pass http://backend:8000; }
    location /openapi.json { proxy_pass http://backend:8000; }

    location / {
        proxy_pass http://frontend:5173;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
```

```
        proxy_set_header Host $host;
    }
}
```

nginx/nginx.prod.conf (39 строк)

```
# Продакшн (v2.0), SSL. Переменная ${DOMAIN} подставляется официальным образом nginx (templates).
#
# Получение сертификата (certbot, один раз, пока nginx работает на :80):
#   docker run --rm -v $PWD/nginx/certbot-www:/var/www/certbot -v $PWD/nginx/ssl:/etc/letsencrypt \
#     certbot/certbot certonly --webroot -w /var/www/certbot -d $DOMAIN --agree-tos -m you@example.com
# Затем скопируйте/слинкуйте файлы в nginx/ssl/ так, чтобы существовали:
#   nginx/ssl/fullchain.pem  и  nginx/ssl/privkey.pem     (из live/$DOMAIN/ certbot)
# Продление: повторить команду с `renew` по cron и выполнить `docker compose -f docker-compose.prod.yml exec nginx nginx -s reload`.

server {
    listen 80;
    server_name ${DOMAIN};
    location /.well-known/acme-challenge/ { root /var/www/certbot; }
    location / { return 301 https://$host$request_uri; }
}

server {
    listen 443 ssl;
    http2 on;
    server_name ${DOMAIN};
    client_max_body_size 25m;

    ssl_certificate     /etc/nginx/ssl/fullchain.pem;
    ssl_certificate_key /etc/nginx/ssl/privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3;
    add_header Strict-Transport-Security "max-age=31536000" always;
    add_header X-Content-Type-Options nosniff always;
    add_header X-Frame-Options DENY always;

    location /api/      { proxy_pass http://backend:8000; proxy_set_header Host $host; proxy_set_header X-Real-IP $remote_addr; proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for; proxy_set_header X-Forwarded-Proto https; }
    location = /health  { proxy_pass http://backend:8000; }
    location = /ready   { proxy_pass http://backend:8000; }
    location /docs      { proxy_pass http://backend:8000; }
    location /openapi.json { proxy_pass http://backend:8000; }
    # /metrics наружу не публикуется
    location = /metrics { return 404; }

    location / { proxy_pass http://frontend:80; proxy_set_header Host $host; }
}
```

scripts/backup.sh (9 строк)

```
#!/bin/bash
# Резервная копия БД в backups/creditor_ГГГГ-ММ-ДД_ЧЧММ.sql.gz
set -euo pipefail
cd "$(dirname "$0")/.."
COMPOSE_FILE=${COMPOSE_FILE:-docker-compose.yml}
mkdir -p backups
OUT="backups/creditor_$(date +%F_%H%M).sql.gz"
docker compose -f "$COMPOSE_FILE" exec -T db pg_dump -U postgres creditor | gzip > "$OUT"
echo "Сохранено: $OUT"
```

scripts/deploy.sh (18 строк)

```
#!/bin/bash
# Деплой версии 2.0. Запуск из корня проекта: ./scripts/deploy.sh
```

```
set -euo pipefail
cd "$(dirname "$0")/.."

[ -f .env ] || { echo "Нет .env — скопируйте .env.example и заполните (см. README, «Переход на версию 2.0»)"; exit 1; }
set -a; . ./.env; set +a
[ "${SECRET_KEY:-change-me-in-production}" != "change-me-in-production" ] || { echo "Задайте SECRET_KEY в .env"; exit 1; }
[ "${ADMIN_PASSWORD:-admin}" != "admin" ] || { echo "Задайте ADMIN_PASSWORD в .env"; exit 1; }
[ -n "${POSTGRES_PASSWORD:-}" ] && [ "$POSTGRES_PASSWORD" != "postgres" ] || { echo "Задайте POSTGRES_PASSWORD в .env"; exit 1; }
[ -f nginx/ssl/fullchain.pem ] && [ -f nginx/ssl/privkey.pem ] || { echo "Нет SSL-сертификатов в nginx/ssl/ (fullchain.pem, privkey.pem) — см. nginx/nginx.prod.conf"; exit 1; }

COMPOSE="docker compose -f docker-compose.prod.yml"
$COMPOSE down
$COMPOSE up -d --build
# Миграции выполняются при старте backend (CMD: alembic upgrade head); повторно — на всякий случай
$COMPOSE exec -T backend alembic upgrade head
echo "Готово. Проверь https://${DOMAIN}/health"
```

scripts/restore.sh (13 строк)

```
#!/bin/bash
# Восстановление: ./scripts/restore.sh backups/creditor_2026-11-08_1200.sql.gz
# ВНИМАНИЕ: текущая БД будет пересоздана.
set -euo pipefail
cd "$(dirname "$0")/.."
[ $# -eq 1 ] && [ -f "$1" ] || { echo "Использование: $0 <файл.sql.gz>"; exit 1; }
COMPOSE_FILE=${COMPOSE_FILE:-docker-compose.yml}
read -r -p "Текущая БД будет удалена и заменена содержимым $1. Продолжить? [y/N] " a
[ "$a" = "y" ] || exit 0
DC="docker compose -f $COMPOSE_FILE"
$DC exec -T db psql -U postgres -c "DROP DATABASE IF EXISTS creditor;" -c "CREATE DATABASE creditor;"
gunzip -c "$1" | $DC exec -T db psql -U postgres creditor
echo "Восстановлено из $1"
```

.github/workflows/windows-installer.yml (62 строк)

```
name: Windows installer
on:
  workflow_dispatch:
  push:
    tags: ["v*"]
    branches: ["claude/lucid-bell-5mfl49"]   # ручной запуск доступен только из основной ветки
    paths: ["desktop/**", ".github/workflows/windows-installer.yml"]
jobs:
  build:
    runs-on: windows-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: {node-version: 20}
      - uses: actions/setup-python@v5
        with: {python-version: "3.12"}
      - name: Frontend build
        working-directory: frontend
        run: |
          npm ci --no-audit --no-fund
          npx vite build
      - name: Backend deps + tests
        working-directory: backend
        run: |
          pip install -r requirements-dev.txt
          python -m pytest -q
      - name: Desktop deps
```

```
        run: pip install -r desktop/requirements-desktop.txt
      - name: PyInstaller
        working-directory: desktop
        run: pyinstaller --noconfirm --distpath ../dist --workpath ../build creditor.spec
      - name: Smoke-test exe
        shell: pwsh
        run: |
          $env:CP_PORT = "8799"; $env:CP_HEADLESS = "1"
          $p = Start-Process -FilePath dist\ProfilKreditora.exe -PassThru -WindowStyle Hidden
          $ok = $false
          foreach ($i in 1..30) { Start-Sleep 2; try { if ((Invoke-RestMethod
http://127.0.0.1:8799/health).status -eq "ok") { $ok = $true; break } } catch {} }
          Stop-Process -Id $p.Id -Force
          if (-not $ok) { throw "exe не запустился" }
      - name: Скачать офлайн-пакет WebView2 (включается в установщик целиком)
        shell: pwsh
        run: |
          New-Item -ItemType Directory -Force desktop\redist | Out-Null
          $f = "desktop\redist\WebView2RuntimeX64.exe"
          # Evergreen Standalone Installer x64 (официальная ссылка Microsoft)
          Invoke-WebRequest "https://go.microsoft.com/fwlink/?linkid=2124701" -OutFile $f
          $mb = [math]::Round((Get-Item $f).Length / 1MB)
          $sig = Get-AuthenticodeSignature $f
          "Размер: $mb МБ; подпись: $($sig.Status) / $($sig.SignerCertificate.Subject)"
          if ($mb -lt 100) { throw "Скачан не офлайн-пакет (слишком мал: $mb МБ)" }
          if ($sig.Status -ne "Valid" -or $sig.SignerCertificate.Subject -notmatch "Microsoft
Corporation") { throw "Подпись пакета не Microsoft" }
      - name: Inno Setup installer
        run: |
          choco install innosetup -y --no-progress
          & "C:\Program Files (x86)\Inno Setup 6\ISCC.exe" desktop\installer.iss
      - uses: actions/upload-artifact@v4
        with:
          name: ProfilKreditora-Setup
          path: |
            dist/ProfilKreditora-Setup-*.exe
            dist/ProfilKreditora.exe
```

Автономная версия для Windows (desktop)

desktop/creditor.spec (19 строк)

```
# PyInstaller: pyinstaller desktop/creditor.spec  (запускать из корня проекта, после `npm run build` во frontend)
from PyInstaller.utils.hooks import collect_all, collect_submodules

datas = [("../frontend/dist", "frontend_dist")]
hidden = collect_submodules("app") + collect_submodules("uvicorn") + ["aiosqlite", "sqlalchemy.dialects.sqlite",
                                                                      "sqlalchemy.ext.asyncio", "greenlet"]
binaries = []
for pkg in ("webview", "pdfplumber", "pdfminer", "docx", "pytesseract", "prometheus_fastapi_instrumentator", "structlog"):
    try:
        d, b, h = collect_all(pkg)
        datas += d; binaries += b; hidden += h
    except Exception:
        pass

a = Analysis(["launcher.py"], pathex=["../backend", "."], datas=datas, binaries=binaries, hiddenimports=hidden,
             excludes=["playwright", "celery", "asyncpg", "tkinter", "pytest"])
pyz = PYZ(a.pure)
exe = EXE(pyz, a.scripts, a.binaries, a.datas, [], name="ProfilKreditora", console=False,
          icon="../frontend/public/icons/icon.ico" if __import__("os").path.exists("../frontend/public/icons/icon.ico") else None)
```

desktop/installer.iss (57 строк)

```
; Inno Setup: iscc desktop\installer.iss  (после сборки dist\ProfilKreditora.exe)
#define AppName "Профиль кредитора"
#define AppVersion "1.0.0"

[Setup]
AppId={{7C1F6C9E-3B1A-4E5B-9A55-C0DE0CRED001}
AppName={#AppName}
AppVersion={#AppVersion}
DefaultDirName={autopf}\ProfilKreditora
DefaultGroupName={#AppName}
OutputDir=..\dist
OutputBaseFilename=ProfilKreditora-Setup-{#AppVersion}
Compression=lzma2
SolidCompression=yes
PrivilegesRequired=lowest
ArchitecturesInstallIn64BitMode=x64compatible
SetupIconFile=..\frontend\public\icons\icon.ico
UninstallDisplayIcon={app}\ProfilKreditora.exe
WizardStyle=modern

[Languages]
Name: "russian"; MessagesFile: "compiler:Languages\Russian.isl"

[Files]
Source: "..\dist\ProfilKreditora.exe"; DestDir: "{app}"; Flags: ignoreversion
; Полный офлайн-пакет WebView2 (скачивается на сборке, см. workflow): интернет при установке не нужен.
; Ставится только если компонента на компьютере нет.
Source: "redist\WebView2RuntimeX64.exe"; DestDir: "{tmp}"; Flags: deleteafterinstall nocompression; Check: NeedsWebView2

[Icons]
Name: "{group}\{#AppName}"; Filename: "{app}\ProfilKreditora.exe"
Name: "{autodesktop}\{#AppName}"; Filename: "{app}\ProfilKreditora.exe"; Tasks: desktopicon
```

```
[Tasks]
Name: "desktopicon"; Description: "Создать ярлык на рабочем столе"; Flags: unchecked

[Run]
Filename: "{tmp}\WebView2RuntimeX64.exe"; Parameters: "/silent /install"; StatusMsg: "Установка встроенного компонента отображения (WebView2)..."; Check: NeedsWebView2; Flags: waituntilterminated
Filename: "{app}\ProfilKreditora.exe"; Description: "Запустить {#AppName}"; Flags: nowait postinstall skipifsilent

[UninstallDelete]
; данные пользователя (%LOCALAPPDATA%\CreditorProfile) при удалении НЕ трогаем
Type: filesandordirs; Name: "{app}"

[Code]
// WebView2 Runtime: есть в Windows 11 и в обновлённой Windows 10; иначе ставим вместе с программой
const WV2 = '{F3017226-FE2A-4295-8BDF-00C3A9A7E4C5}';

function NeedsWebView2(): Boolean;
var v: String;
begin
  Result := not (
    RegQueryStringValue(HKLM, 'SOFTWARE\WOW6432Node\Microsoft\EdgeUpdate\Clients\' + WV2, 'pv', v) or
    RegQueryStringValue(HKLM, 'SOFTWARE\Microsoft\EdgeUpdate\Clients\' + WV2, 'pv', v) or
    RegQueryStringValue(HKCU, 'Software\Microsoft\EdgeUpdate\Clients\' + WV2, 'pv', v));
  if not Result then Result := (v = '') or (v = '0.0.0.0');
end;
```

desktop/launcher.py (125 строк)

```
"""Автономный запуск «Профиля кредитора»: собственное окно (pywebview), без консоли, Docker, Postgres и Redis.

SQLite, файлы и лог (creditor.log) — в %LOCALAPPDATA%\\CreditorProfile (Windows) или ~/.creditor-profile.
Режимы:  по умолчанию — окно приложения; CP_HEADLESS=1 — только сервер (для CI/отладки);
если нативное окно недоступно (нет WebView2/GUI) — открывается браузер.
"""
import multiprocessing
import os
import socket
import sys
import threading
import time
import urllib.request
import webbrowser
from pathlib import Path


def data_dir() -> Path:
    base = os.environ.get("LOCALAPPDATA")
    d = Path(base) / "CreditorProfile" if base else Path.home() / ".creditor-profile"
    d.mkdir(parents=True, exist_ok=True)
    return d


def resource(rel: str) -> Path:
    root = Path(getattr(sys, "_MEIPASS", Path(__file__).resolve().parent.parent))
    return root / rel


def free_port(preferred: int) -> int:
    for p in (preferred, 0):
        with socket.socket() as s:
            try:
                s.bind(("127.0.0.1", p))
                return s.getsockname()[1]
            except OSError:
```

```
                continue
    raise RuntimeError("Нет свободного порта")


def redirect_streams(d: Path) -> None:
    """В оконной сборке (console=False) sys.stdout/stderr == None — пишем в лог-файл."""
    if sys.stdout is None or sys.stderr is None or getattr(sys, "frozen", False):
        log = open(d / "creditor.log", "a", encoding="utf-8", buffering=1)
        sys.stdout = sys.stderr = log


def wait_ready(url: str, timeout: float = 60) -> bool:
    end = time.time() + timeout
    while time.time() < end:
        try:
            with urllib.request.urlopen(url + "/ready", timeout=2) as r:
                if r.status == 200:
                    return True
        except Exception:  # noqa: BLE001
            time.sleep(0.4)
    return False


def show_error(msg: str) -> None:
    try:
        if os.name == "nt":
            import ctypes
            ctypes.windll.user32.MessageBoxW(0, msg, "Профиль кредитора", 0x10)
    except Exception:  # noqa: BLE001
        pass


def main() -> None:
    d = data_dir()
    redirect_streams(d)
    port = free_port(int(os.environ.get("CP_PORT", "8765")))
    dist = resource("frontend_dist") if getattr(sys, "frozen", False) else resource("frontend/dist")
    os.environ.setdefault("DATABASE_URL", f"sqlite+aiosqlite:///{(d / 'creditor.db').as_posix()}")
    os.environ.setdefault("STORAGE_PATH", str(d / "storage"))
    os.environ.setdefault("ENVIRONMENT", "local")
    os.environ.setdefault("PARSERS_MODE", "mock")
    os.environ.setdefault("REDIS_URL", "redis://127.0.0.1:1/0")   # Redis нет → кэш и лимиты в памяти
    os.environ.setdefault("CORS_ORIGINS", f"http://127.0.0.1:{port}")
    os.environ["FRONTEND_DIST"] = str(dist)
    os.environ.setdefault("LOCAL_AUTOLOGIN", "true")   # одна машина, один пользователь — без окна входа
    key = d / "secret.key"          # SECRET_KEY генерируется один раз и хранится рядом с БД
    if not key.exists():
        import secrets
        key.write_text(secrets.token_urlsafe(48))
    os.environ.setdefault("SECRET_KEY", key.read_text().strip())

    if not getattr(sys, "frozen", False):          # запуск из исходников: backend рядом
        sys.path.insert(0, str(resource("backend")))
    import uvicorn
    from app.main import app

    server = uvicorn.Server(uvicorn.Config(app, host="127.0.0.1", port=port, log_level="warning",
log_config=None))
    url = f"http://127.0.0.1:{port}"
    print(f"Профиль кредитора: {url}  (данные: {d})", flush=True)

    if os.environ.get("CP_HEADLESS") == "1":
        server.run()
        return

    t = threading.Thread(target=server.run, daemon=True)
    t.start()
    if not wait_ready(url):
```

```
        show_error(f"Сервер не запустился. Подробности: {d / 'creditor.log'}")
        server.should_exit = True
        return
    try:
        import webview
        webview.create_window("Профиль кредитора", url, width=1280, height=860, min_size=(420, 600))
        webview.start()                      # блокирует до закрытия окна
    except Exception as e:  # noqa: BLE001 — нет WebView2/GUI-бэкенда: запасной вариант — браузер
        print(f"Нативное окно недоступно ({e!r}), открываю браузер", flush=True)
        webbrowser.open(url)
        show_error("Нативное окно недоступно (нужен Microsoft Edge WebView2). Приложение открыто в браузере; "
                   "чтобы остановить его, завершите ProfilKreditora.exe в диспетчере задач.")
        while t.is_alive():
            time.sleep(1)
        return
    server.should_exit = True                # окно закрыто → останавливаем сервер
    t.join(timeout=5)


if __name__ == "__main__":
    multiprocessing.freeze_support()
    main()
```

desktop/requirements-desktop.txt (3 строк)

```
-r ../backend/requirements-dev.txt
pyinstaller>=6.6
pywebview>=5.0
```

Backend: приложение (app)

backend/app/__init__.py (1 строк)

```

```

backend/app/api/__init__.py (1 строк)

```

```

backend/app/api/deps.py (41 строк)

```
from fastapi import Depends, Header, Request
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from app.core.config import settings
from app.core.exceptions import Unauthorized
from app.core.security import decode_token, hash_api_key
from app.db.models import ApiKey, AuditLog, User
from app.db.session import get_db


async def get_current_user(request: Request, authorization: str | None = Header(default=None),
                           x_api_key: str | None = Header(default=None),
                           db: AsyncSession = Depends(get_db)) -> User:
    user = None
    if x_api_key:
        res = await db.execute(select(ApiKey).where(ApiKey.key_hash == hash_api_key(x_api_key),
                                                    ApiKey.revoked_at.is_(None)))
        key = res.scalar_one_or_none()
        if key:
            user = await db.get(User, key.user_id)
    elif authorization and authorization.lower().startswith("bearer "):
        try:
            payload = decode_token(authorization[7:])
            if payload.get("kind") == "access":
                user = await db.get(User, int(payload["sub"]))
        except Exception:  # noqa: BLE001
            user = None
    if user is None:
        raise Unauthorized("Требуется авторизация")
    if not settings.BILLING_ENABLED:
        user.tier = "admin"
    request.state.user_id = user.id
    return user


async def audit(db: AsyncSession, request: Request, user: User, action: str, obj_type: str | None = None,
                obj_id: int | None = None) -> None:
    db.add(AuditLog(user_id=user.id, action=action, object_type=obj_type, object_id=obj_id,
                    ip_address=request.client.host if request.client else None))
    await db.commit()
```

backend/app/api/health.py (46 строк)

```
import httpx
from fastapi import APIRouter, Response
from sqlalchemy import text

from app.core.cache import redis_ping
from app.core.config import settings
```

```python
from app.db.session import engine

router = APIRouter(tags=["health"])


async def _check_db() -> bool:
    try:
        async with engine.connect() as c:
            await c.execute(text("SELECT 1"))
        return True
    except Exception:  # noqa: BLE001
        return False


async def _check_url(url: str) -> str:
    if settings.PARSERS_MODE == "mock":
        return "mock"
    try:
        async with httpx.AsyncClient(timeout=5) as c:
            r = await c.get(url)
        return "ok" if r.status_code < 500 else "down"
    except Exception:  # noqa: BLE001
        return "down"


@router.get("/health")
async def health():
    db, redis = await _check_db(), await redis_ping()
    return {"status": "ok" if db else "degraded", "db": "ok" if db else "down",
            "redis": "ok" if redis else "unavailable (in-memory fallback)",
            "efrsb": await _check_url(settings.EFRSB_API_URL), "kad": await
_check_url(settings.KAD_BASE_URL),
            "version": "1.0", "public_mode": settings.PUBLIC_MODE}


@router.get("/ready")
async def ready(response: Response):
    if await _check_db():
        return {"ready": True}
    response.status_code = 503
    return {"ready": False}
```

backend/app/api/v1/__init__.py (8 строк)

```python
from fastapi import APIRouter

from app.api.v1 import analysis, api_keys, auth, cases, documents, users, webhooks

api_router = APIRouter(prefix="/api/v1")
for r in (auth.router, analysis.router, cases.router, documents.claims_router, documents.docs_router,
          api_keys.router, webhooks.router, users.router):
    api_router.include_router(r)
```

backend/app/api/v1/analysis.py (139 строк)

```python
from datetime import date
from typing import Literal

from fastapi import APIRouter, Depends, Request
from pydantic import BaseModel, Field, model_validator
from sqlalchemy.ext.asyncio import AsyncSession

from app.api.deps import audit, get_current_user
from app.core.exceptions import AppError
from app.db.models import User
from app.db.session import get_db
```

```python
from app.services import analysis as svc

router = APIRouter(tags=["analysis"])

MODES = Literal["P0", "P1", "P2", "P3", "P4", "P5", "P6"]


class ClaimIn(BaseModel):
    amount: float | None = Field(default=None, gt=0, description="Сумма требования, ₽")
    construction_type: str = "supply"
    is_credit_contract: bool = False
    claim_date: date | None = None
    execution_date: date | None = None
    executed_amount: float = Field(default=0, ge=0)
    status: Literal["independent", "affiliated", "kdl"] = "independent"
    crisis: bool = False
    rescue_form: bool = False
    unusual_conditions: bool = False
    exceptions: dict[str, bool] = {}
    beneficial_interest: bool = False
    only_covenants: bool = False
    is_equivalent: bool = True
    has_harm_intent: bool = False
    is_gratuitous: bool = False
    is_interested: bool = False


class JudgeIn(BaseModel):
    name: str = "Судья (ввод вручную)"
    n_cases: int = Field(ge=0)
    approval_rate: float = Field(ge=0, le=1)
    is_significant: bool = False


class AUIn(BaseModel):
    name: str = "АУ (ввод вручную)"
    sro: str | None = None
    n_cases: int = Field(ge=0)
    filed_on_your_deal: bool = False
    filed_similar: int = Field(default=0, ge=0)
    passive_on_suspicious: bool = False
    losses_recovered: bool = False
    rau_top20: bool = False


class CollateralIn(BaseModel):
    C: float = Field(gt=0, description="Рыночная стоимость")
    L: float = Field(gt=0, le=1, description="Ликвидность 0…1")


class ManualIn(BaseModel):
    """Данные дела, введённые вручную (перекрывают данные парсеров)."""
    filing_date: date | None = None
    total_assets: float | None = Field(default=None, ge=0)
    current_expenses: float | None = Field(default=None, ge=0)
    first_priority: float | None = Field(default=None, ge=0)
    second_priority: float | None = Field(default=None, ge=0)
    sigma3: float | None = Field(default=None, gt=0, description="Сумма требований третьей очереди")
    sigma_sub: float | None = Field(default=None, ge=0)
    court_avg: float | None = Field(default=None, ge=0, le=1)
    judge: JudgeIn | None = None
    au: AUIn | None = None
    au_signals: dict[str, bool] | None = None
    collateral: list[CollateralIn] | None = None


class AnalyzeIn(BaseModel):
    inn: str | None = Field(default=None, pattern=r"^(\d{10}|\d{12})$")
```

```
    case_number: str | None = Field(default=None, max_length=50)
    claim: ClaimIn | None = None
    manual: ManualIn | None = None
    mode: MODES | None = None

    @model_validator(mode="after")
    def need_one(self):
        if not self.inn and not self.case_number:
            raise ValueError("Укажите ИНН или номер дела")
        return self


@router.post("/analyze")
async def analyze(body: AnalyzeIn, request: Request, db: AsyncSession = Depends(get_db),
                  user: User = Depends(get_current_user)):
    claim = body.claim.model_dump(mode="json") if body.claim else {}
    p = await svc.run_analysis(db, user, inn=body.inn, case_number=body.case_number, claim=claim,
                               manual=body.manual.model_dump(mode="json", exclude_none=True) if
body.manual else None,
                               mode_override=body.mode)
    await audit(db, request, user, "analyze", "creditor_profile", p.id)
    return svc.summary_view(p)


@router.get("/analysis/{profile_id}")
async def get_analysis(profile_id: int, db: AsyncSession = Depends(get_db), user: User =
Depends(get_current_user)):
    return svc.summary_view(await svc.get_profile(db, profile_id, user))


@router.get("/analysis/{profile_id}/details")
async def get_details(profile_id: int, db: AsyncSession = Depends(get_db), user: User =
Depends(get_current_user)):
    return svc.details_view(await svc.get_profile(db, profile_id, user))


@router.get("/analysis/{profile_id}/audit")
async def get_audit(profile_id: int, request: Request, db: AsyncSession = Depends(get_db),
                    user: User = Depends(get_current_user)):
    p = await svc.get_profile(db, profile_id, user)
    await audit(db, request, user, "view_audit", "creditor_profile", p.id)
    return svc.audit_view(p)


@router.get("/analysis/{profile_id}/registry")
async def get_registry(profile_id: int, db: AsyncSession = Depends(get_db), user: User =
Depends(get_current_user)):
    p = await svc.get_profile(db, profile_id, user)
    r = p.report or {}
    return {"id": p.id, "mode": p.mode, "available": p.mode in ("P5", "P6"), "rows":
r.get("registry_rows", []),
            "note": None if p.mode in ("P5", "P6") else "Сравнение с реестром доступно с режима P5
(реестр закрыт).",
            "demo_data": r.get("bundle_source") == "mock"}


@router.get("/analyses")
async def list_analyses(db: AsyncSession = Depends(get_db), user: User = Depends(get_current_user)):
    from sqlalchemy import select
    from app.db.models import CreditorProfile
    rows = (await db.execute(select(CreditorProfile).where(CreditorProfile.user_id == user.id)
                             .order_by(CreditorProfile.id.desc()).limit(50))).scalars().all()
    return [{"id": p.id, "mode": p.mode, "case": (p.report or {}).get("case"),
             "expected_amount": (p.report or {}).get("expected_amount"),
             "expected_pct": (p.report or {}).get("expected_pct"),
             "calculated_at": p.calculated_at.isoformat() if p.calculated_at else None} for p in rows]
```

backend/app/api/v1/api_keys.py (45 строк)

```python
import secrets
from datetime import datetime

from fastapi import APIRouter, Depends
from pydantic import BaseModel
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from app.api.deps import get_current_user
from app.core.exceptions import NotFound
from app.core.security import hash_api_key
from app.db.models import ApiKey, User
from app.db.session import get_db

router = APIRouter(prefix="/api-keys", tags=["api-keys"])


class KeyIn(BaseModel):
    name: str = "default"


@router.post("")
async def create(body: KeyIn, db: AsyncSession = Depends(get_db), user: User = Depends(get_current_user)):
    raw = "cp_" + secrets.token_urlsafe(32)
    k = ApiKey(user_id=user.id, key_hash=hash_api_key(raw), name=body.name)
    db.add(k)
    await db.commit()
    return {"id": k.id, "name": k.name, "key": raw, "note": "Ключ показывается один раз"}


@router.get("")
async def list_keys(db: AsyncSession = Depends(get_db), user: User = Depends(get_current_user)):
    rows = (await db.execute(select(ApiKey).where(ApiKey.user_id == user.id, ApiKey.revoked_at.is_(None)))).scalars().all()
    return [{"id": k.id, "name": k.name, "created_at": k.created_at.isoformat() if k.created_at else None,
             "last_used_at": k.last_used_at.isoformat() if k.last_used_at else None} for k in rows]


@router.delete("/{key_id}")
async def revoke(key_id: int, db: AsyncSession = Depends(get_db), user: User = Depends(get_current_user)):
    k = await db.get(ApiKey, key_id)
    if not k or k.user_id != user.id:
        raise NotFound("Ключ не найден")
    k.revoked_at = datetime.utcnow()
    await db.commit()
    return {"revoked": True}
```

backend/app/api/v1/auth.py (77 строк)

```python
from fastapi import APIRouter, Depends, Request
from pydantic import BaseModel
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from app.api.deps import get_current_user
from app.core.config import settings
from app.core.exceptions import Forbidden, Unauthorized
from app.core.security import create_token, decode_token, hash_password, verify_password
from app.db.models import User
from app.db.session import get_db

router = APIRouter(prefix="/auth", tags=["auth"])
```

```


class LoginIn(BaseModel):
    email: str
    password: str


class RegisterIn(BaseModel):
    email: str
    password: str


class RefreshIn(BaseModel):
    refresh_token: str


def _tokens(user_id: int) -> dict:
    return {"access_token": create_token(user_id), "refresh_token": create_token(user_id, "refresh", 60 * 24 * 14),
            "token_type": "bearer"}


@router.post("/login")
async def login(body: LoginIn, db: AsyncSession = Depends(get_db)):
    user = (await db.execute(select(User).where(User.email == body.email))).scalar_one_or_none()
    if not user or not user.password_hash or not verify_password(body.password, user.password_hash):
        raise Unauthorized("Неверный email или пароль")
    return _tokens(user.id)


@router.post("/register")
async def register(body: RegisterIn, db: AsyncSession = Depends(get_db)):
    if not settings.PUBLIC_SIGNUP:
        raise Forbidden("Регистрация временно недоступна")
    if (await db.execute(select(User).where(User.email == body.email))).scalar_one_or_none():
        raise Forbidden("Пользователь уже существует")
    user = User(email=body.email, password_hash=hash_password(body.password), role="user", tier="free")
    db.add(user)
    await db.commit()
    return _tokens(user.id)


@router.post("/refresh")
async def refresh(body: RefreshIn, db: AsyncSession = Depends(get_db)):
    try:
        payload = decode_token(body.refresh_token)
        assert payload.get("kind") == "refresh"
        user = await db.get(User, int(payload["sub"]))
    except Exception:  # noqa: BLE001
        raise Unauthorized("Недействительный токен")
    if user is None:
        raise Unauthorized("Недействительный токен")
    return _tokens(user.id)


@router.post("/local")
async def local_login(request: Request, db: AsyncSession = Depends(get_db)):
    """Вход без пароля: только при LOCAL_AUTOLOGIN=true, не в публичном режиме и только с этого же компьютера."""
    host = request.client.host if request.client else ""
    if not settings.LOCAL_AUTOLOGIN or settings.PUBLIC_MODE or host not in ("127.0.0.1", "::1"):
        raise Forbidden("Автовход недоступен")
    user = (await db.execute(select(User).where(User.email == "admin@local"))).scalar_one_or_none()
    if user is None:
        raise Unauthorized("Пользователь не найден")
    return _tokens(user.id)
```

backend/app/api/v1/cases.py (52 строк)

```
from typing import Literal

from fastapi import APIRouter, Depends, Query
from pydantic import BaseModel
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from app.api.deps import get_current_user
from app.core.exceptions import NotFound
from app.db.models import Case, Claim, CreditorProfile, User
from app.db.session import get_db
from app.services import analysis as svc
from app.services.mode_detector import detect_mode

router = APIRouter(prefix="/cases", tags=["cases"])


class SetModeIn(BaseModel):
    mode: Literal["P0", "P1", "P2", "P3", "P4", "P5", "P6"]


async def _latest_profile(db, case_id: int, user: User) -> CreditorProfile:
    q = (select(CreditorProfile).join(Claim, Claim.id == CreditorProfile.claim_id)
         .where(Claim.case_id == case_id, CreditorProfile.user_id == user.id)
         .order_by(CreditorProfile.id.desc()).limit(1))
    p = (await db.execute(q)).scalar_one_or_none()
    if p is None:
        raise NotFound("Анализ по делу не найден")
    return p


@router.get("/detect-mode")
async def detect(inn: str | None = Query(default=None), case_number: str | None = Query(default=None),
                 user: User = Depends(get_current_user)):
    return await detect_mode(inn=inn, case_number=case_number)


@router.post("/{case_id}/set-mode")
async def set_mode(case_id: int, body: SetModeIn, db: AsyncSession = Depends(get_db),
                   user: User = Depends(get_current_user)):
    base = await _latest_profile(db, case_id, user)
    req = (base.report or {}).get("_request", {})
    new = await svc.run_analysis(db, user, inn=req.get("inn"), case_number=req.get("case_number"),
                                 claim=req.get("claim"), manual=req.get("manual"),
mode_override=body.mode, base_profile=base)
    return {**svc.summary_view(new), "warning": "Вы выбрали режим вручную. Автоматический расчёт может "
            "отличаться. Убедитесь, что данные актуальны."}


@router.get("/{case_id}/levels")
async def levels(case_id: int, db: AsyncSession = Depends(get_db), user: User =
Depends(get_current_user)):
    p = await _latest_profile(db, case_id, user)
    return {"case_id": case_id, "mode": p.mode, "levels": p.information_levels}
```

backend/app/api/v1/documents.py (113 строк)

```
from datetime import datetime

from fastapi import APIRouter, Depends, File, Request, UploadFile
from pydantic import BaseModel
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from app.api.deps import audit, get_current_user
```

```
from app.core.config import settings
from app.core.exceptions import AppError, NotFound
from app.db.models import Claim, CreditorProfile, Document, ExtractedFact, User
from app.db.session import get_db
from app.services import analysis as svc
from app.services import document_processor as dp

router = APIRouter(tags=["documents"])
claims_router = APIRouter(prefix="/claims", tags=["documents"])
docs_router = APIRouter(prefix="/documents", tags=["documents"])


class ConfirmIn(BaseModel):
    fact_ids: list[int] | None = None   # None — подтвердить все факты документа


async def _own_claim(db, claim_id: int, user: User) -> Claim:
    claim = await db.get(Claim, claim_id)
    if claim is None:
        raise NotFound("Требование не найдено")
    q = select(CreditorProfile.id).where(CreditorProfile.claim_id == claim_id, CreditorProfile.user_id ==
user.id)
    if user.role != "admin" and (await db.execute(q)).first() is None:
        raise NotFound("Требование не найдено")
    return claim


@claims_router.post("/{claim_id}/documents")
async def upload(claim_id: int, request: Request, file: UploadFile = File(...),
                 db: AsyncSession = Depends(get_db), user: User = Depends(get_current_user)):
    await _own_claim(db, claim_id, user)
    mime = file.content_type or ""
    if mime not in dp.ALLOWED:
        raise AppError("Поддерживаются PDF, DOCX, JPG, PNG", 415)
    data = await file.read()
    if len(data) > settings.MAX_FILE_SIZE_MB * 1024 * 1024:
        raise AppError(f"Файл больше {settings.MAX_FILE_SIZE_MB} МБ", 413)
    path = dp.storage_path(settings.STORAGE_PATH, claim_id, file.filename or "file" + dp.ALLOWED[mime])
    path.write_bytes(data)
    doc = Document(claim_id=claim_id, document_type="other", file_path=str(path), file_size=len(data),
                   mime_type=mime, processing_status="processing")
    db.add(doc)
    await db.flush()
    try:
        text = dp.extract_text(str(path), mime)
        doc.document_type = dp.classify(text)
        facts = dp.extract_facts(text, doc.document_type)
        for f in facts:
            db.add(ExtractedFact(document_id=doc.id, field_name=f["field"], field_value=f["value"],
                                 confidence=f["confidence"], confirmed=False))
        doc.processing_status = "processed" if text.strip() else "no_text"
    except Exception as e:  # noqa: BLE001
        doc.processing_status = "failed"
        facts = []
    await db.commit()
    await audit(db, request, user, "upload_document", "document", doc.id)
    return {"id": doc.id, "status": doc.processing_status, "document_type": doc.document_type,
            "facts": [{"field": f["field"], "value": f["value"], "confidence": f["confidence"]} for f in
facts],
            "needs_confirmation": bool(facts)}


@claims_router.get("/{claim_id}/documents")
async def list_documents(claim_id: int, db: AsyncSession = Depends(get_db), user: User =
Depends(get_current_user)):
    await _own_claim(db, claim_id, user)
    docs = (await db.execute(select(Document).where(Document.claim_id ==
claim_id).order_by(Document.id))).scalars().all()
```

```
    return [{"id": d.id, "document_type": d.document_type, "mime_type": d.mime_type, "file_size": d.file_size,
             "status": d.processing_status, "created_at": d.created_at.isoformat() if d.created_at else None}
            for d in docs]


@docs_router.get("/{doc_id}/status")
async def status(doc_id: int, db: AsyncSession = Depends(get_db), user: User = Depends(get_current_user)):
    doc = await db.get(Document, doc_id)
    if doc is None:
        raise NotFound("Документ не найден")
    await _own_claim(db, doc.claim_id, user)
    facts = (await db.execute(select(ExtractedFact).where(ExtractedFact.document_id == doc_id))).scalars().all()
    return {"id": doc.id, "status": doc.processing_status, "document_type": doc.document_type,
            "facts": [{"id": f.id, "field": f.field_name, "value": f.field_value,
                       "confidence": float(f.confidence or 0), "confirmed": f.confirmed,
                       "label": "подтверждено" if f.confirmed else "требует проверки"} for f in facts]}


@docs_router.post("/{doc_id}/confirm-facts")
async def confirm(doc_id: int, body: ConfirmIn, db: AsyncSession = Depends(get_db),
                  user: User = Depends(get_current_user)):
    doc = await db.get(Document, doc_id)
    if doc is None:
        raise NotFound("Документ не найден")
    await _own_claim(db, doc.claim_id, user)
    facts = (await db.execute(select(ExtractedFact).where(ExtractedFact.document_id == doc_id))).scalars().all()
    for f in facts:
        if body.fact_ids is None or f.id in body.fact_ids:
            f.confirmed = True
    await db.commit()
    # пересчёт профиля с учётом подтверждённых фактов
    base = (await db.execute(select(CreditorProfile).where(CreditorProfile.claim_id == doc.claim_id)
                             .order_by(CreditorProfile.id.desc()).limit(1))).scalar_one_or_none()
    new_id = None
    if base:
        req = (base.report or {}).get("_request", {})
        mode = base.mode if (base.report or {}).get("mode_manual") else None
        new = await svc.run_analysis(db, user, inn=req.get("inn"), case_number=req.get("case_number"),
                                     claim=req.get("claim"), manual=req.get("manual"), mode_override=mode, base_profile=base)
        new_id = new.id
    return {"confirmed": sum(f.confirmed for f in facts), "total": len(facts), "analysis_id": new_id}
```

backend/app/api/v1/users.py (44 строк)

```
from fastapi import APIRouter, Depends
from pydantic import BaseModel, Field
from sqlalchemy.ext.asyncio import AsyncSession

from app.api.deps import get_current_user
from app.core.config import settings
from app.core.exceptions import AppError
from app.core.security import hash_password, verify_password
from app.db.models import User
from app.db.session import get_db

router = APIRouter(prefix="/users", tags=["users"])


@router.get("/me")
async def me(user: User = Depends(get_current_user)):
    return {"id": user.id, "email": user.email, "role": user.role, "tier": user.tier,
            "free_calculations_left": user.free_calculations_left,
```

```
            "flags": {"billing_enabled": settings.BILLING_ENABLED, "public_signup": settings.PUBLIC_SIGNUP,
                      "public_mode": settings.PUBLIC_MODE, "show_cookie_banner": settings.SHOW_COOKIE_BANNER,
                      "parsers_mode": settings.PARSERS_MODE}}


@router.get("/me/subscription")
async def subscription(user: User = Depends(get_current_user)):
    if not settings.BILLING_ENABLED:
        return {"status": "disabled", "tier": "admin", "message": "Оплата скоро"}
    return {"status": "none", "tier": user.tier,
            "until": user.subscription_until.isoformat() if user.subscription_until else None}


class PasswordIn(BaseModel):
    old_password: str
    new_password: str = Field(min_length=6)


@router.post("/me/password")
async def change_password(body: PasswordIn, user: User = Depends(get_current_user),
                          db: AsyncSession = Depends(get_db)):
    if not user.password_hash or not verify_password(body.old_password, user.password_hash):
        raise AppError("Неверный текущий пароль", 400)
    user.password_hash = hash_password(body.new_password)
    await db.commit()
    return {"changed": True}
```

backend/app/api/v1/webhooks.py (42 строк)

```
import secrets

from fastapi import APIRouter, Depends
from pydantic import BaseModel, HttpUrl
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from app.api.deps import get_current_user
from app.core.exceptions import NotFound
from app.db.models import User, WebhookSubscription
from app.db.session import get_db

router = APIRouter(prefix="/webhooks", tags=["webhooks"])


class HookIn(BaseModel):
    url: HttpUrl
    events: list[str] = ["analysis.updated"]


@router.post("")
async def create(body: HookIn, db: AsyncSession = Depends(get_db), user: User = Depends(get_current_user)):
    h = WebhookSubscription(user_id=user.id, url=str(body.url), events=body.events, secret=secrets.token_hex(16))
    db.add(h)
    await db.commit()
    return {"id": h.id, "url": h.url, "events": h.events, "secret": h.secret}


@router.get("")
async def list_hooks(db: AsyncSession = Depends(get_db), user: User = Depends(get_current_user)):
    rows = (await db.execute(select(WebhookSubscription).where(WebhookSubscription.user_id == user.id))).scalars().all()
    return [{"id": h.id, "url": h.url, "events": h.events} for h in rows]


```

```
@router.delete("/{hook_id}")
async def delete(hook_id: int, db: AsyncSession = Depends(get_db), user: User =
Depends(get_current_user)):
    h = await db.get(WebhookSubscription, hook_id)
    if not h or h.user_id != user.id:
        raise NotFound("Webhook не найден")
    await db.delete(h)
    await db.commit()
    return {"deleted": True}
```

backend/app/core/__init__.py (1 строк)

```

```

backend/app/core/cache.py (59 строк)

```
"""Кэш: Redis, при недоступности — in-memory (локальный запуск без Redis не падает)."""
import json
import time

from app.core.config import settings

_mem: dict[str, tuple[float, str]] = {}
_redis = None
_redis_failed = False


def _client():
    global _redis, _redis_failed
    if _redis_failed:
        return None
    if _redis is None:
        try:
            import redis.asyncio as aioredis
            _redis = aioredis.from_url(settings.REDIS_URL, socket_connect_timeout=1, socket_timeout=1)
        except Exception:
            _redis_failed = True
            return None
    return _redis


async def cache_get(key: str):
    c = _client()
    if c is not None:
        try:
            v = await c.get(key)
            return json.loads(v) if v else None
        except Exception:
            pass
    item = _mem.get(key)
    if item and item[0] > time.time():
        return json.loads(item[1])
    return None


async def cache_set(key: str, value, ttl: int) -> None:
    raw = json.dumps(value, default=str)
    c = _client()
    if c is not None:
        try:
            await c.set(key, raw, ex=ttl)
            return
        except Exception:
            pass
    _mem[key] = (time.time() + ttl, raw)

```

```

async def redis_ping() -> bool:
    c = _client()
    if c is None:
        return False
    try:
        return bool(await c.ping())
    except Exception:
        return False
```

backend/app/core/config.py (62 строк)

```
"""Конфигурация и feature flags. Версия 2.0 отличается от 1.0 только значениями ENV."""
from pydantic_settings import BaseSettings, SettingsConfigDict


class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")

    # Режим
    PUBLIC_MODE: bool = False
    ENVIRONMENT: str = "local"

    # Доступ
    PUBLIC_SIGNUP: bool = False
    BILLING_ENABLED: bool = False
    EMAIL_SENDING: bool = False

    # Функции
    ANALYTICS_ENABLED: bool = True
    METRICS_ENABLED: bool = True
    RATE_LIMITING_ENABLED: bool = True
    CIRCUIT_BREAKER_ENABLED: bool = True
    SHOW_COOKIE_BANNER: bool = False

    # Внешние сервисы
    PARSERS_MODE: str = "mock"          # mock | live — mock не ходит во внешние сети
    EFRSB_API_URL: str = "https://fedresurs.ru/api/v1"
    EFRSB_API_KEY: str = ""
    KAD_BASE_URL: str = "https://kad.arbitr.ru"
    PAYMENT_PROVIDER: str = "mock"
    EMAIL_PROVIDER: str = "mock"

    # База данных
    DATABASE_URL: str = "postgresql+asyncpg://postgres:postgres@db:5432/creditor"
    REDIS_URL: str = "redis://redis:6379/0"

    # Файлы
    STORAGE_TYPE: str = "local"
    STORAGE_PATH: str = "/app/storage"
    MAX_FILE_SIZE_MB: int = 20

    # Безопасность
    ADMIN_PASSWORD: str = "admin"      # пароль admin@local при первом создании; в v2.0 задать свой
    SECRET_KEY: str = "change-me-in-production"
    JWT_ALGORITHM: str = "HS256"
    JWT_EXPIRE_MINUTES: int = 1440
    CORS_ORIGINS: str = "http://localhost:5173"
    RATE_LIMIT_PER_MINUTE: int = 100

    # Публичный доступ (версия 2.0)
    DOMAIN: str = "localhost"
    SSL_ENABLED: bool = False

    # Автономная сборка (desktop): каталог со сборкой фронтенда, раздаётся самим backend
    FRONTEND_DIST: str = ""
    # Вход без пароля для локальной однопользовательской программы (включает только desktop-лаунчер)
```

```
    LOCAL_AUTOLOGIN: bool = False

    # Celery
    CELERY_TASK_ALWAYS_EAGER: bool = True


settings = Settings()
```

backend/app/core/exceptions.py (20 строк)

```
class AppError(Exception):
    status_code = 400

    def __init__(self, message: str, status_code: int | None = None):
        super().__init__(message)
        self.message = message
        if status_code:
            self.status_code = status_code


class NotFound(AppError):
    status_code = 404


class Forbidden(AppError):
    status_code = 403


class Unauthorized(AppError):
    status_code = 401
```

backend/app/core/logging.py (41 строк)

```
"""structlog: JSON-логи в stdout с маскированием персональных данных."""
import logging
import re
import sys

import structlog

EMAIL_RE = re.compile(r"\b([A-Za-z0-9._%+-])[A-Za-z0-9._%+-]*@([A-Za-z0-9.-]+\.[A-Za-z]{2,}|local)\b")
INN_RE = re.compile(r"\b(\d{4})\d{2,6}(\d{2})\b")


def mask(value: str) -> str:
    """email → a***@domain, ИНН → 7701****67."""
    value = EMAIL_RE.sub(lambda m: f"{m.group(1)}***@{m.group(2)}", value)
    return INN_RE.sub(lambda m: f"{m.group(1)}****{m.group(2)}", value)


def _mask_processor(_, __, event_dict):
    for k, v in list(event_dict.items()):
        if isinstance(v, str):
            event_dict[k] = mask(v)
    return event_dict


def configure_logging(level: int = logging.INFO) -> None:
    logging.basicConfig(format="%(message)s", stream=sys.stdout, level=level)
    structlog.configure(
        processors=[
            structlog.contextvars.merge_contextvars,
            structlog.processors.add_log_level,
            structlog.processors.TimeStamper(fmt="iso"),
            _mask_processor,
            structlog.processors.JSONRenderer(ensure_ascii=False),
```

```
        ],
        wrapper_class=structlog.make_filtering_bound_logger(level),
        cache_logger_on_first_use=False,
    )


def get_logger(name: str = "app"):
    return structlog.get_logger(name)
```

backend/app/core/middleware/__init__.py (1 строк)

```

```

backend/app/core/middleware/cors.py (11 строк)

```
from fastapi.middleware.cors import CORSMiddleware

from app.core.config import settings


def add_cors(app):
    origins = [o.strip() for o in settings.CORS_ORIGINS.split(",") if o.strip()]
    if settings.PUBLIC_MODE:
        origins.append(f"{'https' if settings.SSL_ENABLED else 'http'}://{settings.DOMAIN}")
    app.add_middleware(CORSMiddleware, allow_origins=origins, allow_credentials=True,
                       allow_methods=["*"], allow_headers=["*"])
```

backend/app/core/middleware/errors.py (35 строк)

```
"""Глобальные обработчики ошибок: {"error": "...", "request_id": "..."}."""
from fastapi import Request
from fastapi.encoders import jsonable_encoder
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse
from starlette.exceptions import HTTPException as StarletteHTTPException

from app.core.exceptions import AppError
from app.core.logging import get_logger

log = get_logger("errors")


def _rid(request: Request):
    return getattr(request.state, "request_id", None)


def register_error_handlers(app):
    @app.exception_handler(AppError)
    async def app_error(request: Request, exc: AppError):
        return JSONResponse({"error": exc.message, "request_id": _rid(request)},
status_code=exc.status_code)

    @app.exception_handler(StarletteHTTPException)
    async def http_error(request: Request, exc: StarletteHTTPException):
        return JSONResponse({"error": str(exc.detail), "request_id": _rid(request)},
status_code=exc.status_code)

    @app.exception_handler(RequestValidationError)
    async def validation_error(request: Request, exc: RequestValidationError):
        return JSONResponse({"error": "Некорректные данные запроса", "details":
jsonable_encoder(exc.errors(), custom_encoder={Exception: str}),
                             "request_id": _rid(request)}, status_code=422)

    @app.exception_handler(Exception)
    async def unhandled(request: Request, exc: Exception):
```

```
        log.error("unhandled", error=repr(exc), request_id=_rid(request))
        return JSONResponse({"error": "Внутренняя ошибка сервера", "request_id": _rid(request)},
status_code=500)
```

backend/app/core/middleware/logging.py (17 строк)

```
import time

from starlette.middleware.base import BaseHTTPMiddleware

from app.core.logging import get_logger

log = get_logger("http")


class LoggingMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request, call_next):
        start = time.perf_counter()
        response = await call_next(request)
        log.info("request", method=request.method, path=request.url.path, status=response.status_code,
                 ms=round((time.perf_counter() - start) * 1000, 1),
                 request_id=getattr(request.state, "request_id", None))
        return response
```

backend/app/core/middleware/rate_limit.py (43 строк)

```
"""Лимит по IP (100 запросов/мин). Redis; при его отсутствии — счётчик в памяти."""
import time

from fastapi.responses import JSONResponse
from starlette.middleware.base import BaseHTTPMiddleware

from app.core import cache
from app.core.config import settings

_local: dict[str, tuple[int, int]] = {}
EXEMPT = ("/health", "/ready", "/metrics")


class RateLimitMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request, call_next):
        if not settings.RATE_LIMITING_ENABLED or request.url.path in EXEMPT:
            return await call_next(request)
        ip = request.client.host if request.client else "unknown"
        window = int(time.time() // 60)
        count = await self._incr(f"rl:{ip}:{window}")
        if count > settings.RATE_LIMIT_PER_MINUTE:
            return JSONResponse({"error": "Слишком много запросов",
                                 "request_id": getattr(request.state, "request_id", None)},
                                status_code=429, headers={"Retry-After": "60"})
        return await call_next(request)

    async def _incr(self, key: str) -> int:
        c = cache._client()
        if c is not None:
            try:
                n = await c.incr(key)
                if n == 1:
                    await c.expire(key, 61)
                return int(n)
            except Exception:
                pass
        window = int(key.rsplit(":", 1)[1])
        base = key.rsplit(":", 1)[0]
        for k in [k for k, (w, _) in _local.items() if w != window]:
            _local.pop(k, None)
```

```
        w, n = _local.get(base, (window, 0))
        _local[base] = (window, n + 1)
        return n + 1
```

backend/app/core/middleware/request_id.py (17 строк)

```
import uuid

import structlog
from starlette.middleware.base import BaseHTTPMiddleware


class RequestIDMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request, call_next):
        rid = request.headers.get("X-Request-ID") or uuid.uuid4().hex
        request.state.request_id = rid
        structlog.contextvars.bind_contextvars(request_id=rid)
        try:
            response = await call_next(request)
        finally:
            structlog.contextvars.clear_contextvars()
        response.headers["X-Request-ID"] = rid
        return response
```

backend/app/core/security.py (33 строк)

```
"""Пароли (bcrypt), JWT, хеш API-ключей."""
import hashlib
from datetime import datetime, timedelta, timezone

import bcrypt
import jwt

from app.core.config import settings


def hash_password(password: str) -> str:
    return bcrypt.hashpw(password.encode()[:72], bcrypt.gensalt()).decode()


def verify_password(password: str, hashed: str) -> bool:
    try:
        return bcrypt.checkpw(password.encode()[:72], hashed.encode())
    except ValueError:
        return False


def create_token(user_id: int, kind: str = "access", minutes: int | None = None) -> str:
    exp = datetime.now(timezone.utc) + timedelta(minutes=minutes or settings.JWT_EXPIRE_MINUTES)
    return jwt.encode({"sub": str(user_id), "kind": kind, "exp": exp},
                      settings.SECRET_KEY, algorithm=settings.JWT_ALGORITHM)


def decode_token(token: str) -> dict:
    return jwt.decode(token, settings.SECRET_KEY, algorithms=[settings.JWT_ALGORITHM])


def hash_api_key(key: str) -> str:
    return hashlib.sha256(key.encode()).hexdigest()
```

backend/app/db/__init__.py (1 строк)

```

```

backend/app/db/base.py (16 строк)

```
from datetime import date, datetime
from decimal import Decimal

from sqlalchemy import JSON, DateTime, func
from sqlalchemy.dialects.postgresql import JSONB
from sqlalchemy.orm import DeclarativeBase, mapped_column

JSONType = JSON().with_variant(JSONB(), "postgresql")


class Base(DeclarativeBase):
    pass


def created_col():
    return mapped_column(DateTime, server_default=func.now(), default=datetime.utcnow)
```

backend/app/db/models/__init__.py (8 строк)

```
from app.db.models.audit import Alert, AuditLog, Event, ProfileUpdate  # noqa
from app.db.models.cases import Case, Collateral, Debtor  # noqa
from app.db.models.creditors import Claim, ClaimDocument, Creditor  # noqa
from app.db.models.documents import Document, ExtractedFact  # noqa
from app.db.models.forecasts import CreditorProfile, Priority, Scenario  # noqa
from app.db.models.reference import AU, AUProfile, Court, DisqualifiedAU, Judge, JudgeStats, Legislation
# noqa
from app.db.models.registry import Calculation, Mass, Registry  # noqa
from app.db.models.users import ApiKey, EmailLog, Payment, Subscription, User, WebhookSubscription  #
noqa
```

backend/app/db/models/audit.py (51 строк)

```
"""Слой 6. Аудит."""
from datetime import datetime

from sqlalchemy import Boolean, DateTime, ForeignKey, Integer, String, Text
from sqlalchemy.orm import Mapped, mapped_column

from app.db.base import Base, JSONType, created_col


class Event(Base):
    __tablename__ = "events"
    id: Mapped[int] = mapped_column(primary_key=True)
    case_id: Mapped[int | None] = mapped_column(ForeignKey("cases.id"))
    event_type: Mapped[str | None] = mapped_column(String(50))
    source: Mapped[str | None] = mapped_column(String(50))
    event_date: Mapped[datetime | None] = mapped_column(DateTime, default=datetime.utcnow)
    payload: Mapped[dict | None] = mapped_column(JSONType)
    relevance: Mapped[bool] = mapped_column(Boolean, default=True)


class ProfileUpdate(Base):
    __tablename__ = "profile_updates"
    id: Mapped[int] = mapped_column(primary_key=True)
    profile_id: Mapped[int | None] = mapped_column(ForeignKey("creditor_profiles.id"))
    trigger: Mapped[str | None] = mapped_column(String(100))
    old_value: Mapped[dict | None] = mapped_column(JSONType)
    new_value: Mapped[dict | None] = mapped_column(JSONType)
    version: Mapped[int | None] = mapped_column(Integer)
    created_at = created_col()


class Alert(Base):
```

```
    __tablename__ = "alerts"
    id: Mapped[int] = mapped_column(primary_key=True)
    claim_id: Mapped[int | None] = mapped_column(ForeignKey("claims.id"))
    alert_type: Mapped[str | None] = mapped_column(String(30))
    channel: Mapped[str | None] = mapped_column(String(20))
    content: Mapped[str | None] = mapped_column(Text)
    status: Mapped[str | None] = mapped_column(String(20))
    created_at = created_col()


class AuditLog(Base):
    __tablename__ = "audit_log"
    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int | None] = mapped_column(Integer)
    action: Mapped[str | None] = mapped_column(String(100))
    object_type: Mapped[str | None] = mapped_column(String(50))
    object_id: Mapped[int | None] = mapped_column(Integer)
    ip_address: Mapped[str | None] = mapped_column(String(45))
    created_at = created_col()
```

backend/app/db/models/cases.py (54 строк)

```
"""Слой 2. Дела."""
from datetime import date
from decimal import Decimal

from sqlalchemy import Boolean, Date, ForeignKey, Numeric, String
from sqlalchemy.orm import Mapped, mapped_column

from app.db.base import Base, created_col


class Debtor(Base):
    __tablename__ = "debtors"
    id: Mapped[int] = mapped_column(primary_key=True)
    inn: Mapped[str | None] = mapped_column(String(12), index=True)
    ogrn: Mapped[str | None] = mapped_column(String(15))
    type: Mapped[str | None] = mapped_column(String(20))
    name: Mapped[str | None] = mapped_column(String(500))
    industry: Mapped[str | None] = mapped_column(String(100))
    region: Mapped[str | None] = mapped_column(String(100))
    total_assets: Mapped[Decimal | None] = mapped_column(Numeric(15, 2))
    current_expenses: Mapped[Decimal | None] = mapped_column(Numeric(15, 2))
    first_priority: Mapped[Decimal | None] = mapped_column(Numeric(15, 2))
    second_priority: Mapped[Decimal | None] = mapped_column(Numeric(15, 2))
    executive_fee: Mapped[Decimal | None] = mapped_column(Numeric(15, 2))
    has_property: Mapped[bool | None] = mapped_column(Boolean)
    created_at = created_col()


class Case(Base):
    __tablename__ = "cases"
    id: Mapped[int] = mapped_column(primary_key=True)
    case_number: Mapped[str] = mapped_column(String(50), unique=True)
    court_id: Mapped[int | None] = mapped_column(ForeignKey("courts.id"))
    judge_id: Mapped[int | None] = mapped_column(ForeignKey("judges.id"))
    au_id: Mapped[int | None] = mapped_column(ForeignKey("au.id"))
    debtor_id: Mapped[int | None] = mapped_column(ForeignKey("debtors.id"))
    filing_date: Mapped[date | None] = mapped_column(Date)
    procedure_start_date: Mapped[date | None] = mapped_column(Date)
    current_stage: Mapped[str | None] = mapped_column(String(10))
    status: Mapped[str | None] = mapped_column(String(50))
    created_at = created_col()
    updated_at = created_col()


class Collateral(Base):
```

```
    __tablename__ = "collateral"
    id: Mapped[int] = mapped_column(primary_key=True)
    debtor_id: Mapped[int | None] = mapped_column(ForeignKey("debtors.id"))
    asset_type: Mapped[str | None] = mapped_column(String(100))
    market_value: Mapped[Decimal | None] = mapped_column(Numeric(15, 2))
    liquidity: Mapped[Decimal | None] = mapped_column(Numeric(3, 2))
    pledgee: Mapped[str | None] = mapped_column(String(255))
    registered: Mapped[bool] = mapped_column(Boolean, default=False)
    created_at = created_col()
```

backend/app/db/models/creditors.py (43 строк)

```
"""Слой 3. Кредиторы."""
from datetime import date
from decimal import Decimal

from sqlalchemy import Boolean, Date, ForeignKey, Numeric, String
from sqlalchemy.orm import Mapped, mapped_column

from app.db.base import Base, JSONType, created_col


class Creditor(Base):
    __tablename__ = "creditors"
    id: Mapped[int] = mapped_column(primary_key=True)
    inn: Mapped[str | None] = mapped_column(String(12))
    name: Mapped[str | None] = mapped_column(String(500))
    connection_type: Mapped[str | None] = mapped_column(String(30))
    created_at = created_col()


class Claim(Base):
    __tablename__ = "claims"
    id: Mapped[int] = mapped_column(primary_key=True)
    case_id: Mapped[int | None] = mapped_column(ForeignKey("cases.id"))
    creditor_id: Mapped[int | None] = mapped_column(ForeignKey("creditors.id"))
    amount: Mapped[Decimal | None] = mapped_column(Numeric(15, 2))
    claim_date: Mapped[date | None] = mapped_column(Date)
    construction_type: Mapped[str | None] = mapped_column(String(50))
    is_credit_contract: Mapped[bool] = mapped_column(Boolean, default=False)
    collateral_id: Mapped[int | None] = mapped_column(ForeignKey("collateral.id"))
    status: Mapped[str | None] = mapped_column(String(50))
    execution_date: Mapped[date | None] = mapped_column(Date)
    created_at = created_col()


class ClaimDocument(Base):
    __tablename__ = "claim_documents"
    id: Mapped[int] = mapped_column(primary_key=True)
    claim_id: Mapped[int | None] = mapped_column(ForeignKey("claims.id"))
    document_type: Mapped[str | None] = mapped_column(String(50))
    file_path: Mapped[str | None] = mapped_column(String(500))
    extracted_data: Mapped[dict | None] = mapped_column(JSONType)
    processing_status: Mapped[str | None] = mapped_column(String(30))
    created_at = created_col()
```

backend/app/db/models/documents.py (30 строк)

```
"""Слой 7. Документы."""
from decimal import Decimal

from sqlalchemy import Boolean, ForeignKey, Integer, Numeric, String, Text
from sqlalchemy.orm import Mapped, mapped_column

from app.db.base import Base, created_col

```

```python

class Document(Base):
    __tablename__ = "documents"
    id: Mapped[int] = mapped_column(primary_key=True)
    claim_id: Mapped[int | None] = mapped_column(ForeignKey("claims.id"))
    document_type: Mapped[str | None] = mapped_column(String(50))
    file_path: Mapped[str | None] = mapped_column(String(500))
    file_size: Mapped[int | None] = mapped_column(Integer)
    mime_type: Mapped[str | None] = mapped_column(String(100))
    processing_status: Mapped[str | None] = mapped_column(String(30))
    created_at = created_col()


class ExtractedFact(Base):
    __tablename__ = "extracted_facts"
    id: Mapped[int] = mapped_column(primary_key=True)
    document_id: Mapped[int] = mapped_column(ForeignKey("documents.id"))
    field_name: Mapped[str | None] = mapped_column(String(100))
    field_value: Mapped[str | None] = mapped_column(Text)
    confidence: Mapped[Decimal | None] = mapped_column(Numeric(3, 2))
    confirmed: Mapped[bool] = mapped_column(Boolean, default=False)
    created_at = created_col()
```

backend/app/db/models/forecasts.py (52 строк)

```python
"""Слой 5. Прогнозы."""
from decimal import Decimal

from sqlalchemy import ForeignKey, Integer, Numeric, String, Text
from sqlalchemy.orm import Mapped, mapped_column

from app.db.base import Base, JSONType, created_col


class CreditorProfile(Base):
    __tablename__ = "creditor_profiles"
    id: Mapped[int] = mapped_column(primary_key=True)
    claim_id: Mapped[int | None] = mapped_column(ForeignKey("claims.id"))
    user_id: Mapped[int | None] = mapped_column(Integer, index=True)
    p_sub: Mapped[Decimal | None] = mapped_column(Numeric(5, 4))
    p_ch: Mapped[Decimal | None] = mapped_column(Numeric(5, 4))
    p_zero: Mapped[Decimal | None] = mapped_column(Numeric(5, 4))
    delta_practice: Mapped[Decimal | None] = mapped_column(Numeric(5, 4))
    delta_au: Mapped[Decimal | None] = mapped_column(Numeric(5, 4))
    expected_amount: Mapped[Decimal | None] = mapped_column(Numeric(15, 2))
    expected_pct: Mapped[Decimal | None] = mapped_column(Numeric(5, 4))
    range_low: Mapped[Decimal | None] = mapped_column(Numeric(15, 2))
    range_high: Mapped[Decimal | None] = mapped_column(Numeric(15, 2))
    confidence: Mapped[str | None] = mapped_column(String(20))
    mode: Mapped[str | None] = mapped_column(String(10))
    information_levels: Mapped[dict | None] = mapped_column(JSONType)
    flags: Mapped[dict | None] = mapped_column(JSONType)
    report: Mapped[dict | None] = mapped_column(JSONType)   # полный отчёт расчёта (для details/audit)
    version: Mapped[int] = mapped_column(Integer, default=1)
    calculated_at = created_col()


class Scenario(Base):
    __tablename__ = "scenarios"
    id: Mapped[int] = mapped_column(primary_key=True)
    profile_id: Mapped[int] = mapped_column(ForeignKey("creditor_profiles.id"))
    name: Mapped[str | None] = mapped_column(String(30))
    probability: Mapped[Decimal | None] = mapped_column(Numeric(5, 4))
    payout_rub: Mapped[Decimal | None] = mapped_column(Numeric(15, 2))
    payout_pct: Mapped[Decimal | None] = mapped_column(Numeric(5, 4))
    description: Mapped[str | None] = mapped_column(Text)

```

```

class Priority(Base):
    __tablename__ = "priority"
    id: Mapped[int] = mapped_column(primary_key=True)
    profile_id: Mapped[int] = mapped_column(ForeignKey("creditor_profiles.id"))
    priority_1: Mapped[str | None] = mapped_column(String(100))
    priority_2: Mapped[str | None] = mapped_column(String(100))
    deferred: Mapped[str | None] = mapped_column(String(200))
    trigger: Mapped[str | None] = mapped_column(String(100))
    updated_at = created_col()
```

backend/app/db/models/reference.py (81 строк)

```
"""Слой 1. Справочники."""
from datetime import date, datetime
from decimal import Decimal

from sqlalchemy import Boolean, Date, DateTime, ForeignKey, Integer, Numeric, String, Text
from sqlalchemy.orm import Mapped, mapped_column

from app.db.base import Base, created_col


class Court(Base):
    __tablename__ = "courts"
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(255))
    region: Mapped[str | None] = mapped_column(String(100))
    created_at = created_col()


class Judge(Base):
    __tablename__ = "judges"
    id: Mapped[int] = mapped_column(primary_key=True)
    court_id: Mapped[int | None] = mapped_column(ForeignKey("courts.id"))
    full_name: Mapped[str] = mapped_column(String(255))
    created_at = created_col()


class JudgeStats(Base):
    __tablename__ = "judge_stats"
    id: Mapped[int] = mapped_column(primary_key=True)
    judge_id: Mapped[int] = mapped_column(ForeignKey("judges.id"))
    category: Mapped[str | None] = mapped_column(String(50))
    n_cases: Mapped[int] = mapped_column(Integer, default=0)
    approval_rate: Mapped[Decimal | None] = mapped_column(Numeric(5, 4))
    style: Mapped[str | None] = mapped_column(String(20))
    avg_duration_days: Mapped[int | None] = mapped_column(Integer)
    confidence_interval: Mapped[Decimal | None] = mapped_column(Numeric(5, 4))
    is_significant: Mapped[bool] = mapped_column(Boolean, default=False)
    version: Mapped[int] = mapped_column(Integer, default=1)
    updated_at = created_col()


class AU(Base):
    __tablename__ = "au"
    id: Mapped[int] = mapped_column(primary_key=True)
    inn: Mapped[str | None] = mapped_column(String(12), unique=True)
    full_name: Mapped[str | None] = mapped_column(String(255))
    sro: Mapped[str | None] = mapped_column(String(255))
    rating_rau: Mapped[int | None] = mapped_column(Integer)
    created_at = created_col()


class AUProfile(Base):
    __tablename__ = "au_profiles"
    id: Mapped[int] = mapped_column(primary_key=True)
```

```
    au_id: Mapped[int] = mapped_column(ForeignKey("au.id"))
    n_cases: Mapped[int] = mapped_column(Integer, default=0)
    challenging_activity: Mapped[Decimal | None] = mapped_column(Numeric(5, 4))
    challenging_success: Mapped[Decimal | None] = mapped_column(Numeric(5, 4))
    losses_recovered: Mapped[int] = mapped_column(Integer, default=0)
    avg_discount: Mapped[Decimal | None] = mapped_column(Numeric(5, 4))
    version: Mapped[int] = mapped_column(Integer, default=1)
    updated_at = created_col()


class DisqualifiedAU(Base):
    __tablename__ = "disqualified_au"
    id: Mapped[int] = mapped_column(primary_key=True)
    inn: Mapped[str | None] = mapped_column(String(12))
    full_name: Mapped[str | None] = mapped_column(String(255))
    disqualification_date: Mapped[date | None] = mapped_column(Date)
    reason: Mapped[str | None] = mapped_column(Text)
    source: Mapped[str | None] = mapped_column(String(255))


class Legislation(Base):
    __tablename__ = "legislation"
    id: Mapped[int] = mapped_column(primary_key=True)
    act_name: Mapped[str | None] = mapped_column(String(500))
    act_date: Mapped[date | None] = mapped_column(Date)
    category: Mapped[str | None] = mapped_column(String(50))
    url: Mapped[str | None] = mapped_column(Text)
```

backend/app/db/models/registry.py (49 строк)

```
"""Слой 4. Реестр и расчёты."""
from datetime import date
from decimal import Decimal

from sqlalchemy import Boolean, Date, ForeignKey, Integer, Numeric, String
from sqlalchemy.orm import Mapped, mapped_column

from app.db.base import Base, JSONType, created_col


class Registry(Base):
    __tablename__ = "registry"
    id: Mapped[int] = mapped_column(primary_key=True)
    case_id: Mapped[int | None] = mapped_column(ForeignKey("cases.id"))
    claim_id: Mapped[int | None] = mapped_column(ForeignKey("claims.id"))
    included_amount: Mapped[Decimal | None] = mapped_column(Numeric(15, 2))
    queue: Mapped[int | None] = mapped_column(Integer)
    is_subordinated: Mapped[bool] = mapped_column(Boolean, default=False)
    inclusion_date: Mapped[date | None] = mapped_column(Date)


class Mass(Base):
    __tablename__ = "mass"
    id: Mapped[int] = mapped_column(primary_key=True)
    case_id: Mapped[int | None] = mapped_column(ForeignKey("cases.id"))
    total_assets: Mapped[Decimal | None] = mapped_column(Numeric(15, 2))
    current_expenses: Mapped[Decimal | None] = mapped_column(Numeric(15, 2))
    first_priority: Mapped[Decimal | None] = mapped_column(Numeric(15, 2))
    second_priority: Mapped[Decimal | None] = mapped_column(Numeric(15, 2))
    pledge_deductions: Mapped[Decimal | None] = mapped_column(Numeric(15, 2))
    executive_fee: Mapped[Decimal | None] = mapped_column(Numeric(15, 2))
    net_mass: Mapped[Decimal | None] = mapped_column(Numeric(15, 2))
    p_zero: Mapped[Decimal | None] = mapped_column(Numeric(5, 4))
    version: Mapped[int] = mapped_column(Integer, default=1)
    calculated_at = created_col()


```

```python
class Calculation(Base):
    __tablename__ = "calculations"
    id: Mapped[int] = mapped_column(primary_key=True)
    claim_id: Mapped[int | None] = mapped_column(ForeignKey("claims.id"))
    mode: Mapped[str | None] = mapped_column(String(20))
    scenario: Mapped[str | None] = mapped_column(String(30))
    formula: Mapped[str | None] = mapped_column(String(60))
    result_rub: Mapped[Decimal | None] = mapped_column(Numeric(15, 2))
    result_pct: Mapped[Decimal | None] = mapped_column(Numeric(5, 4))
    inputs: Mapped[dict | None] = mapped_column(JSONType)
    version: Mapped[int] = mapped_column(Integer, default=1)
    calculated_at = created_col()
```

backend/app/db/models/users.py (75 строк)

```python
"""Слой 8. Пользователи и монетизация (готовность к версии 2.0)."""
from datetime import datetime
from decimal import Decimal

from sqlalchemy import DateTime, ForeignKey, Integer, Numeric, String, Text
from sqlalchemy.orm import Mapped, mapped_column

from app.db.base import Base, JSONType, created_col


class User(Base):
    __tablename__ = "users"
    id: Mapped[int] = mapped_column(primary_key=True)
    email: Mapped[str] = mapped_column(String(255), unique=True)
    password_hash: Mapped[str | None] = mapped_column(String(255))
    role: Mapped[str | None] = mapped_column(String(20))
    tier: Mapped[str | None] = mapped_column(String(20))
    subscription_until: Mapped[datetime | None] = mapped_column(DateTime)
    free_calculations_left: Mapped[int] = mapped_column(Integer, default=3)
    created_at = created_col()


class Subscription(Base):
    __tablename__ = "subscriptions"
    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"))
    plan: Mapped[str | None] = mapped_column(String(50))
    status: Mapped[str | None] = mapped_column(String(20))
    started_at: Mapped[datetime | None] = mapped_column(DateTime)
    expires_at: Mapped[datetime | None] = mapped_column(DateTime)
    payment_provider: Mapped[str | None] = mapped_column(String(50))
    payment_id: Mapped[str | None] = mapped_column(String(255))


class Payment(Base):
    __tablename__ = "payments"
    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"))
    amount: Mapped[Decimal | None] = mapped_column(Numeric(10, 2))
    currency: Mapped[str] = mapped_column(String(3), default="RUB")
    status: Mapped[str | None] = mapped_column(String(20))
    provider: Mapped[str | None] = mapped_column(String(50))
    provider_payment_id: Mapped[str | None] = mapped_column(String(255))
    created_at = created_col()


class ApiKey(Base):
    __tablename__ = "api_keys"
    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"))
    key_hash: Mapped[str | None] = mapped_column(String(255))
    name: Mapped[str | None] = mapped_column(String(100))
```

```
    created_at = created_col()
    last_used_at: Mapped[datetime | None] = mapped_column(DateTime)
    revoked_at: Mapped[datetime | None] = mapped_column(DateTime)


class WebhookSubscription(Base):
    __tablename__ = "webhook_subscriptions"
    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"))
    url: Mapped[str | None] = mapped_column(String(500))
    events: Mapped[list | None] = mapped_column(JSONType)
    secret: Mapped[str | None] = mapped_column(String(255))
    created_at = created_col()


class EmailLog(Base):
    __tablename__ = "email_log"
    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int | None] = mapped_column(Integer)
    template: Mapped[str | None] = mapped_column(String(100))
    status: Mapped[str | None] = mapped_column(String(20))
    sent_at: Mapped[datetime | None] = mapped_column(DateTime)
    error: Mapped[str | None] = mapped_column(Text)
```

backend/app/db/repositories/__init__.py (1 строк)

```

```

backend/app/db/seed.py (17 строк)

```
"""Начальные данные v1.0: администратор admin@local / admin."""
from sqlalchemy import select

from app.core.config import settings
from app.core.security import hash_password
from app.db.models import User


async def seed_admin(session) -> User:
    res = await session.execute(select(User).where(User.email == "admin@local"))
    user = res.scalar_one_or_none()
    if user is None:
        user = User(email="admin@local", password_hash=hash_password(settings.ADMIN_PASSWORD),
role="admin",
                    tier="admin", free_calculations_left=9999)
        session.add(user)
        await session.commit()
    return user
```

backend/app/db/session.py (14 строк)

```
from sqlalchemy.pool import NullPool
from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker, create_async_engine

from app.core.config import settings

# В тестах новый event loop на каждый тест — пул соединений не переиспользуем
_kw = {"poolclass": NullPool} if settings.ENVIRONMENT == "test" else {"pool_pre_ping": True}
engine = create_async_engine(settings.DATABASE_URL, **_kw)
SessionLocal = async_sessionmaker(engine, expire_on_commit=False)


async def get_db():
    async with SessionLocal() as session:
        yield session
```

backend/app/main.py (71 строк)

```
"""Точка входа FastAPI «Профиль кредитора»."""
from contextlib import asynccontextmanager

from fastapi import FastAPI

from app.api.health import router as health_router
from app.api.v1 import api_router
from app.core.config import settings
from app.core.logging import configure_logging, get_logger
from app.core.middleware.cors import add_cors
from app.core.middleware.errors import register_error_handlers
from app.core.middleware.logging import LoggingMiddleware
from app.core.middleware.rate_limit import RateLimitMiddleware
from app.core.middleware.request_id import RequestIDMiddleware
from app.db.base import Base
from app.db.seed import seed_admin
from app.db.session import SessionLocal, engine

configure_logging()
log = get_logger("main")


@asynccontextmanager
async def lifespan(app: FastAPI):
    if settings.ENVIRONMENT in ("local", "test"):
        # Для локального запуска без alembic; в prod схему создаёт `alembic upgrade head`
        import app.db.models  # noqa: F401
        async with engine.begin() as conn:
            await conn.run_sync(Base.metadata.create_all)
    async with SessionLocal() as s:
        await seed_admin(s)
    if settings.PUBLIC_MODE and (settings.SECRET_KEY == "change-me-in-production" or
settings.ADMIN_PASSWORD == "admin"):
        log.warning("insecure_public_config", hint="Задайте SECRET_KEY и ADMIN_PASSWORD")
    log.info("startup", environment=settings.ENVIRONMENT, public_mode=settings.PUBLIC_MODE,
             parsers=settings.PARSERS_MODE)
    yield


app = FastAPI(title="Профиль кредитора", version="1.0", lifespan=lifespan)
register_error_handlers(app)
# порядок: последний добавленный выполняется первым
add_cors(app)
app.add_middleware(RateLimitMiddleware)
app.add_middleware(LoggingMiddleware)
app.add_middleware(RequestIDMiddleware)
app.include_router(health_router)
app.include_router(api_router)

if settings.METRICS_ENABLED:
    from prometheus_fastapi_instrumentator import Instrumentator
    Instrumentator().instrument(app).expose(app, endpoint="/metrics", include_in_schema=False)


if settings.FRONTEND_DIST:
    # Автономный режим: backend раздаёт собранный фронтенд (SPA с fallback на index.html)
    from pathlib import Path

    from fastapi import HTTPException
    from fastapi.responses import FileResponse

    _dist = Path(settings.FRONTEND_DIST).resolve()
    _API = ("api/", "docs", "openapi.json", "health", "ready", "metrics")

    @app.get("/{path:path}", include_in_schema=False)
    async def spa(path: str):
```

```
        if path.startswith(_API):
            raise HTTPException(404, "Not found")
        f = (_dist / path).resolve()
        if path and f.is_file() and _dist in f.parents:
            return FileResponse(f)
        return FileResponse(_dist / "index.html")
```

backend/app/services/__init__.py (1 строк)

```

```

backend/app/services/analysis.py (335 строк)

```
"""Пайплайн анализа: источники → расчёт → сохранение профиля, сценариев, приоритетов, аудита."""
from datetime import date, datetime, timedelta
from decimal import Decimal

from sqlalchemy import select

from app.core.exceptions import NotFound
from app.db.models import (AU, Calculation, Case, Claim, Court, Creditor, CreditorProfile, Debtor,
Document,
                           ExtractedFact, Judge, Mass, Priority, Scenario)
from app.services.analytics import track
from app.services.calculator import CalcInput, calculate
from app.services.document_processor import levels_from_documents
from app.services.mode_detector import mode_from_bundle
from app.services.parsers import fetch_case_bundle

DEMO_AMOUNT = 10_000_000.0

DISCLAIMERS = {
    "general": "Сервис носит информационно-аналитический характер. Не является юридической услугой. "
               "Прогнозы вероятностны и не предсказывают исход конкретного дела. Не заменяет консультацию
юриста.",
    "practice": "Построен на опубликованных судебных актах. Не является оценкой личности судьи "
                "или рекомендацией воздействия на суд.",
    "au": "Баллы отражают публичные факты и являются аналитической гипотезой, а не утверждением о
нарушении. "
          "Решение об отстранении принимает только суд.",
    "pd": "Обрабатываются только общедоступные персональные данные (ст. 8 152-ФЗ). "
          "Вы вправе потребовать уточнения или удаления (ст. 14).",
    "fee": "Размер и очерёдность исполнительского сбора зависят от статуса основного долга. "
           "Расчёт носит ориентировочный характер.",
    "criminal": "Система выявляет публичные признаки рисков, но не квалифицирует действия как
преступление. "
                "Рекомендуется консультация профильного юриста.",
    "ads": "Сервис соблюдает требования ст. 28.1 Закона «О рекламе». Материалы содержат обязательные "
           "указания о негативных последствиях банкротства.",
}


def _d(v) -> date | None:
    if not v:
        return None
    if isinstance(v, date):
        return v
    for fmt in ("%Y-%m-%d", "%d.%m.%Y"):
        try:
            return datetime.strptime(str(v), fmt).date()
        except ValueError:
            continue
    return None


async def _get_or_create(session, model, defaults=None, **filters):
```

```
    res = await session.execute(select(model).filter_by(**filters))
    obj = res.scalars().first()
    if obj is None:
        obj = model(**filters, **(defaults or {}))
        session.add(obj)
        await session.flush()
    return obj


def build_input(bundle: dict, claim: dict, mode: str, doc_flags: dict, facts_confirmed: bool) -> tuple[CalcInput, list[str]]:
    notes: list[str] = []
    debtor = bundle.get("debtor") or {}
    filing = _d(bundle.get("filing_date"))
    if filing is None:
        filing = date.today()
        notes.append("Дата принятия заявления неизвестна: для оценки окон оспаривания принята сегодняшняя дата.")
    amount = claim.get("amount")
    if not amount:
        amount = DEMO_AMOUNT
        notes.append(f"Сумма требования не указана: использовано демонстрационное значение {DEMO_AMOUNT:,.0f} ₽.".replace(",", " "))
    judge = bundle.get("judge")
    au = bundle.get("au")
    registry = bundle.get("registry") or {}
    inp = CalcInput(
        mode=mode, amount=float(amount), construction_type=claim.get("construction_type") or "supply",
        is_credit_contract=bool(claim.get("is_credit_contract")),
        claim_date=_d(claim.get("claim_date")), filing_date=filing,
        execution_date=_d(claim.get("execution_date")),
        executed_amount=float(claim.get("executed_amount") or 0),
        status=claim.get("status") or "independent", crisis=bool(claim.get("crisis")),
        rescue_form=bool(claim.get("rescue_form")), unusual_conditions=bool(claim.get("unusual_conditions")),
        exceptions=claim.get("exceptions") or {}, beneficial_interest=bool(claim.get("beneficial_interest")),
        only_covenants=bool(claim.get("only_covenants")),
        is_equivalent=claim.get("is_equivalent", True) is not False,
        has_harm_intent=bool(claim.get("has_harm_intent")), is_gratuitous=bool(claim.get("is_gratuitous")),
        is_interested=bool(claim.get("is_interested")),
        debtor_type=debtor.get("type") or "legal", industry=debtor.get("industry"),
        total_assets=debtor.get("total_assets") if "total_assets" in debtor else None,
        current_expenses=float(debtor.get("current_expenses") or 0),
        first_priority=float(debtor.get("first_priority") or 0),
        second_priority=float(debtor.get("second_priority") or 0),
        collateral=[{"C": c["C"], "L": c["L"]} for c in bundle.get("collateral", [])],
        sigma3=registry.get("sigma3"), sigma_sub=float(registry.get("sigma_sub") or 0),
        judge_stats=judge, court_avg=bundle.get("court_avg", 0.43), au_stats=au,
        au_signals=bundle.get("au_signals") or {}, data_flags=doc_flags, facts_confirmed=facts_confirmed,
    )
    # сделка, по которой есть исполнение, но не указана сумма — берём сумму исполнения
    if inp.execution_date and not inp.executed_amount:
        notes.append("Дата исполнения указана без суммы исполнения: окно оспаривания оценено, сумма принята 0.")
    return inp, notes


def apply_manual(bundle: dict, manual: dict) -> dict:
    """Ручной ввод данных дела перекрывает данные источников (для partial/live и для точной настройки)."""
    if not manual:
        return bundle
    b = dict(bundle)
    b["found"] = True if any(manual.get(k) is not None for k in
                             ("filing_date", "total_assets", "sigma3", "judge", "au")) else b.get("found")
    deb = dict(b.get("debtor") or {})
```

```
    for k in ("total_assets", "current_expenses", "first_priority", "second_priority", "industry",
"type"):
        if manual.get(k) is not None:
            deb[k] = manual[k]
    b["debtor"] = deb
    reg = dict(b.get("registry") or {})
    for k in ("sigma3", "sigma_sub"):
        if manual.get(k) is not None:
            reg[k] = manual[k]
    b["registry"] = reg
    for k in ("filing_date", "judge", "au", "au_signals", "court_avg", "case_number"):
        if manual.get(k) is not None:
            b[k] = manual[k]
    if manual.get("collateral") is not None:
        b["collateral"] = manual["collateral"]
    b.setdefault("messages", [])
    b["status"] = "ok"
    b["manual_input_required"] = False
    b["source"] = "manual" if b.get("source") != "mock" else "mock+manual"
    b["warnings"] = [w for w in b.get("warnings", []) if "mock" not in w] + \
        ["Часть данных введена вручную и не проверена системой."]
    return b


async def run_analysis(session, user, *, inn=None, case_number=None, claim: dict | None = None,
                       manual: dict | None = None,
                       mode_override: str | None = None, base_profile: CreditorProfile | None = None) ->
CreditorProfile:
    claim = claim or {}
    manual = manual or {}
    bundle = apply_manual(await fetch_case_bundle(inn=inn, case_number=case_number), manual)
    auto = mode_from_bundle(bundle)
    mode = mode_override or auto["mode"]

    # --- документы: подтверждённые факты поднимают уровни ---
    doc_flags, facts_confirmed = {}, True
    claim_id_existing = base_profile.claim_id if base_profile else None
    if claim_id_existing:
        docs = (await session.execute(select(Document).where(Document.claim_id ==
claim_id_existing))).scalars().all()
        if docs:
            confirmed = True
            for d in docs:
                facts = (await session.execute(select(ExtractedFact).where(ExtractedFact.document_id ==
d.id))).scalars().all()
                if facts and not all(f.confirmed for f in facts):
                    confirmed = False
            facts_confirmed = confirmed
            doc_flags = levels_from_documents({d.document_type for d in docs}, confirmed)

    inp, notes = build_input(bundle, claim, mode, doc_flags, facts_confirmed)
    result = calculate(inp)
    result["assumptions"] = notes + result["assumptions"]
    result["warnings"] = list(bundle.get("warnings", [])) + result["warnings"]
    result["auto_mode"] = auto
    result["mode_manual"] = bool(mode_override)
    result["bundle_source"] = bundle.get("source")
    result["status"] = bundle.get("status", "ok")
    result["manual_input_required"] = bool(bundle.get("manual_input_required"))
    result["case"] = {"number": bundle.get("case_number") or case_number, "court": bundle.get("court"),
                      "filing_date": bundle.get("filing_date"), "debtor": (bundle.get("debtor") or
{}).get("name"),
                      "inn": inn}
    result["judge"] = ({"name": bundle["judge"].get("name"), **bundle["judge"]} if bundle.get("judge")
else None)
    result["au"] = ({"name": bundle["au"].get("name"), "sro": bundle["au"].get("sro"), **bundle["au"]}
                    if bundle.get("au") else None)
    result["registry_rows"] = build_registry_view(result, bundle)
    result["checklist"], result["calendar"] = _checklist_calendar(result, bundle)
```

```
    result["_request"] = {"inn": inn, "case_number": case_number, "claim": claim, "manual": manual}

    # --- сохранение ---
    case_id = None
    if bundle.get("found"):
        court = await _get_or_create(session, Court, name=bundle.get("court") or "Не указан",
                                     defaults={"region": bundle.get("region")})
        jid = aid = None
        if bundle.get("judge"):
            jid = (await _get_or_create(session, Judge, full_name=bundle["judge"]["name"],
court_id=court.id)).id
        if bundle.get("au"):
            aid = (await _get_or_create(session, AU, full_name=bundle["au"]["name"],
                                        defaults={"sro": bundle["au"].get("sro")})).id
        deb = bundle.get("debtor") or {}
        debtor = await _get_or_create(session, Debtor, inn=inn or deb.get("inn"),
                                      defaults={"name": deb.get("name"), "type": deb.get("type"),
                                                "industry": deb.get("industry"),
                                                "total_assets": deb.get("total_assets")})
        cn = bundle.get("case_number") or f"manual-{inn}"
        case = await _get_or_create(session, Case, case_number=cn,
                                    defaults={"court_id": court.id, "judge_id": jid, "au_id": aid,
                                              "debtor_id": debtor.id, "filing_date":
_d(bundle.get("filing_date"))})
        case.current_stage = mode
        case_id = case.id
    if base_profile and base_profile.claim_id:
        claim_row = await session.get(Claim, base_profile.claim_id)
    else:
        creditor = await _get_or_create(session, Creditor, inn=f"user-{user.id}", defaults={"name":
user.email})
        claim_row = Claim(case_id=case_id, creditor_id=creditor.id, amount=Decimal(str(inp.amount)),
                          claim_date=inp.claim_date, construction_type=inp.construction_type,
                          is_credit_contract=inp.is_credit_contract, status=result["queue"],
                          execution_date=inp.execution_date)
        session.add(claim_row)
        await session.flush()

    result["claim_id"], result["case_id"] = claim_row.id, case_id or claim_row.case_id
    version = (base_profile.version + 1) if base_profile else 1
    profile = CreditorProfile(
        claim_id=claim_row.id, user_id=user.id, p_sub=result["p_sub"], p_ch=result["p_ch"],
p_zero=result["p_zero"],
        delta_practice=result["m4"]["delta_ch"], delta_au=result["m4"]["delta_au_ch"],
        expected_amount=result["expected_amount"], expected_pct=result["expected_pct"],
        range_low=result["range_low"], range_high=result["range_high"], confidence=result["confidence"],
        mode=mode, information_levels=result["information_levels"],
        flags={"low_data": result["confidence"] == "low", "au_critical": result["au_index"]["critical"],
               "mode_manual": result["mode_manual"]},
        report=result, version=version)
    session.add(profile)
    await session.flush()
    for s in result["scenarios"]:
        session.add(Scenario(profile_id=profile.id, name=s["name"], probability=s["probability"],
                             payout_rub=s["payout_rub"], payout_pct=s["payout_pct"],
description=s["title"]))
        session.add(Calculation(claim_id=claim_row.id, mode=mode, scenario=s["name"],
formula=result["formula"],
                                result_rub=s["payout_rub"], result_pct=s["payout_pct"],
                                inputs={"p_sub": result["p_sub"], "p_ch": result["p_ch"], "p_zero":
result["p_zero"]},
                                version=version))
    pr = result["priorities"]
    session.add(Priority(profile_id=profile.id, priority_1=(pr[0] if pr else None)[:100] if pr else None,
                         priority_2=pr[1][:100] if len(pr) > 1 else None,
                         deferred="; ".join(pr[2:])[:200] if len(pr) > 2 else None, trigger="analysis"))
    if case_id:
        session.add(Mass(case_id=case_id, total_assets=result["audit"]["M0"], net_mass=result["audit"]
["M"],
```

```
                          p_zero=result["p_zero"], version=version))
    await track(session, "analysis_run", case_id, "api", {"mode": mode, "profile_id": profile.id})
    await session.commit()
    return profile


def build_registry_view(result: dict, bundle: dict) -> list[dict]:
    """Таблица сравнения с другими кредиторами (P5–P6). Остальные строки — оценка по Σ3 (демо)."""
    if result["mode"] not in ("P5", "P6"):
        return []
    sigma3 = result["audit"]["sigma3"]
    S = result["claim_amount"]
    rest = max(0.0, sigma3 - S)
    shares = [0.4, 0.3, 0.2, 0.1]
    rows = [{"name": "Вы", "amount": S, "share": round(S / sigma3, 4) if sigma3 else 0, "p_ch":
result["p_ch"],
             "challenge_candidate": False, "own": True}]
    for i, sh in enumerate(shares):
        amt = rest * sh
        pch = round(0.30 + 0.15 * i, 2)
        share = amt / sigma3 if sigma3 else 0
        rows.append({"name": f"Кредитор {i + 1}", "amount": round(amt, 2), "share": round(share, 4),
"p_ch": pch,
                     "challenge_candidate": pch > 0.5 and (S / sigma3 if sigma3 else 0) > 0.10, "own":
False})
    return rows


def _checklist_calendar(result: dict, bundle: dict):
    checklist = ["Документы, подтверждающие основание требования (договор, акты, накладные)"]
    if result["p_ch"] >= 0.3:
        checklist += ["Платёжные документы по полученному исполнению", "Переписка о цели и условиях
сделки",
                      "Доказательства встречного предоставления", "Подтверждение рыночности условий"]
    if result["p_sub"] >= 0.2:
        checklist += ["Документы о независимости от должника", "Обоснование исключений (п. 2.3
методики)"]
    if result["queue"] == "secured":
        checklist.append("Договор залога, оценка, регистрация в реестре уведомлений")
    cal = []
    msgs = {m["type"]: _d(m.get("date")) for m in bundle.get("messages", []) if m.get("date")}
    if msgs.get("au_appointed"):
        cal.append({"date": (msgs["au_appointed"] + timedelta(days=30)).isoformat(),
                    "title": "Срок включения в реестр (30 дней) — ориентир"})
    if msgs.get("registry_closed"):
        cal.append({"date": (msgs["registry_closed"] + timedelta(days=10)).isoformat(),
                    "title": "Срок обжалования (10 дней) — ориентир"})
    if not cal:
        cal.append({"date": None, "title": "Следите за публикациями в ЕФРСБ: окно 15–30 дней"})
    return checklist, cal


async def get_profile(session, profile_id: int, user) -> CreditorProfile:
    p = await session.get(CreditorProfile, profile_id)
    if p is None or (p.user_id != user.id and user.role != "admin"):
        raise NotFound("Анализ не найден")
    return p


def summary_view(p: CreditorProfile) -> dict:
    r = p.report or {}
    top = next((a for a in r.get("actions", []) if a["key"] != "registry"), None) or (r.get("actions") or
[None])[0]
    return {
        "id": p.id, "version": p.version, "mode": p.mode, "case": r.get("case"),
        "claim_id": r.get("claim_id") or p.claim_id, "case_id": r.get("case_id"),
        "claim_amount": r.get("claim_amount"), "executed_amount": r.get("executed_amount"),
        "expected_amount": r.get("expected_amount"), "expected_pct": r.get("expected_pct"),
        "range_low_pct": r.get("range_low_pct"), "range_high_pct": r.get("range_high_pct"),
```

```
        "range_low": r.get("range_low"), "range_high": r.get("range_high"),
        "confidence": r.get("confidence"), "probabilistic_label": r.get("probabilistic_label"),
        "risks": r.get("risks"), "queue": r.get("queue"),
        "recommendation": ({"title": top["title"], "reason": top["reason"]} if top else None),
        "assumptions": r.get("assumptions", []), "warnings": r.get("warnings", []),
        "status": r.get("status"), "manual_input_required": r.get("manual_input_required", False),
        "mode_manual": r.get("mode_manual", False), "auto_mode": r.get("auto_mode"),
        "demo_data": r.get("bundle_source") == "mock",
        "manual_data": "manual" in (r.get("bundle_source") or ""),
        "disclaimers": {"general": DISCLAIMERS["general"], "ads": DISCLAIMERS["ads"]},
    }


def details_view(p: CreditorProfile) -> dict:
    r = p.report or {}
    return {
        "id": p.id, "scenarios": r.get("scenarios"), "actions": r.get("actions"), "priorities":
r.get("priorities"),
        "au": r.get("au"), "au_index": r.get("au_index"), "judge": r.get("judge"),
        "checklist": r.get("checklist"), "calendar": r.get("calendar"), "information_levels":
r.get("information_levels"),
        "m4": r.get("m4"),
        "disclaimers": {k: DISCLAIMERS[k] for k in ("general", "practice", "au", "fee", "criminal")},
    }


def audit_view(p: CreditorProfile) -> dict:
    r = p.report or {}
    j = (r.get("m4") or {}).get("judge", {})
    return {
        "id": p.id, "version": p.version, "formula": r.get("formula"), "method": r.get("method"),
        "formulas": (r.get("audit") or {}).get("formulas"), "audit": r.get("audit"),
        "modules": {k: r.get(k) for k in ("m0", "m1", "m2_1", "m2_2", "m3", "m4")},
        "probabilities": {"p_sub": r.get("p_sub"), "p_ch": r.get("p_ch"), "p_zero": r.get("p_zero"),
                          "delta_practice": float(p.delta_practice or 0), "delta_au": float(p.delta_au or
0)},
        "judge": r.get("judge"), "judge_delta": j, "au": r.get("au"), "au_index": r.get("au_index"),
        "information_levels": r.get("information_levels"), "assumptions": r.get("assumptions"),
        "sources": [{"title": "КАД Арбитр", "url": "https://kad.arbitr.ru"},
                    {"title": "ЕФРСБ", "url": "https://fedresurs.ru"}],
        "disclaimers": {k: DISCLAIMERS[k] for k in ("general", "practice", "au", "pd", "fee",
"criminal")},
    }
```

backend/app/services/analytics.py (10 строк)

```
"""Внутренняя аналитика: запись в таблицу events."""
from app.core.config import settings
from app.db.models import Event


async def track(session, event_type: str, case_id: int | None = None, source: str = "app",
                payload: dict | None = None) -> None:
    if not settings.ANALYTICS_ENABLED:
        return
    session.add(Event(case_id=case_id, event_type=event_type, source=source, payload=payload or {}))
```

backend/app/services/calculator.py (305 строк)

```
"""Оркестратор расчёта «Профиль кредитора»: M0 → M1 → M2.1 → M2.2 → M3 → M4 → M5 → решения."""
from __future__ import annotations

from dataclasses import dataclass, field
from datetime import date

from app.services.formulas import D, E_R, E_R_final, R_3, R_sec, R_sub, clamp
```

```
from app.services.level_detector import detect_levels
from app.services.modules.m0_filter import m0_filter
from app.services.modules.m1_queue import m1_queue
from app.services.modules.m2_1_subordination import m2_1_subordination
from app.services.modules.m2_2_challenging import m2_2_challenging
from app.services.modules.m3_economics import executive_fee, m3_economics
from app.services.modules.m4_practice import au_independence_score, m4_practice
from app.services.modules.m5_calc import (
    P_CH_INDUSTRY, P_SUB_INDUSTRY, P_ZERO_INDUSTRY, base_payout, exact_discounted, m5_probabilistic,
)

DEFENSE_SUCCESS = 0.5          # допущение о вероятности успешной защиты сделки
MIN_ECONOMIC_GAIN = 500_000.0  # порог экономической целесообразности (раздел 18 ТЗ)
DEFAULT_SIGMA3_FACTOR = 10.0   # если реестр неизвестен: Σ3 ≈ 10 × требование (консервативно)

ACTION_COSTS = {
    "file_claim":       {"title": "Подача заявления",  "fee": 6000, "months": "1–3", "lawyers": (50_000,
150_000)},
    "registry":         {"title": "Включение в реестр", "fee": 0,    "months": "1–2", "lawyers": (30_000,
80_000)},
    "defend_deal":      {"title": "Защита сделки от оспаривания", "fee": 6000, "months": "3–6",
"lawyers": (100_000, 300_000)},
    "objection":        {"title": "Возражения",         "fee": 0,    "months": "1–2", "lawyers": (50_000,
100_000)},
    "appeal":           {"title": "Обжалование",        "fee": 3000, "months": "1–2", "lawyers": (50_000,
100_000)},
    "remove_au":        {"title": "Отстранение АУ",     "fee": 0,    "months": "1–2", "lawyers": (50_000,
100_000)},
    "fight_subordination": {"title": "Защита от субординации", "fee": 0, "months": "2–4", "lawyers":
(100_000, 250_000)},
}


@dataclass
class CalcInput:
    mode: str = "P4"
    # требование
    amount: float = 0.0
    construction_type: str = "supply"
    is_credit_contract: bool = False
    claim_date: date | None = None
    filing_date: date | None = None
    execution_date: date | None = None
    executed_amount: float = 0.0
    status: str = "independent"  # independent | affiliated | kdl
    crisis: bool = False
    rescue_form: bool = False
    unusual_conditions: bool = False
    exceptions: dict = field(default_factory=dict)
    beneficial_interest: bool = False
    only_covenants: bool = False
    is_equivalent: bool = True
    has_harm_intent: bool = False
    is_gratuitous: bool = False
    is_interested: bool = False
    # должник
    debtor_type: str = "legal"
    industry: str | None = None
    total_assets: float | None = None
    current_expenses: float = 0.0
    first_priority: float = 0.0
    second_priority: float = 0.0
    executive_fee: float | None = None
    collateral: list[dict] = field(default_factory=list)   # [{"C":..., "L":...}]
    sigma3: float | None = None
    sigma_sub: float = 0.0
    subsidiary_expected: float = 0.0
    # судья / АУ
    judge_stats: dict | None = None
```

```
    court_avg: float = 0.43
    au_stats: dict | None = None
    au_signals: dict = field(default_factory=dict)
    # предыдущий уровень данных
    data_flags: dict = field(default_factory=dict)
    facts_confirmed: bool = True


def _r(x: float) -> float:
    return round(float(x), 2)


def _level_label(p: float, hi: float, mid: float) -> str:
    return "высокий" if p >= hi else "средний" if p >= mid else "низкий"


def calculate(inp: CalcInput) -> dict:
    S = float(inp.amount)
    X = float(inp.executed_amount)
    warnings: list[str] = []
    assumptions: list[str] = []

    # --- M0 / M1 ---
    m0 = m0_filter(inp.construction_type)
    affiliated = inp.status in ("affiliated", "kdl")
    m1 = m1_queue(inp.claim_date, inp.filing_date, inp.construction_type, inp.is_credit_contract,
affiliated)
    queue = m1["queue"]
    if not m0["passes"]:
        queue = "third"

    # --- M3 (экономика) ---
    assets_known = inp.total_assets is not None
    M0 = float(inp.total_assets or 0.0)
    fee = inp.executive_fee if inp.executive_fee is not None else 0.0
    m3 = m3_economics(M0, inp.current_expenses, inp.first_priority, inp.second_priority, fee,
                      inp.collateral, inp.industry, inp.debtor_type)
    M, C_net, p_zero_base = m3["M"], m3["total_collateral_net"], m3["p_zero"]
    if not assets_known:
        p_zero_base = P_ZERO_INDUSTRY if inp.debtor_type == "legal" else 0.94
        assumptions.append("Данные об имуществе должника отсутствуют: применён отраслевой p_zero.")
    alpha = m1["alpha"] or 0.7
    G_total = min(S, alpha * C_net) if queue == "secured" else 0.0
    M3 = max(0.0, M - (alpha * C_net if C_net else 0.0))
    if inp.sigma3 is None:
        sigma3 = S * DEFAULT_SIGMA3_FACTOR
        assumptions.append("Реестр неизвестен: Σ3 оценена как 10 × требование (консервативно).")
    else:
        sigma3 = float(inp.sigma3)
    sigma3 = max(sigma3, S)
    sigma_sub = max(inp.sigma_sub, S)
    if inp.subsidiary_expected:
        M3 += inp.subsidiary_expected

    # --- M2.1 / M2.2 ---
    ex = inp.exceptions
    m21 = m2_1_subordination(
        inp.status, inp.unusual_conditions, inp.crisis, inp.rescue_form,
        ex.get("surety_before_crisis", False), ex.get("non_voluntary", False),
        ex.get("market_practice", False), ex.get("majority_consent", False),
        inp.beneficial_interest, inp.only_covenants)
    m22 = m2_2_challenging(inp.execution_date, inp.filing_date, inp.is_equivalent, inp.has_harm_intent,
                           inp.is_gratuitous, inp.is_interested, X, M0)
    if not inp.facts_confirmed:
        assumptions.append("Факты из документов не подтверждены: применены консервативные допущения.")
        m22["p_ch"] = max(m22["p_ch"], 0.0 if m22["p_ch"] == 0 else m22["p_ch"] + 0.05)

    # --- M4 ---
```

```
    m4 = m4_practice(inp.judge_stats, inp.au_stats, inp.court_avg)
    p_sub = clamp(m21["p_sub"])
    p_ch = clamp(m22["p_ch"] + m4["delta_ch"] + m4["delta_au_ch"]) if m22["p_ch"] > 0 else 0.0
    p_zero = clamp(p_zero_base + m4["delta_au_zero"])

    au_idx = au_independence_score(inp.au_signals)
    if au_idx["critical"]:
        p_zero = max(p_zero, 0.85)

    # --- уровни данных / режим ---
    levels = detect_levels(inp.mode, {
        "has_assets": assets_known, "has_registry": inp.sigma3 is not None,
        "has_collateral": bool(inp.collateral), "has_judge_profile": m4["judge"]["profile"],
        "has_au_profile": m4["au"]["profile"], "has_execution": inp.execution_date is not None,
        "facts_confirmed": inp.facts_confirmed, **inp.data_flags})
    low_data = min(levels["M3"], levels["M2.1"], levels["M2.2"]) <= 1

    # --- M5: сценарии ---
    total_claim = S + X  # исходная сумма вместе с уже полученным
    r_base_v = base_payout(queue, S, M, M3, sigma3, C_net, alpha)
    base_total = r_base_v + X                                  # деньги на руках сохранены
    # оспаривание: X возвращается в массу, требование восстанавливается как реестровое
    ch_total = R_3(S + X, M3 + X, sigma3 + X) if X > 0 else R_3(S, M3, sigma3)
    sub_total = R_sub(S, M3, sigma3, sigma_sub) + X
    sub_after_ch = R_sub(S + X, M3 + X, sigma3 + X, sigma_sub + X) if X > 0 else R_sub(S, M3, sigma3,
sigma_sub)

    method = "probabilistic" if inp.mode in ("P0", "P1", "P2") else \
             "scenario" if inp.mode in ("P3", "P4") else "exact"

    if method == "probabilistic":
        frac = m5_probabilistic(queue)
        expected = frac * total_claim
        formula = "E[R] = (1−p_sub)(1−p_ch)·R_base·(1−p_zero)"
    else:
        e_r = E_R(p_sub, p_ch, base_total, ch_total, sub_total, sub_after_ch)
        expected = E_R_final(e_r, p_zero)
        formula = "Ф5 + Ф6"
        if method == "exact":
            expected = exact_discounted(expected)
            formula += " + дисконтирование"
    if au_idx["critical"]:
        expected = min(expected, 0.05 * total_claim)
    expected = min(expected, total_claim)

    spread = 0.5 if low_data else 0.2
    range_low, range_high = max(0.0, expected * (1 - spread)), min(total_claim, expected * (1 + spread))
    pct = expected / total_claim if total_claim else 0.0

    scenarios = _scenarios(p_sub, p_ch, p_zero, base_total, ch_total, sub_total, sub_after_ch,
total_claim)
    risks = _risks(p_sub, p_ch, p_zero, M3, C_net)
    actions, priorities = _decide(inp, queue, p_sub, p_ch, p_zero, expected, base_total, ch_total,
au_idx, X)
    confidence = "low" if low_data else ("medium" if method == "scenario" else "high")

    return {
        "mode": inp.mode, "method": method, "formula": formula,
        "queue": queue, "m0": m0, "m1": m1, "m2_1": m21, "m2_2": m22, "m3": {**m3, "M3": M3}, "m4": m4,
        "p_sub": _r(p_sub), "p_ch": _r(p_ch), "p_zero": _r(p_zero),
        "claim_amount": _r(S), "executed_amount": _r(X), "total_claim": _r(total_claim),
        "expected_amount": _r(expected), "expected_pct": round(pct, 4),
        "range_low": _r(range_low), "range_high": _r(range_high),
        "range_low_pct": round(range_low / total_claim, 4) if total_claim else 0,
        "range_high_pct": round(range_high / total_claim, 4) if total_claim else 0,
        "confidence": confidence, "probabilistic_label": "Вероятностный прогноз",
        "information_levels": levels, "scenarios": scenarios, "risks": risks,
        "au_index": au_idx, "actions": actions, "priorities": priorities,
```

```
        "assumptions": assumptions, "warnings": warnings,
        "audit": {
            "M": _r(M), "M0": _r(M0), "C_net": _r(C_net), "M3": _r(M3), "sigma3": _r(sigma3),
            "sigma_sub": _r(sigma_sub), "G": _r(G_total), "alpha": alpha if queue == "secured" else None,
            "R_base": _r(base_total), "R_challenged": _r(ch_total), "R_sub": _r(sub_total),
            "R_sub_after_ch": _r(sub_after_ch),
            "formulas": _formula_lines(S, X, M0, inp, M, C_net, alpha, queue, M3, sigma3, base_total,
                                       p_sub, p_ch, p_zero, expected),
        },
    }


def _scenarios(p_sub, p_ch, p_zero, base, ch, sub, sub_ch, total):
    k = 1 - p_zero
    raw = [
        ("base", "Без оспаривания и субординации", (1 - p_sub) * (1 - p_ch) * k, base),
        ("challenged", "Оспаривание сделки", p_ch * (1 - p_sub) * k, ch),
        ("subordinated", "Субординация", p_sub * (1 - p_ch) * k, sub),
        ("subordinated_after_ch", "Оспаривание и субординация", p_sub * p_ch * k, sub_ch),
        ("zero", "Нулевая масса", p_zero, 0.0),
    ]
    return [{"name": n, "title": t, "probability": round(p, 4), "payout_rub": _r(v),
             "payout_pct": round(v / total, 4) if total else 0.0} for n, t, p, v in raw]


def _risks(p_sub, p_ch, p_zero, M3, C_net):
    def light(p, hi, mid):
        return "red" if p >= hi else "yellow" if p >= mid else "green"
    return [
        {"key": "challenging", "title": "Оспаривание (статистическая частота по аналогичным делам)",
         "p": _r(p_ch), "light": light(p_ch, 0.5, 0.3), "label": _level_label(p_ch, 0.5, 0.3)},
        {"key": "subordination", "title": "Субординация", "p": _r(p_sub),
         "light": light(p_sub, 0.5, 0.2), "label": _level_label(p_sub, 0.5, 0.2)},
        {"key": "mass", "title": "Конкурсная масса", "p": _r(p_zero),
         "light": light(p_zero, 0.6, 0.4),
         "label": "есть активы" if p_zero < 0.4 else "масса под вопросом" if p_zero < 0.6 else "масса
вероятно пуста"},
    ]


def _cost(key: str) -> dict:
    c = ACTION_COSTS[key]
    lo, hi = c["lawyers"]
    return {"key": key, "title": c["title"], "fee_rub": c["fee"], "months": c["months"],
            "lawyers_rub": [lo, hi], "total_mid_rub": c["fee"] + (lo + hi) / 2}


def _decide(inp, queue, p_sub, p_ch, p_zero, expected, base_total, ch_total, au_idx, X):
    actions: list[dict] = []
    priorities: list[str] = []

    def add(key, why, gain=None, rank=50):
        cost = _cost(key)
        worth = True if gain is None else gain > cost["total_mid_rub"] and gain >= MIN_ECONOMIC_GAIN
        actions.append({**cost, "reason": why, "expected_gain_rub": None if gain is None else _r(gain),
                        "economically_justified": worth, "rank": rank})

    # П0: дедлайн всегда первый
    add("registry", "П0: включение в реестр — первое действие; пропуск 30-дневного срока необратим.",
rank=0)
    priorities.append("П0 Дедлайн: включение в реестр")

    fight_sub = p_sub > 0.75
    if fight_sub:
        add("fight_subordination", f"П1: p_sub = {p_sub:.2f} > 0,75 — 99% субординированных требований не
погашаются.",
            gain=max(base_total, 0) * 0.5, rank=10)
        priorities.append("П1 Борьба с субординацией")
```

```
    loss = max(0.0, base_total - ch_total)
    low_pay = p_zero > 0.80 and expected < MIN_ECONOMIC_GAIN
    if p_ch > 0.70 and not fight_sub and not low_pay:
        add("defend_deal", f"П2: p_ch = {p_ch:.2f} > 0,70 — приоритетная защита сделки; рассмотреть мировое соглашение.",
            gain=loss * DEFENSE_SUCCESS, rank=20)
        priorities.append("П2 Защита сделки")
    elif 0.5 <= p_ch <= 0.70 and X > 0 and not low_pay:
        add("defend_deal", f"Активная защита: p_ch = {p_ch:.2f}. Собрать доказательства встречного предоставления и рыночности условий.",
            gain=loss * DEFENSE_SUCCESS, rank=30)
        priorities.append("Активная защита сделки")
    elif 0.3 <= p_ch < 0.5 and X > 0:
        priorities.append("Пассивная защита сделки (хранить документы)")
    if au_idx["critical"]:
        add("remove_au", "П5: сумма баллов АУ > 60 — признаки аффилированности, требующие судебной проверки; "
            "ходатайство об отстранении (решение принимает суд).", rank=15)
        priorities.append("П5 Ходатайство об отстранении АУ")
    if low_pay:
        priorities.append("П3 Только включение в реестр (p_zero > 0,80, выплата < 500 тыс. ₽)")
    if p_zero > 0.80:
        priorities.append("Рассмотреть отказ от активных действий")
    actions.sort(key=lambda a: a["rank"])
    return actions, priorities


def _formula_lines(S, X, M0, inp, M, C_net, alpha, queue, M3, sigma3, base_total, p_sub, p_ch, p_zero, expected):
    f = lambda v: f"{v:,.0f}".replace(",", " ")
    lines = [
        f"M = max(0, {f(M0)} − {f(inp.current_expenses)} − {f(inp.first_priority)} − {f(inp.second_priority)} − {f(inp.executive_fee or 0)}) = {f(M)} ₽",
        f"C_net = Σ(C·L) = {f(C_net)} ₽",
    ]
    if queue == "secured":
        lines.append(f"G = α·C_net = {alpha}·{f(C_net)} = {f(min(S, alpha * C_net))} ₽ (Ф2)")
    lines += [
        f"M3 = {f(M3)} ₽, Σ3 = {f(sigma3)} ₽",
        f"Ф1: D({f(S)}, {f(M3)}, {f(sigma3)}) = {f(D(S, M3, sigma3))} ₽",
        f"p_sub = {p_sub:.2f}, p_ch = {p_ch:.2f}, p_zero = {p_zero:.2f}",
        f"Ф5/Ф6: E[R] = {f(expected)} ₽",
    ]
    return lines
```

backend/app/services/document_processor.py (129 строк)

```
"""Загрузка документов: извлечение текста (PDF/DOCX/OCR) и фактов (regex-эвристики).

Факты — кандидаты: пользователь подтверждает их вручную; до подтверждения они
«требуют проверки», и расчёт применяет консервативные допущения.
Фактическая аффилированность, мотив и осведомлённость не извлекаются (только экспертно).
"""
import re
from pathlib import Path

ALLOWED = {
    "application/pdf": ".pdf", "image/jpeg": ".jpg", "image/png": ".png",
    "application/vnd.openxmlformats-officedocument.wordprocessingml.document": ".docx",
}

DOC_TYPES = [
    ("pledge", ("договор залога", "предмет залога", "залогодержатель")),
    ("loan", ("договор займа", "заимодавец", "процент")),
    ("payment", ("платёжное поручение", "платежное поручение", "платёжн", "платежн")),
    ("act", ("акт сверки", "акт выполненных", "универсальный передаточный", "накладная")),
    ("claim_letter", ("претензия", "требование об оплате")),
    ("supply", ("договор поставки", "поставщик", "покупатель")),
```

```
]

DATE_RE = r"(\d{2}\.\d{2}\.\d{4})"
AMOUNT_RE = r"(\d[\d\s ]*(?:[.,]\d{1,2})?)\s*(?:руб|₽|р\.)"


def extract_text(path: str, mime: str) -> str:
    if mime == "application/pdf":
        import pdfplumber
        with pdfplumber.open(path) as pdf:
            text = "\n".join((p.extract_text() or "") for p in pdf.pages)
        return text if text.strip() else _ocr_pdf(path)
    if mime in ("image/jpeg", "image/png"):
        return _ocr_image(path)
    if mime.endswith("wordprocessingml.document"):
        import docx
        return "\n".join(p.text for p in docx.Document(path).paragraphs)
    raise ValueError(f"Неподдерживаемый тип файла: {mime}")


def _ocr_image(path: str) -> str:
    try:
        import pytesseract
        from PIL import Image
        return pytesseract.image_to_string(Image.open(path), lang="rus+eng")
    except Exception:  # noqa: BLE001 — OCR недоступен: пустой текст, факты не извлекутся
        return ""


def _ocr_pdf(path: str) -> str:
    try:
        import pdfplumber
        import pytesseract
        with pdfplumber.open(path) as pdf:
            return "\n".join(pytesseract.image_to_string(p.to_image(resolution=200).original,
lang="rus+eng")
                             for p in pdf.pages)
    except Exception:  # noqa: BLE001
        return ""


def classify(text: str) -> str:
    low = text.lower()
    best, score = "other", 0
    for name, keys in DOC_TYPES:
        s = sum(k in low for k in keys)
        if s > score:
            best, score = name, s
    return best


def _num(s: str) -> float:
    return float(re.sub(r"[\s ]", "", s).replace(",", "."))


def extract_facts(text: str, doc_type: str) -> list[dict]:
    """Возвращает [{field, value, confidence}]."""
    facts: list[dict] = []
    low = text.lower()

    def add(field, value, conf):
        facts.append({"field": field, "value": str(value), "confidence": conf})

    if m := re.search(r"(?:договор|контракт)[^\n]{0,60}?от\s*" + DATE_RE, low) or \
            re.search(r"от\s*" + DATE_RE, low):
        add("contract_date", m.group(1), 0.7)
    if m := re.search(r"(?:оплачено|оплата|платёж|платеж|исполнен)[^\n]{0,60}?" + DATE_RE, low):
        add("execution_date", m.group(1), 0.6)
```

```
    amounts = [_num(a) for a in re.findall(AMOUNT_RE, text, flags=re.I)]
    if amounts:
        add("principal_amount", max(amounts), 0.6)
    if m := re.search(r"(?:неустойк|штраф|пени)[^\n]{0,60}?" + AMOUNT_RE, text, flags=re.I):
        add("penalty_amount", _num(m.group(1)), 0.5)
    for inn in sorted(set(re.findall(r"инн\D{0,5}(\d{10}|\d{12})", low))):
        add("party_inn", inn, 0.9)
    if m := re.search(r"(\d{1,2}(?:[.,]\d+)?)\s*%\s*годовых", low):
        add("interest_rate", m.group(1), 0.7)
    if "отсрочк" in low or "рассрочк" in low or "пролонгац" in low:
        add("rescue_form_marker", "отсрочка/пролонгация", 0.5)
    if doc_type == "pledge":
        add("collateral_registered", "да" if "реестр уведомлений" in low else "нет", 0.5)
        if m := re.search(r"(?:оценочная|рыночная)\s+стоимость[^\n]{0,40}?" + AMOUNT_RE, text,
flags=re.I):
            add("collateral_value", _num(m.group(1)), 0.6)
    if any(k in low for k in ("просроч", "иск о взыскании", "претензи")):
        add("crisis_marker", "просрочка/иск/претензия", 0.5)
    if doc_type in ("act", "supply") or "встречн" in low:
        add("counter_performance", "да", 0.4)
    return facts


def levels_from_documents(doc_types: set[str], confirmed: bool) -> dict:
    """Какие флаги данных повышают уровни (раздел 9.6). Без подтверждения уровни не повышаются."""
    if not confirmed:
        return {}
    flags = {}
    if "payment" in doc_types and ({"supply", "loan", "act"} & doc_types):
        flags["has_payment_docs"] = True
    if "pledge" in doc_types:
        flags["has_collateral"] = True
    if "affiliation" in doc_types:
        flags["has_affiliation_docs"] = True
    return flags


def storage_path(base: str, claim_id: int, filename: str) -> Path:
    safe = re.sub(r"[^\w.\-]", "_", filename)[:100] or "file"
    p = Path(base) / "documents" / str(claim_id)
    p.mkdir(parents=True, exist_ok=True)
    return p / safe
```

backend/app/services/email/__init__.py (15 строк)

```
from app.core.config import settings
from app.services.email.base import EmailProvider
from app.services.email.mock import MockEmailProvider
from app.services.email.smtp import SMTPProvider


class EmailService:
    """При EMAIL_SENDING=false всегда логирует письмо (mock), независимо от EMAIL_PROVIDER."""

    def __init__(self):
        real = settings.EMAIL_SENDING and settings.EMAIL_PROVIDER == "smtp"
        self.provider: EmailProvider = SMTPProvider() if real else MockEmailProvider()

    async def send(self, to: str, subject: str, body: str) -> dict:
        return await self.provider.send(to, subject, body)
```

backend/app/services/email/base.py (6 строк)

```
from abc import ABC, abstractmethod


```

```
class EmailProvider(ABC):
    @abstractmethod
    async def send(self, to: str, subject: str, body: str) -> dict: ...
```

backend/app/services/email/mock.py (12 строк)

```
from app.core.logging import get_logger
from app.services.email.base import EmailProvider

log = get_logger("email.mock")


class MockEmailProvider(EmailProvider):
    """Логирует письмо, отправка не выполняется."""

    async def send(self, to: str, subject: str, body: str) -> dict:
        log.info("mock_email", to=to, subject=subject, body_len=len(body))
        return {"sent": False, "logged": True}
```

backend/app/services/email/smtp.py (8 строк)

```
from app.services.email.base import EmailProvider


class SMTPProvider(EmailProvider):
    """Заглушка. Реализовать при переходе на версию 2.0 (SMTP_HOST, SMTP_USER, SMTP_PASSWORD)."""

    async def send(self, to: str, subject: str, body: str) -> dict:
        raise NotImplementedError("Реализовать при переходе на версию 2.0")
```

backend/app/services/formulas.py (53 строк)

```
"""Универсальные формулы Ф1–Ф6 (раздел 6.8 ТЗ, глава 5 методики).

Все функции чистые (без БД и IO) и работают с float.
"""


def D(S: float, P: float, Sigma: float) -> float:
    """Ф1. Пропорциональное распределение: min(S, P·S/Σ)."""
    if Sigma <= 0 or S <= 0 or P <= 0:
        return 0.0
    return min(S, P * S / Sigma)


def R_sec(S: float, C_net: float, alpha: float, M3: float, Sigma3: float) -> float:
    """Ф2. Залоговый кредитор: G + D(S − G, M3, Σ3), где G = α·C_net (G не больше S)."""
    G = min(S, alpha * C_net)
    return G + D(S - G, M3, Sigma3)


def R_3(S: float, M3: float, Sigma3: float) -> float:
    """Ф3. Незалоговый кредитор третьей очереди."""
    return D(S, M3, Sigma3)


def R_sub(S: float, M3: float, Sigma3: float, Sigma_sub: float) -> float:
    """Ф4. Субординированный: получает только остаток после независимых кредиторов."""
    remaining = max(0.0, M3 - Sigma3)
    return D(S, remaining, Sigma_sub)


def E_R(p_sub: float, p_ch: float, R_base: float, R_3_: float, R_sub_: float,
        R_sub_after_ch: float = 0.0) -> float:
```

```
    """Ф5. Сценарное взвешивание (четыре исхода)."""
    return (
        (1 - p_sub) * (1 - p_ch) * R_base
        + p_ch * (1 - p_sub) * R_3_
        + p_sub * (1 - p_ch) * R_sub_
        + p_sub * p_ch * R_sub_after_ch
    )


def E_R_final(e_r: float, p_zero: float) -> float:
    """Ф6. Поправка на нулевую массу."""
    return (1 - p_zero) * e_r


def discount_factor(rate: float, years: float) -> float:
    """D_t = 1/(1+r)^t — дисконтирование выплаты (средняя процедура ≈ 1265 дней)."""
    return 1.0 / ((1 + rate) ** years)


def clamp(x: float, lo: float = 0.0, hi: float = 1.0) -> float:
    return max(lo, min(hi, x))
```

backend/app/services/level_detector.py (24 строк)

```
"""Уровни полноты данных (0–5) по каждому модулю."""

MODE_BASE_LEVEL = {"P0": 0, "P1": 1, "P2": 1, "P3": 2, "P4": 3, "P5": 4, "P6": 5}


def detect_levels(mode: str, data: dict) -> dict:
    base = MODE_BASE_LEVEL.get(mode, 0)
    m3 = base if data.get("has_assets") else min(base, 1)
    if data.get("has_assets") and data.get("has_collateral") and base >= 4:
        m3 = 5
    m1 = max(base, 4 if data.get("has_registry") else base)
    m21 = 2 if base >= 2 else 1
    if data.get("facts_confirmed") and data.get("has_affiliation_docs"):
        m21 = 4
    m22 = 1 if not data.get("has_execution") else max(2, min(base, 3))
    if data.get("has_execution") and data.get("facts_confirmed") and data.get("has_payment_docs"):
        m22 = 5
    m4 = 0
    if data.get("has_judge_profile"):
        m4 = 3
    if data.get("has_au_profile"):
        m4 = max(m4, 4)
    m5 = min(5, max(base, 1)) if data.get("has_registry") or base >= 3 else min(base, 2)
    return {"M0": 5, "M1": min(5, m1), "M2.1": m21, "M2.2": m22, "M3": m3, "M4": m4, "M5": m5}
```

backend/app/services/mode_detector.py (24 строк)

```
"""Автоматическое определение режима P0–P6 по данным КАД/ЕФРСБ."""
from app.services.parsers import fetch_case_bundle


def mode_from_bundle(bundle: dict) -> dict:
    if not bundle.get("found"):
        if bundle.get("crisis_signs"):
            return {"mode": "P2", "reason": "Признаки кризиса, дело не возбуждено"}
        return {"mode": "P1", "reason": "Сделка заключена, банкротства нет"}
    types = {m.get("type") for m in bundle.get("messages", [])}
    if not bundle.get("messages") and not bundle.get("debtor", {}).get("name"):
        return {"mode": "P3", "reason": "Заявление подано, данные ЕФРСБ ещё не опубликованы"}
    if "trading_started" in types:
        return {"mode": "P6", "reason": "Торги начались"}
    if "registry_closed" in types:
```

```
        return {"mode": "P5", "reason": "Реестр закрыт"}
    if {"judge_appointed", "au_appointed"} <= types:
        return {"mode": "P4", "reason": "Судья и АУ известны"}
    return {"mode": "P3", "reason": "Дело возбуждено, судья не определён"}


async def detect_mode(inn: str | None = None, case_number: str | None = None) -> dict:
    bundle = await fetch_case_bundle(inn=inn, case_number=case_number)
    return {**mode_from_bundle(bundle), "status": bundle.get("status"), "warnings":
bundle.get("warnings", [])}
```

backend/app/services/modules/__init__.py (1 строк)

```

```

backend/app/services/modules/m0_filter.py (12 строк)

```
"""M0. Фильтр конструкций (тест Бевзенко): устоит ли обеспечение в банкротстве."""

PASSES = ["pledge", "surety", "guarantee", "escrow", "title"]
FAILS = ["penalty", "deposit"]


def m0_filter(construction_type: str) -> dict:
    if construction_type in PASSES:
        return {"passes": True, "queue": "third_or_secured"}
    if construction_type in FAILS:
        return {"passes": False, "queue": "third"}
    return {"passes": True, "queue": "third"}
```

backend/app/services/modules/m1_queue.py (13 строк)

```
"""M1. Очередь: базовый «потолок» требования."""
from datetime import date


def m1_queue(claim_date: date | None, filing_date: date | None, construction_type: str,
             is_credit_contract: bool, is_affiliated: bool) -> dict:
    if claim_date and filing_date and claim_date > filing_date:
        return {"queue": "current", "alpha": None}
    if construction_type == "pledge":
        return {"queue": "secured", "alpha": 0.8 if is_credit_contract else 0.7}
    if is_affiliated:
        return {"queue": "third_subordinated", "alpha": None}
    return {"queue": "third", "alpha": None}
```

backend/app/services/modules/m2_1_subordination.py (25 строк)

```
"""M2.1. Субординация (Пленум ВС РФ № 41 от 23.12.2025)."""


def m2_1_subordination(status: str, unusual_conditions: bool = False, crisis: bool = False,
                       rescue_form: bool = False, exception_surety_before_crisis: bool = False,
                       exception_non_voluntary: bool = False, exception_market_practice: bool = False,
                       exception_majority_consent: bool = False, beneficial_interest: bool = False,
                       only_covenants: bool = False) -> dict:
    if exception_surety_before_crisis:
        return {"p_sub": 0.05, "reason": "Поручительство до кризиса"}
    if exception_non_voluntary:
        return {"p_sub": 0.05, "reason": "Недобровольный кредитор"}
    if exception_market_practice:
        return {"p_sub": 0.10, "reason": "Обычная рыночная практика"}
    if exception_majority_consent:
```

```
        return {"p_sub": 0.10, "reason": "Согласие мажоритарных кредиторов"}
    if only_covenants and not beneficial_interest:
        return {"p_sub": 0.15, "reason": "Только контроль возврата, бенефициарного интереса нет"}
    if status in ("kdl", "affiliated") and crisis and rescue_form:
        if unusual_conditions:
            return {"p_sub": 0.85, "reason": "Три элемента + необычные условия"}
        return {"p_sub": 0.70, "reason": "Три элемента"}
    if status in ("kdl", "affiliated") and crisis:
        return {"p_sub": 0.50, "reason": "Статус + кризис"}
    return {"p_sub": 0.10, "reason": "Нет оснований"}
```

backend/app/services/modules/m2_2_challenging.py (28 строк)

```
"""M2.2. Оспаривание (ст. 61.2, 61.3 Закона о банкротстве)."""
from datetime import date


def m2_2_challenging(execution_date: date | None, filing_date: date | None,
                     is_equivalent: bool = True, has_harm_intent: bool = False,
                     is_gratuitous: bool = False, is_interested: bool = False,
                     amount: float = 0.0, total_assets: float = 0.0) -> dict:
    if execution_date is None or filing_date is None:
        return {"p_ch": 0.0, "reason": "Нет исполнения", "window": None}
    months = (filing_date - execution_date).days / 30
    if total_assets > 0 and amount <= 0.01 * total_assets:
        return {"p_ch": 0.05, "reason": "Обычная хозяйственная деятельность (≤1% активов)",
                "window": "ordinary"}
    if months < 0:
        return {"p_ch": 0.0, "reason": "Исполнение после принятия заявления (текущий платёж)",
                "window": None}
    if months <= 1:
        return {"p_ch": 0.70, "reason": "Окно 1 месяц (предпочтение предполагается, ст. 61.3)",
                "window": "1m"}
    if months <= 6:
        return {"p_ch": 0.50, "reason": "Окно 6 месяцев (нужно доказать осведомлённость)",
                "window": "6m"}
    if months <= 12 and not is_equivalent:
        return {"p_ch": 0.60, "reason": "Неравноценность в пределах 1 года (ст. 61.2)", "window": "1y"}
    if months <= 36 and (has_harm_intent or is_gratuitous or is_interested):
        return {"p_ch": 0.40, "reason": "Признаки вреда в пределах 3 лет (ст. 61.2)", "window": "3y"}
    return {"p_ch": 0.05, "reason": "Вне окон оспаривания", "window": None}
```

backend/app/services/modules/m3_economics.py (26 строк)

```
"""M3. Экономика: чистая конкурсная масса, ликвидный залог, вероятность нулевой массы."""


def executive_fee(base: float, is_citizen: bool = False) -> float:
    """Исполнительский сбор с 09.01.2026: 12%, минимум 2 000 ₽ (граждане) / 20 000 ₽ (организации)."""
    if base <= 0:
        return 0.0
    return max(0.12 * base, 2000.0 if is_citizen else 20000.0)


def estimate_p_zero(industry: str | None, mass: float, collateral: float,
                    debtor_type: str = "legal") -> float:
    if collateral > 0 and mass > 0:
        return 0.20
    if mass > 0:
        return 0.50
    return 0.85


def m3_economics(M0: float, T: float, Q1: float, Q2: float, S_sbor: float,
                 collateral_list: list[dict], industry: str | None = None,
                 debtor_type: str = "legal") -> dict:
```

```
    M = max(0.0, M0 - T - Q1 - Q2 - S_sbor)
    total_collateral = sum(c["C"] * c["L"] for c in collateral_list)
    p_zero = estimate_p_zero(industry, M, total_collateral, debtor_type)
    return {"M": M, "total_collateral_net": total_collateral, "p_zero": p_zero}
```

backend/app/services/modules/m4_practice.py (85 строк)

```
"""M4. Анализ судебной практики и признаки аффилированности АУ, требующие проверки."""
from app.services.formulas import clamp

BASE_P_CH = 0.43
MIN_JUDGE_CASES = 10
MIN_AU_CASES = 5


def judge_delta(judge_stats: dict | None, court_avg: float = BASE_P_CH) -> dict:
    """Δ_ch по отклонению доли удовлетворения от средней по суду."""
    if not judge_stats or judge_stats.get("n_cases", 0) < MIN_JUDGE_CASES:
        return {"delta_ch": 0.0, "profile": False,
                "note": "Менее 10 дел: профиль судьи не строится, используется статистика суда"}
    dev = judge_stats["approval_rate"] - court_avg
    if abs(dev) <= 0.05:
        delta = 0.0
    else:
        delta = clamp(0.6 * dev, -0.15, 0.15)
        # при |отклонении| > 15 п.п. модификатор не слабее 0,10
        if abs(dev) > 0.15:
            delta = max(0.10, abs(delta)) * (1 if dev > 0 else -1)
    significant = bool(judge_stats.get("is_significant"))
    if not significant:
        delta *= 0.5
    return {"delta_ch": round(delta, 4), "profile": True, "deviation": round(dev, 4),
            "significant": significant,
            "note": "Статистически значимо" if significant else "Статистически незначимо (ДИ широкий)"}


def au_delta(au_stats: dict | None) -> dict:
    """Модификаторы АУ к p_ch и p_zero."""
    if not au_stats or au_stats.get("n_cases", 0) < MIN_AU_CASES:
        return {"delta_ch": 0.0, "delta_zero": 0.0, "profile": False,
                "note": "Менее 5 дел: профиль АУ не строится"}
    d_ch = d_zero = 0.0
    notes = []
    if au_stats.get("filed_on_your_deal"):
        d_ch += 0.10
        notes.append("АУ подал заявление по вашей сделке (+0,10 к p_ch)")
    else:
        similar = au_stats.get("filed_similar", 0)
        if similar:
            d_ch += min(0.20, 0.03 * similar)
            notes.append(f"АУ подавал по аналогичным сделкам ×{similar} (+{min(0.20, 0.03 *
similar):.2f})")
    if au_stats.get("passive_on_suspicious"):
        d_zero += 0.10
        notes.append("АУ не подал по явно подозрительной сделке (+0,10 к p_zero)")
    if au_stats.get("losses_recovered"):
        d_zero += 0.10
        notes.append("С АУ взысканы убытки (+0,10 к p_zero)")
    if au_stats.get("rau_top20"):
        d_zero -= 0.05
        notes.append("Верхние 20% рейтинга РАУ (−0,05 к p_zero)")
    return {"delta_ch": round(d_ch, 4), "delta_zero": round(d_zero, 4), "profile": True,
            "note": "; ".join(notes) or "Особых сигналов нет"}


def m4_practice(judge_stats: dict | None, au_stats: dict | None, court_avg: float = BASE_P_CH) -> dict:
    j = judge_delta(judge_stats, court_avg)
```

```
    a = au_delta(au_stats)
    return {"delta_ch": j["delta_ch"], "delta_au_ch": a["delta_ch"], "delta_au_zero": a["delta_zero"],
            "judge": j, "au": a}


AU_FLAG_POINTS = {
    "removed": 50, "disqualified": 60, "disqualification_filed": 30, "applicant_affiliated": 40,
    "former_representative": 40, "sro_match": 20, "anomalous_passivity": 30,
    "sro_complaint_upheld": 20, "losses_recovered": 20,
}
AU_FLAG_LABELS = {
    "removed": "АУ отстранён в деле", "disqualified": "АУ дисквалифицирован",
    "disqualification_filed": "Подано заявление о дисквалификации",
    "applicant_affiliated": "Заявитель аффилирован с должником",
    "former_representative": "Бывший представитель или доверенное лицо",
    "sro_match": "Совпадение по СРО", "anomalous_passivity": "Аномальная пассивность",
    "sro_complaint_upheld": "Жалоба в СРО признана обоснованной", "losses_recovered": "Взысканы убытки",
}


def au_independence_score(signals: dict) -> dict:
    """Модуль независимости АУ (макс. 100, >60 — флаг). Терминология: «признаки, требующие проверки»."""
    items = [{"flag": k, "label": AU_FLAG_LABELS[k], "points": p}
             for k, p in AU_FLAG_POINTS.items() if signals.get(k)]
    score = min(100, sum(i["points"] for i in items))
    return {"score": score, "critical": score > 60, "items": items}
```

backend/app/services/modules/m5_calc.py (32 строк)

```
"""M5. Итоговый расчёт по режиму: вероятностный (P0–P2), сценарный (P3–P4), точный (P5–P6)."""
from app.services.formulas import D, E_R, E_R_final, R_3, R_sec, R_sub, discount_factor

# Отраслевые ориентиры для вероятностного режима (доли от суммы требования)
R_BASE_BY_TYPE = {"secured": 0.40, "third": 0.06, "third_subordinated": 0.06, "current": 0.90}
P_CH_INDUSTRY = 0.43
P_SUB_INDUSTRY = 0.10
P_ZERO_INDUSTRY = 0.43
AVG_PROCEDURE_DAYS = 1265
DISCOUNT_RATE = 0.16


def m5_probabilistic(queue: str) -> float:
    """E[R] в долях требования по отраслевым ориентирам (P0–P2):
    (1−p_sub_отр)(1−p_ch_отр)·R_base_тип·(1−p_zero_отр)."""
    r_base = R_BASE_BY_TYPE.get(queue, 0.06)
    return (1 - P_SUB_INDUSTRY) * (1 - P_CH_INDUSTRY) * r_base * (1 - P_ZERO_INDUSTRY)


def base_payout(queue: str, S: float, M: float, M3: float, Sigma3: float,
                C_net: float, alpha: float | None) -> float:
    """Выплата в базовом сценарии по очереди требования."""
    if queue == "current":
        return min(S, M)
    if queue == "secured":
        return R_sec(S, C_net, alpha or 0.7, M3, Sigma3)
    return R_3(S, M3, Sigma3)


def exact_discounted(payout: float, years: float | None = None, rate: float = DISCOUNT_RATE) -> float:
    t = years if years is not None else AVG_PROCEDURE_DAYS / 365
    return payout * discount_factor(rate, t)
```

backend/app/services/parsers/__init__.py (36 строк)

```
"""Фасад источников данных: live (ЕФРСБ + КАД) или mock — переключается PARSERS_MODE."""
from app.core.config import settings
```

```
from app.services.parsers import mock
from app.services.parsers.resilience import SourceUnavailable


async def fetch_case_bundle(*, inn: str | None = None, case_number: str | None = None) -> dict:
    """Возвращает единый словарь данных по делу. При сбое источников: status=partial + ручной ввод."""
    if settings.PARSERS_MODE == "mock":
        return {**mock.case_bundle(inn, case_number), "status": "ok", "warnings": [
            "Режим PARSERS_MODE=mock: используются демонстрационные данные, а не реальные сведения КАД/ЕФРСБ."]}
    warnings: list[str] = []
    bundle: dict = {"source": "live", "found": False, "messages": []}
    try:
        from app.services.parsers import kad
        cases = await kad.search_cases(inn=inn, case_number=case_number)
        if cases:
            bundle.update(found=True, case_number=cases[0]["number"])
    except SourceUnavailable as e:
        warnings.append(f"КАД Арбитр недоступен: {e}")
    except Exception as e:  # noqa: BLE001
        warnings.append(f"КАД Арбитр: ошибка {e!r}")
    if inn:
        try:
            from app.services.parsers import efrsb
            b = await efrsb.get_bankrupt(inn)
            if b:
                bundle.update(found=True, debtor={"inn": inn, "name": b.get("name")},
                              efrsb_guid=b.get("guid"))
                bundle["messages"] = await efrsb.get_messages(b["guid"]) if b.get("guid") else []
        except SourceUnavailable as e:
            warnings.append(f"ЕФРСБ недоступен: {e}")
        except Exception as e:  # noqa: BLE001
            warnings.append(f"ЕФРСБ: ошибка {e!r}")
    status = "partial" if warnings else "ok"
    return {**bundle, "status": status, "warnings": warnings, "manual_input_required": status == "partial"}
```

backend/app/services/parsers/efrsb.py (56 строк)

```
"""ЕФРСБ (httpx): карточка должника и сообщения. Кэш: карточка 7 суток, сообщения 24 ч."""
from typing import Optional

import httpx

from app.core.cache import cache_get, cache_set
from app.core.config import settings
from app.services.parsers.resilience import efrsb_breaker, with_retry


def _headers() -> dict:
    return {"Authorization": f"Bearer {settings.EFRSB_API_KEY}"} if settings.EFRSB_API_KEY else {}


async def get_bankrupt(inn: str) -> Optional[dict]:
    key = f"efrsb:bankrupt:{inn}"
    if (c := await cache_get(key)) is not None:
        return c or None

    async def call():
        async with httpx.AsyncClient(timeout=30, headers=_headers()) as client:
            r = await client.get(f"{settings.EFRSB_API_URL}/bankrupts", params={"inn": inn})
            if r.status_code == 404:
                return None
            r.raise_for_status()
            return r.json()

    data = await with_retry(call, efrsb_breaker)
```

```
    await cache_set(key, data or {}, 7 * 86400)
    return data


async def get_messages(bankrupt_guid: str, page: int = 1) -> list:
    key = f"efrsb:messages:{bankrupt_guid}:{page}"
    if (c := await cache_get(key)) is not None:
        return c

    async def call():
        async with httpx.AsyncClient(timeout=30, headers=_headers()) as client:
            r = await client.get(f"{settings.EFRSB_API_URL}/bankruptcy-messages",
                                 params={"bankruptGuid": bankrupt_guid, "page": page, "limit": 1000})
            r.raise_for_status()
            return r.json().get("items", [])

    items = await with_retry(call, efrsb_breaker)
    await cache_set(key, items, 86400)
    return items


async def get_arbitr_manager(guid: str) -> Optional[dict]:
    async def call():
        async with httpx.AsyncClient(timeout=30, headers=_headers()) as client:
            r = await client.get(f"{settings.EFRSB_API_URL}/arbitr-managers", params={"guid": guid})
            r.raise_for_status()
            return r.json()
    return await with_retry(call, efrsb_breaker)
```

backend/app/services/parsers/kad.py (48 строк)

```
"""КАД Арбитр (Playwright, headless Chromium). Cookies кэшируются в Redis на 24 ч.

Селекторы зависят от вёрстки kad.arbitr.ru и могут потребовать актуализации;
при сбое срабатывают retry и circuit breaker, API отвечает status=partial + ручной ввод.
"""
from app.core.cache import cache_get, cache_set
from app.core.config import settings
from app.services.parsers.resilience import kad_breaker, with_retry


async def _context(p):
    browser = await p.chromium.launch(headless=True, args=["--no-sandbox"])
    context = await browser.new_context()
    cookies = await cache_get("kad:cookies")
    if cookies:
        await context.add_cookies(cookies)
    return browser, context


async def search_cases(*, inn: str | None = None, case_number: str | None = None) -> list[dict]:
    key = f"kad:search:{inn or case_number}"
    if (c := await cache_get(key)) is not None:
        return c

    async def call():
        from playwright.async_api import async_playwright
        async with async_playwright() as p:
            browser, context = await _context(p)
            try:
                page = await context.new_page()
                await page.goto(settings.KAD_BASE_URL, timeout=30000)
                await cache_set("kad:cookies", await context.cookies(), 86400)
                await page.fill('input[placeholder*="ИНН" i], input[name="inn"]', inn or case_number or
"")
                await page.click('button[type="submit"]')
                await page.wait_for_selector(".b-found-cases, .case-list", timeout=30000)
```

```
                items = await page.query_selector_all(".b-found-cases tr, .case-item")
                out = []
                for it in items:
                    text = (await it.inner_text()).strip()
                    if text:
                        out.append({"raw": text, "number": text.split()[0]})
                return out
            finally:
                await browser.close()

    result = await with_retry(call, kad_breaker)
    await cache_set(key, result, 12 * 3600)
    return result
```

backend/app/services/parsers/mock.py (57 строк)

```
"""Детерминированные демо-данные (PARSERS_MODE=mock): результат зависит от ИНН / номера дела.

ИНН, оканчивающийся на 00 — дело не возбуждено (…000 → P1, иначе P2); иначе стадия определяется остатком от деления.
Данные НЕ реальные: интерфейс помечает такой расчёт как демонстрационный.
"""
import hashlib
from datetime import date, timedelta

JUDGES = [("Иванова А.А.", 45, 0.70, True), ("Сидоров В.Н.", 18, 0.38, False),
          ("Кузнецова М.П.", 8, 0.55, False), ("Орлов Д.С.", 62, 0.30, True)]
AUS = [("Петров П.П.", "СРО «Лидер»", 2, True), ("Смирнов К.Л.", "СРО «Альянс»", 0, False),
       ("Новиков Е.В.", "СРО «Лидер»", 1, False)]
STAGES = ["P3", "P4", "P4", "P5", "P6"]


def _seed(s: str) -> int:
    return int(hashlib.sha256(s.encode()).hexdigest(), 16)


def case_bundle(inn: str | None = None, case_number: str | None = None, today: date | None = None) -> dict:
    today = today or date.today()
    key = inn or case_number or "0"
    h = _seed(key)
    if inn and inn.endswith("00"):
        # ...000 — сделка без признаков кризиса (P1); прочие ...00 — признаки кризиса, дело не возбуждено (P2)
        return {"source": "mock", "found": False, "crisis_signs": not inn.endswith("000")}
    stage = STAGES[h % len(STAGES)]
    j = JUDGES[(h >> 3) % len(JUDGES)]
    a = AUS[(h >> 5) % len(AUS)]
    filing = today - timedelta(days=60 + (h >> 7) % 300)
    assets = float(50_000_000 + (h >> 9) % 200_000_000)
    sigma3 = assets * (0.6 + ((h >> 11) % 100) / 100)
    year = filing.year
    messages = [{"type": "case_opened", "date": filing.isoformat()}]
    if stage != "P3":
        messages += [{"type": "judge_appointed", "date": (filing + timedelta(days=5)).isoformat()},
                     {"type": "au_appointed", "date": (filing + timedelta(days=30)).isoformat()}]
    if stage in ("P5", "P6"):
        messages.append({"type": "registry_closed", "date": (filing + timedelta(days=120)).isoformat()})
    if stage == "P6":
        messages.append({"type": "trading_started", "date": (filing + timedelta(days=200)).isoformat()})
    return {
        "source": "mock", "found": True,
        "case_number": case_number or f"A40-{10000 + h % 89999}/{year}",
        "court": "Арбитражный суд города Москвы", "region": "Москва",
        "filing_date": filing.isoformat(), "stage": stage, "messages": messages,
        "debtor": {"inn": inn, "name": f"ООО «Демо-должник {str(h)[:4]}»", "type": "legal",
                   "industry": "trade", "total_assets": assets, "current_expenses": assets * 0.07,
```

```
                    "first_priority": 0.0, "second_priority": assets * 0.01},
        "judge": {"name": j[0], "n_cases": j[1], "approval_rate": j[2], "is_significant": j[3]},
        "court_avg": 0.43,
        "au": {"name": a[0], "sro": a[1], "n_cases": 20, "filed_similar": a[2],
               "filed_on_your_deal": False, "rau_top20": a[3], "losses_recovered": 0},
        "au_signals": {},
        "registry": {"sigma3": sigma3, "sigma_sub": sigma3 * 0.05},
        "collateral": [{"C": assets * 0.3, "L": 0.6, "type": "equipment"}] if h % 3 == 0 else [],
    }
```

backend/app/services/parsers/resilience.py (44 строк)

```python
"""Повторные попытки с exponential backoff и circuit breaker (pybreaker)."""
import asyncio

import pybreaker

from app.core.config import settings
from app.core.logging import get_logger

log = get_logger("parsers")

efrsb_breaker = pybreaker.CircuitBreaker(fail_max=5, reset_timeout=60, name="efrsb")
kad_breaker = pybreaker.CircuitBreaker(fail_max=3, reset_timeout=60, name="kad")


class SourceUnavailable(Exception):
    pass


async def with_retry(fn, breaker: pybreaker.CircuitBreaker, attempts: int = 3, base_delay: float = 1.0):
    """Вызывает async-функцию: 3 попытки, задержки 1, 2, 4 с; при открытом breaker — SourceUnavailable."""
    if settings.CIRCUIT_BREAKER_ENABLED and breaker.current_state == pybreaker.STATE_OPEN:
        raise SourceUnavailable(f"{breaker.name}: circuit breaker открыт")
    last = None
    for i in range(attempts):
        try:
            result = await fn()
            if settings.CIRCUIT_BREAKER_ENABLED:
                breaker.call(lambda: None)  # фиксируем успех
            return result
        except Exception as e:  # noqa: BLE001
            last = e
            if settings.CIRCUIT_BREAKER_ENABLED:
                try:
                    breaker.call(_raise, e)
                except Exception:  # noqa: BLE001
                    pass
            log.warning("source_retry", source=breaker.name, attempt=i + 1, error=repr(e))
            if i < attempts - 1:
                await asyncio.sleep(base_delay * (2 ** i))
    raise SourceUnavailable(f"{breaker.name}: {last!r}")


def _raise(e):
    raise e
```

backend/app/services/payments/__init__.py (8 строк)

```python
from app.core.config import settings
from app.services.payments.base import PaymentProvider
from app.services.payments.mock import MockPaymentProvider
from app.services.payments.yookassa import YooKassaProvider


```

```python
def get_payment_provider() -> PaymentProvider:
    return YooKassaProvider() if settings.PAYMENT_PROVIDER == "yookassa" else MockPaymentProvider()
```

backend/app/services/payments/base.py (12 строк)

```python
from abc import ABC, abstractmethod


class PaymentProvider(ABC):
    @abstractmethod
    async def create_payment(self, user_id, amount, plan) -> dict: ...

    @abstractmethod
    async def check_status(self, payment_id) -> str: ...

    @abstractmethod
    async def handle_webhook(self, payload) -> dict: ...
```

backend/app/services/payments/mock.py (23 строк)

```python
import uuid

from app.core.logging import get_logger
from app.services.payments.base import PaymentProvider

log = get_logger("payments.mock")


class MockPaymentProvider(PaymentProvider):
    """Только логирует вызов; списание не выполняется."""

    async def create_payment(self, user_id, amount, plan) -> dict:
        pid = f"mock-{uuid.uuid4().hex[:12]}"
        log.info("mock_payment_create", user_id=user_id, amount=amount, plan=plan, payment_id=pid)
        return {"payment_id": pid, "status": "mock_succeeded", "confirmation_url": None}

    async def check_status(self, payment_id) -> str:
        log.info("mock_payment_status", payment_id=payment_id)
        return "mock_succeeded"

    async def handle_webhook(self, payload) -> dict:
        log.info("mock_payment_webhook", payload=str(payload)[:200])
        return {"handled": True}
```

backend/app/services/payments/yookassa.py (14 строк)

```python
from app.services.payments.base import PaymentProvider


class YooKassaProvider(PaymentProvider):
    """Заглушка. Реализовать при переходе на версию 2.0 (YOOKASSA_SHOP_ID, YOOKASSA_SECRET_KEY)."""

    async def create_payment(self, user_id, amount, plan) -> dict:
        raise NotImplementedError("Реализовать при переходе на версию 2.0")

    async def check_status(self, payment_id) -> str:
        raise NotImplementedError("Реализовать при переходе на версию 2.0")

    async def handle_webhook(self, payload) -> dict:
        raise NotImplementedError("Реализовать при переходе на версию 2.0")
```

backend/app/tasks/__init__.py (1 строк)

```

```

backend/app/tasks/celery_app.py (6 строк)

```
from celery import Celery

from app.core.config import settings

celery = Celery("creditor", broker=settings.REDIS_URL, backend=settings.REDIS_URL)
celery.conf.task_always_eager = settings.CELERY_TASK_ALWAYS_EAGER
```

backend/app/tasks/parse_case.py (10 строк)

```
"""Фоновый разбор дела: обновляет данные источников."""
from app.services.parsers import fetch_case_bundle
from app.tasks.celery_app import celery


@celery.task(name="parse_case")
def parse_case(inn: str | None = None, case_number: str | None = None) -> dict:
    import asyncio
    b = asyncio.run(fetch_case_bundle(inn=inn, case_number=case_number))
    return {"found": b.get("found"), "status": b.get("status")}
```

backend/app/tasks/recalculate.py (18 строк)

```
"""Пересчёт профилей. Правило мёртвой зоны: изменение p < 0,05–0,10 не пересчитывает приоритеты."""
from app.tasks.celery_app import celery


def dead_zone(n_cases: int) -> float:
    return 0.10 if n_cases < 30 else 0.05


def is_critical(old: float, new: float) -> bool:
    """Критическое изменение > 0,15 либо пересечение порога (0,70 / 0,75 / 0,80)."""
    if abs(new - old) > 0.15:
        return True
    return any((old - t) * (new - t) < 0 for t in (0.70, 0.75, 0.80))


@celery.task(name="recalculate")
def recalculate(profile_id: int) -> dict:
    return {"profile_id": profile_id, "status": "queued"}
```

backend/app/tasks/send_email.py (8 строк)

```
from app.tasks.celery_app import celery


@celery.task(name="send_email")
def send_email(to: str, subject: str, body: str) -> dict:
    import asyncio
    from app.services.email import EmailService
    return asyncio.run(EmailService().send(to, subject, body))
```

backend/app/tasks/update_stats.py (15 строк)

```
"""Обновление статистики судьи: Δ_нов = β·((удовл_судья+новый)/(всего_судья+1) −
(удовл_суд+новый)/(всего_суд+1))."""
```

```
from app.tasks.celery_app import celery


def delta_new(j_ok: int, j_all: int, c_ok: int, c_all: int, new: int, beta: float = 0.6) -> float:
    if j_all < 10:
        return 0.0   # < 10 дел — не обновляется
    d = beta * ((j_ok + new) / (j_all + 1) - (c_ok + new) / (c_all + 1))
    weight = 1.0 if j_all > 30 else (j_all - 10) / 20      # 10–30 дел: вес растёт
    return d * weight


@celery.task(name="update_stats")
def update_stats(judge_id: int) -> dict:
    return {"judge_id": judge_id, "status": "queued"}
```

Backend: миграции, тесты, конфигурация

backend/alembic/env.py (32 строк)

```
"""Alembic (async). URL берётся из DATABASE_URL (app.core.config)."""
import asyncio

from alembic import context
from sqlalchemy.ext.asyncio import create_async_engine

import app.db.models  # noqa: F401
from app.core.config import settings
from app.db.base import Base

target_metadata = Base.metadata


def _run(connection):
    context.configure(connection=connection, target_metadata=target_metadata, compare_type=True)
    with context.begin_transaction():
        context.run_migrations()


async def run_online():
    engine = create_async_engine(settings.DATABASE_URL)
    async with engine.connect() as conn:
        await conn.run_sync(_run)
    await engine.dispose()


if context.is_offline_mode():
    context.configure(url=settings.DATABASE_URL, target_metadata=target_metadata, literal_binds=True)
    with context.begin_transaction():
        context.run_migrations()
else:
    asyncio.run(run_online())
```

backend/alembic/script.py.mako (21 строк)

```
"""${message}

Revision ID: ${up_revision}
Revises: ${down_revision | comma,n}
"""
from alembic import op
import sqlalchemy as sa
${imports if imports else ""}

revision = ${repr(up_revision)}
down_revision = ${repr(down_revision)}
branch_labels = None
depends_on = None


def upgrade() -> None:
    ${upgrades if upgrades else "pass"}


def downgrade() -> None:
    ${downgrades if downgrades else "pass"}
```

backend/alembic/versions/0001_initial.py (25 строк)

```
"""Начальная схема: все 8 слоёв таблиц (31 таблица) + администратор.

Схема строится из метаданных моделей на момент выпуска v1.0. Все последующие изменения —
обычными миграциями `alembic revision --autogenerate`.

Revision ID: 0001
Revises:
"""
from alembic import op

import app.db.models  # noqa: F401
from app.db.base import Base

revision = "0001"
down_revision = None
branch_labels = None
depends_on = None


def upgrade() -> None:
    Base.metadata.create_all(bind=op.get_bind())


def downgrade() -> None:
    Base.metadata.drop_all(bind=op.get_bind())
```

backend/tests/conftest.py (26 строк)

```
import os
import tempfile

_tmp = tempfile.mkdtemp()
# TEST_DATABASE_URL=postgresql+asyncpg://... — прогон тестов на PostgreSQL (по умолчанию SQLite)
os.environ.update(DATABASE_URL=os.environ.get("TEST_DATABASE_URL",
f"sqlite+aiosqlite:///{_tmp}/test.db"), ENVIRONMENT="test", PARSERS_MODE="mock",
                  REDIS_URL="redis://127.0.0.1:1/0", STORAGE_PATH=f"{_tmp}/storage",
                  RATE_LIMITING_ENABLED="false", SECRET_KEY="test-secret-key-0123456789abcdef-xyz")

import pytest  # noqa: E402
from httpx import ASGITransport, AsyncClient  # noqa: E402

from app.main import app  # noqa: E402


@pytest.fixture
async def client():
    async with app.router.lifespan_context(app):
        async with AsyncClient(transport=ASGITransport(app=app), base_url="http://test") as c:
            yield c


@pytest.fixture
async def auth(client):
    r = await client.post("/api/v1/auth/login", json={"email": "admin@local", "password": "admin"})
    return {"Authorization": f"Bearer {r.json()['access_token']}"}
```

backend/tests/test_api.py (202 строк)

```
import io

from app.core.logging import mask


```

```python
async def test_health_ready_metrics(client):
    r = await client.get("/health")
    assert r.status_code == 200 and r.json()["db"] == "ok"
    assert (await client.get("/ready")).json() == {"ready": True}
    assert (await client.get("/metrics")).status_code == 200


async def test_auth_required(client):
    r = await client.get("/api/v1/users/me")
    assert r.status_code == 401 and "request_id" in r.json()


async def test_login_wrong_password(client):
    r = await client.post("/api/v1/auth/login", json={"email": "admin@local", "password": "nope"})
    assert r.status_code == 401


async def test_register_disabled(client):
    r = await client.post("/api/v1/auth/register", json={"email": "a@b.ru", "password": "x"})
    assert r.status_code == 403 and "Регистрация временно недоступна" in r.json()["error"]


async def test_refresh_and_me(client):
    t = (await client.post("/api/v1/auth/login", json={"email": "admin@local", "password": "admin"})).json()
    new = (await client.post("/api/v1/auth/refresh", json={"refresh_token": t["refresh_token"]})).json()
    me = await client.get("/api/v1/users/me", headers={"Authorization": f"Bearer {new['access_token']}"})
    assert me.json()["tier"] == "admin" and me.json()["flags"]["billing_enabled"] is False
    assert (await client.post("/api/v1/auth/refresh", json={"refresh_token": t["access_token"]})).status_code == 401


async def test_subscription_stub(client, auth):
    r = await client.get("/api/v1/users/me/subscription", headers=auth)
    assert r.json()["message"] == "Оплата скоро"


async def test_analyze_validation(client, auth):
    assert (await client.post("/api/v1/analyze", json={}, headers=auth)).status_code == 422
    assert (await client.post("/api/v1/analyze", json={"inn": "123"}, headers=auth)).status_code == 422


async def test_full_cycle(client, auth):
    body = {"inn": "7701234567", "claim": {"amount": 10_000_000, "executed_amount": 2_000_000,
                                           "execution_date": "2026-09-01", "claim_date": "2025-06-01"}}
    r = await client.post("/api/v1/analyze", json=body, headers=auth)
    assert r.status_code == 200, r.text
    s = r.json()
    assert s["demo_data"] is True and s["mode"] in {"P3", "P4", "P5", "P6"}
    assert 0 <= s["expected_pct"] <= 1 and len(s["risks"]) == 3
    assert s["recommendation"] and "Сервис носит информационно-аналитический характер" in s["disclaimers"]["general"]
    pid = s["id"]
    assert (await client.get(f"/api/v1/analysis/{pid}", headers=auth)).json()["id"] == pid
    d = (await client.get(f"/api/v1/analysis/{pid}/details", headers=auth)).json()
    assert d["scenarios"] and d["checklist"] and "au" in d["disclaimers"]
    a = (await client.get(f"/api/v1/analysis/{pid}/audit", headers=auth)).json()
    assert a["formulas"] and a["probabilities"]["p_ch"] is not None and "practice" in a["disclaimers"]
    reg = (await client.get(f"/api/v1/analysis/{pid}/registry", headers=auth)).json()
    assert reg["available"] == (s["mode"] in ("P5", "P6"))
    lv = (await client.get(f"/api/v1/cases/{s['case_id']}/levels", headers=auth)).json()
    assert set(lv["levels"]) == {"M0", "M1", "M2.1", "M2.2", "M3", "M4", "M5"}
    assert any(x["id"] == pid for x in (await client.get("/api/v1/analyses", headers=auth)).json())


async def test_no_case_gives_p1(client, auth):
    assert (await client.post("/api/v1/analyze", json={"inn": "7701234500"}, headers=auth)).json()["mode"] == "P2"
    r = await client.post("/api/v1/analyze", json={"inn": "7701234000"}, headers=auth)
```

```
    s = r.json()
    assert s["mode"] == "P1" and s["confidence"] == "low"
    assert any("демонстрационное" in a for a in s["assumptions"])


async def test_detect_mode_and_set_mode(client, auth):
    d = (await client.get("/api/v1/cases/detect-mode", params={"inn": "7701234567"},
headers=auth)).json()
    assert d["mode"].startswith("P")
    s = (await client.post("/api/v1/analyze", json={"inn": "7701234567"}, headers=auth)).json()
    r = await client.post(f"/api/v1/cases/{s['case_id']}/set-mode", json={"mode": "P6"}, headers=auth)
    assert r.status_code == 200 and r.json()["mode"] == "P6" and "вручную" in r.json()["warning"]
    assert r.json()["mode_manual"] is True


async def test_other_user_cannot_read(client, auth):
    s = (await client.post("/api/v1/analyze", json={"inn": "7701234567"}, headers=auth)).json()
    assert (await client.get("/api/v1/analysis/999999", headers=auth)).status_code == 404
    assert (await client.get(f"/api/v1/analysis/{s['id']}")).status_code == 401


async def test_api_keys_and_webhooks(client, auth):
    k = (await client.post("/api/v1/api-keys", json={"name": "t"}, headers=auth)).json()
    me = await client.get("/api/v1/users/me", headers={"X-API-Key": k["key"]})
    assert me.status_code == 200
    assert (await client.delete(f"/api/v1/api-keys/{k['id']}", headers=auth)).json()["revoked"]
    assert (await client.get("/api/v1/users/me", headers={"X-API-Key": k["key"]})).status_code == 401
    h = (await client.post("/api/v1/webhooks", json={"url": "https://example.com/h"},
headers=auth)).json()
    assert len((await client.get("/api/v1/webhooks", headers=auth)).json()) >= 1
    assert (await client.delete(f"/api/v1/webhooks/{h['id']}", headers=auth)).json()["deleted"]


async def test_documents_flow_raises_levels(client, auth):
    import docx
    s = (await client.post("/api/v1/analyze", json={"inn": "7701234567", "claim": {
        "amount": 5_000_000, "executed_amount": 1_000_000, "execution_date": "2026-09-01"}},
headers=auth)).json()
    before = s["id"]
    mime = "application/vnd.openxmlformats-officedocument.wordprocessingml.document"

    def make(*lines):
        buf = io.BytesIO()
        d = docx.Document()
        for ln in lines:
            d.add_paragraph(ln)
        d.save(buf)
        return buf.getvalue()

    contract = make("Договор поставки от 01.02.2025. Покупатель ИНН 7701234567. Поставщик отгрузил
товар.")
    payment = make("Платёжное поручение. Оплачено 15.08.2026 на сумму 1 000 000 руб.")
    ids = []
    for name, data in (("dogovor.docx", contract), ("pp.docx", payment)):
        r = await client.post(f"/api/v1/claims/{s['claim_id']}/documents", headers=auth,
                              files={"file": (name, data, mime)})
        assert r.status_code == 200 and r.json()["needs_confirmation"], r.text
        ids.append(r.json()["id"])
    st = (await client.get(f"/api/v1/documents/{ids[0]}/status", headers=auth)).json()
    assert st["facts"] and all(f["label"] == "требует проверки" for f in st["facts"])
    # пока подтверждён только один документ — уровень M2.2 не растёт до 5
    c1 = (await client.post(f"/api/v1/documents/{ids[0]}/confirm-facts", json={}, headers=auth)).json()
    mid = (await client.get(f"/api/v1/analysis/{c1['analysis_id']}/audit", headers=auth)).json()
    assert mid["information_levels"]["M2.2"] < 5
    c2 = (await client.post(f"/api/v1/documents/{ids[1]}/confirm-facts", json={}, headers=auth)).json()
    assert c2["confirmed"] == c2["total"] and c2["analysis_id"] != before
    new = (await client.get(f"/api/v1/analysis/{c2['analysis_id']}/audit", headers=auth)).json()
    assert new["information_levels"]["M2.2"] == 5
    assert len((await client.get(f"/api/v1/claims/{s['claim_id']}/documents", headers=auth)).json()) == 2
```

```
async def test_upload_rejects_bad_type(client, auth):
    s = (await client.post("/api/v1/analyze", json={"inn": "7701234567"}, headers=auth)).json()
    r = await client.post(f"/api/v1/claims/{s['claim_id']}/documents", headers=auth,
                          files={"file": ("x.exe", b"MZ", "application/octet-stream")})
    assert r.status_code == 415


def test_pii_masking():
    assert mask("user admin@local") == "user a***@local"
    assert mask("inn 7701234567") == "inn 7701****67"
    assert mask("mail ivan@example.com") == "mail i***@example.com"


async def test_change_password(client, auth):
    bad = await client.post("/api/v1/users/me/password", json={"old_password": "x", "new_password": "abcdef"}, headers=auth)
    assert bad.status_code == 400
    ok = await client.post("/api/v1/users/me/password", json={"old_password": "admin", "new_password": "abcdef"}, headers=auth)
    assert ok.json()["changed"]
    assert (await client.post("/api/v1/auth/login", json={"email": "admin@local", "password": "abcdef"})).status_code == 200
    # вернуть исходный пароль для остальных тестов
    from sqlalchemy import select
    from app.core.security import hash_password
    from app.db.models import User
    from app.db.session import SessionLocal
    async with SessionLocal() as db:
        u = (await db.execute(select(User).where(User.email == "admin@local"))).scalar_one()
        u.password_hash = hash_password("admin")
        await db.commit()


async def test_manual_input_overrides_and_fills_gaps(client, auth):
    body = {"inn": "7701234000", "manual": {
        "filing_date": "2026-06-01", "total_assets": 150e6, "current_expenses": 10e6, "sigma3": 120e6,
        "judge": {"n_cases": 45, "approval_rate": 0.70, "is_significant": True},
        "au": {"n_cases": 10, "filed_on_your_deal": True},
        "collateral": [{"C": 100e6, "L": 0.9}]},
        "claim": {"amount": 8e6, "executed_amount": 2e6, "execution_date": "2026-04-01"}, "mode": "P4"}
    r = await client.post("/api/v1/analyze", json=body, headers=auth)
    s = r.json()
    assert r.status_code == 200 and s["manual_data"] is True and s["status"] == "ok"
    a = (await client.get(f"/api/v1/analysis/{s['id']}/audit", headers=auth)).json()
    assert a["audit"]["M0"] == 150e6 and a["audit"]["sigma3"] == 120e6
    assert a["probabilities"]["p_ch"] == 0.75        # окно 6 мес. 0,50 + судья 0,15 + АУ 0,10, потолок 1
    # set-mode не теряет ручные данные
    cid = s["case_id"]
    r2 = await client.post(f"/api/v1/cases/{cid}/set-mode", json={"mode": "P5"}, headers=auth)
    a2 = (await client.get(f"/api/v1/analysis/{r2.json()['id']}/audit", headers=auth)).json()
    assert a2["audit"]["M0"] == 150e6


async def test_manual_input_validation(client, auth):
    r = await client.post("/api/v1/analyze", json={"inn": "7701234567", "manual": {"judge": {"n_cases": 5, "approval_rate": 2}}}, headers=auth)
    assert r.status_code == 422


async def test_local_autologin_flag(client, monkeypatch):
    from app.core.config import settings
    assert (await client.post("/api/v1/auth/local")).status_code == 403          # по умолчанию выключено
    monkeypatch.setattr(settings, "LOCAL_AUTOLOGIN", True)
    r = await client.post("/api/v1/auth/local")
    assert r.status_code == 200
    me = await client.get("/api/v1/users/me", headers={"Authorization": f"Bearer {r.json()['access_token']}"})
```

```
    assert me.json()["email"].startswith("admin")
    monkeypatch.setattr(settings, "PUBLIC_MODE", True)                                  # в публичном режиме запрещено всегда
    assert (await client.post("/api/v1/auth/local")).status_code == 403
```

backend/tests/test_calculator.py (79 строк)

```
from datetime import date

import pytest

from app.services.calculator import CalcInput, calculate

DOC = dict(amount=8e6, executed_amount=2e6, claim_date=date(2025, 1, 1), filing_date=date(2026, 3, 1),
           execution_date=date(2026, 1, 1), total_assets=150e6, current_expenses=10e6, sigma3=120e6)


def test_doc_chapter14_style():
    r = calculate(CalcInput(mode="P4", **DOC, judge_stats={"n_cases": 45, "approval_rate": 0.70, "is_significant": True}))
    assert r["p_ch"] == 0.65 and r["p_zero"] == 0.5 and r["p_sub"] == 0.1
    assert r["expected_amount"] == pytest.approx(5e6)
    assert sum(s["probability"] for s in r["scenarios"]) == pytest.approx(1, abs=1e-3)
    assert r["range_low"] <= r["expected_amount"] <= r["range_high"]
    assert any(a["key"] == "defend_deal" for a in r["actions"])
    assert r["actions"][0]["key"] == "registry"        # П0 всегда первым


def test_probabilistic_mode_uses_industry():
    r = calculate(CalcInput(mode="P1", amount=10e6))
    assert r["method"] == "probabilistic"
    assert r["expected_pct"] == pytest.approx(0.9 * 0.57 * 0.06 * 0.57, rel=1e-2)
    assert r["confidence"] == "low"
    assert r["range_high"] - r["range_low"] >= 0.9 * r["expected_amount"] - 1   # расширенный диапазон


def test_exact_mode_discounts():
    base = dict(amount=10e6, total_assets=500e6, sigma3=100e6)
    r4 = calculate(CalcInput(mode="P4", **base))
    r6 = calculate(CalcInput(mode="P6", **base))
    assert r6["method"] == "exact" and r6["expected_amount"] < r4["expected_amount"]


def test_subordination_zeroes_payout():
    r = calculate(CalcInput(mode="P4", amount=10e6, status="kdl", crisis=True, rescue_form=True,
                            unusual_conditions=True, total_assets=100e6, sigma3=200e6, sigma_sub=10e6))
    assert r["p_sub"] == 0.85
    assert any(a["key"] == "fight_subordination" for a in r["actions"])
    assert r["priorities"][1].startswith("П1")


def test_critical_au_flag():
    r = calculate(CalcInput(mode="P4", amount=10e6, total_assets=100e6, sigma3=50e6,
                            au_signals={"removed": True, "disqualification_filed": True}))
    assert r["p_zero"] >= 0.85 and r["expected_pct"] <= 0.05
    assert any(a["key"] == "remove_au" for a in r["actions"])


def test_secured_creditor():
    r = calculate(CalcInput(mode="P4", amount=80e6, construction_type="pledge", is_credit_contract=True,
                            claim_date=date(2025, 1, 1), filing_date=date(2026, 1, 1), total_assets=150e6,
                            current_expenses=10e6, collateral=[{"C": 100e6, "L": 0.9}], sigma3=120e6))
    assert r["queue"] == "secured" and r["m1"]["alpha"] == 0.8
    assert r["audit"]["G"] == pytest.approx(72e6)
    assert r["expected_pct"] > 0.5

```

```
def test_penalty_goes_to_third():
    r = calculate(CalcInput(mode="P4", amount=1e6, construction_type="penalty", total_assets=10e6,
sigma3=20e6))
    assert r["m0"]["passes"] is False and r["queue"] == "third"


def test_low_payout_only_registry():
    r = calculate(CalcInput(mode="P4", amount=300_000, total_assets=0, sigma3=50e6, executed_amount=0))
    assert r["p_zero"] == 0.85
    assert "П3" in " ".join(r["priorities"])
    assert [a["key"] for a in r["actions"]] == ["registry"]


def test_no_execution_means_no_challenge():
    r = calculate(CalcInput(mode="P4", amount=5e6, total_assets=100e6, sigma3=50e6))
    assert r["p_ch"] == 0


def test_unknown_data_adds_assumptions():
    r = calculate(CalcInput(mode="P3", amount=5e6))
    assert any("Σ3" in a for a in r["assumptions"]) and r["confidence"] == "low"
```

backend/tests/test_formulas.py (112 строк)

```
import pytest

from app.services.formulas import D, E_R, E_R_final, R_3, R_sec, R_sub, discount_factor


class TestF1:
    def test_doc_example(self):  # 100·10/120
        assert D(10, 100, 120) == pytest.approx(8.3333, rel=1e-3)

    def test_capped_by_claim(self):
        assert D(10, 1000, 120) == 10

    def test_zero_sigma(self):
        assert D(10, 100, 0) == 0

    def test_zero_pool(self):
        assert D(10, 0, 120) == 0

    def test_zero_claim(self):
        assert D(0, 100, 120) == 0

    def test_full_payout_when_pool_equals_sigma(self):
        assert D(10, 120, 120) == 10


class TestF2:
    def test_doc_example(self):  # G=72, остаток 8 → D(8, 68, 120)
        r = R_sec(80, 90, 0.8, 68, 120)
        assert r == pytest.approx(72 + 68 * 8 / 120)

    def test_pledge_covers_all(self):
        assert R_sec(50, 100, 0.8, 0, 100) == 50

    def test_no_collateral(self):
        assert R_sec(50, 0, 0.7, 10, 100) == pytest.approx(5)

    def test_alpha_70(self):
        assert R_sec(100, 100, 0.7, 0, 100) == 70

    def test_never_exceeds_claim(self):
        assert R_sec(10, 100, 0.8, 1000, 10) <= 10

```

```

class TestF3:
    def test_basic(self):
        assert R_3(10, 60, 120) == 5

    def test_full(self):
        assert R_3(10, 1000, 120) == 10

    def test_empty_mass(self):
        assert R_3(10, 0, 120) == 0

    def test_equals_D(self):
        assert R_3(7, 33, 99) == D(7, 33, 99)

    def test_tiny_share(self):
        assert R_3(1, 100, 1_000_000) == pytest.approx(1e-4)


class TestF4:
    def test_zero_when_independents_unpaid(self):
        assert R_sub(10, 100, 120, 10) == 0

    def test_gets_remainder(self):
        assert R_sub(10, 130, 120, 10) == 10

    def test_partial(self):
        assert R_sub(10, 125, 120, 20) == pytest.approx(2.5)

    def test_exact_boundary(self):
        assert R_sub(10, 120, 120, 10) == 0

    def test_zero_sub_sigma(self):
        assert R_sub(10, 200, 120, 0) == 0


class TestF5:
    def test_no_risks(self):
        assert E_R(0, 0, 10, 5, 0) == 10

    def test_certain_challenge(self):
        assert E_R(0, 1, 10, 5, 0) == 5

    def test_certain_subordination(self):
        assert E_R(1, 0, 10, 5, 2) == 2

    def test_weights_sum_to_one(self):
        assert E_R(0.3, 0.4, 1, 1, 1, 1) == pytest.approx(1)

    def test_mixed(self):
        assert E_R(0.1, 0.5, 10, 4, 0) == pytest.approx(0.9 * 0.5 * 10 + 0.5 * 0.9 * 4)

    def test_sub_after_ch(self):
        assert E_R(1, 1, 10, 5, 2, 3) == 3


class TestF6:
    def test_doc_example(self):
        assert E_R_final(5, 0.4) == 3

    def test_zero_pzero(self):
        assert E_R_final(5, 0) == 5

    def test_certain_zero(self):
        assert E_R_final(5, 1) == 0

    def test_monotonic(self):
        assert E_R_final(5, 0.2) > E_R_final(5, 0.6)

```

```
    def test_discount(self):
        assert discount_factor(0.1, 1) == pytest.approx(1 / 1.1)
```

backend/tests/test_infra.py (107 строк)

```
import pytest

from app.core import cache
from app.core.config import settings
from app.services.email import EmailService
from app.services.parsers import fetch_case_bundle
from app.services.parsers import mock as mockdata
from app.services.parsers.resilience import SourceUnavailable, kad_breaker, with_retry
from app.services.payments import get_payment_provider
from app.services.payments.yookassa import YooKassaProvider


async def test_mock_payment_and_yookassa_stub():
    p = get_payment_provider()
    r = await p.create_payment(1, 100, "pro")
    assert r["status"] == "mock_succeeded" and await p.check_status(r["payment_id"]) == "mock_succeeded"
    assert (await p.handle_webhook({"x": 1}))["handled"]
    with pytest.raises(NotImplementedError):
        await YooKassaProvider().create_payment(1, 1, "x")


async def test_email_is_logged_not_sent():
    r = await EmailService().send("a@b.ru", "s", "body")
    assert r == {"sent": False, "logged": True}


async def test_email_stays_mock_when_sending_disabled(monkeypatch):
    monkeypatch.setattr(settings, "EMAIL_PROVIDER", "smtp")
    assert (await EmailService().send("a@b.ru", "s", "b"))["sent"] is False


def test_mock_bundle_deterministic():
    assert mockdata.case_bundle("7701234567") == mockdata.case_bundle("7701234567")
    assert mockdata.case_bundle("7701234567") != mockdata.case_bundle("7702234567")
    assert mockdata.case_bundle("7701234000")["found"] is False


async def test_bundle_mock_has_demo_warning():
    b = await fetch_case_bundle(inn="7701234567")
    assert b["status"] == "ok" and "демонстрационные" in b["warnings"][0]


async def test_live_mode_degrades_to_partial(monkeypatch):
    monkeypatch.setattr(settings, "PARSERS_MODE", "live")
    monkeypatch.setattr(settings, "EFRSB_API_URL", "http://127.0.0.1:1")
    from app.services.parsers import kad

    async def boom(**kw):
        raise SourceUnavailable("kad down")
    monkeypatch.setattr(kad, "search_cases", boom)
    monkeypatch.setattr("app.services.parsers.resilience.asyncio.sleep", lambda s: _noop())
    b = await fetch_case_bundle(inn="7701234567")
    assert b["status"] == "partial" and b["manual_input_required"] and len(b["warnings"]) == 2


async def _noop():
    return None


async def test_retry_and_breaker(monkeypatch):
    monkeypatch.setattr("app.services.parsers.resilience.asyncio.sleep", lambda s: _noop())
    calls = {"n": 0}
```

```
    async def flaky():
        calls["n"] += 1
        if calls["n"] < 3:
            raise RuntimeError("x")
        return "ok"
    kad_breaker.close()
    assert await with_retry(flaky, kad_breaker) == "ok" and calls["n"] == 3

    async def bad():
        raise RuntimeError("down")
    kad_breaker.close()
    for _ in range(2):
        with pytest.raises(SourceUnavailable):
            await with_retry(bad, kad_breaker)
    with pytest.raises(SourceUnavailable, match="breaker"):     # открыт после 3 ошибок подряд
        await with_retry(bad, kad_breaker)
    kad_breaker.close()


async def test_cache_memory_fallback():
    await cache.cache_set("k1", {"a": 1}, 60)
    assert await cache.cache_get("k1") == {"a": 1}
    assert await cache.cache_get("missing") is None
    assert await cache.redis_ping() is False


async def test_rate_limit(client, auth, monkeypatch):
    monkeypatch.setattr(settings, "RATE_LIMITING_ENABLED", True)
    monkeypatch.setattr(settings, "RATE_LIMIT_PER_MINUTE", 3)
    codes = [(await client.get("/api/v1/users/me", headers=auth)).status_code for _ in range(5)]
    assert 429 in codes and codes[0] == 200
    assert (await client.get("/health")).status_code == 200      # health не лимитируется


async def test_register_works_when_public_signup(client, monkeypatch):
    monkeypatch.setattr(settings, "PUBLIC_SIGNUP", True)
    r = await client.post("/api/v1/auth/register", json={"email": "new@x.ru", "password": "secret1"})
    assert r.status_code == 200 and "access_token" in r.json()
    assert (await client.post("/api/v1/auth/register", json={"email": "new@x.ru", "password":
"s"})).status_code == 403


async def test_request_id_header(client):
    r = await client.get("/health", headers={"X-Request-ID": "abc123"})
    assert r.headers["x-request-id"] == "abc123"
```

backend/tests/test_modules.py (130 строк)

```
from datetime import date

import pytest

from app.services.level_detector import detect_levels
from app.services.mode_detector import mode_from_bundle
from app.services.modules.m0_filter import m0_filter
from app.services.modules.m1_queue import m1_queue
from app.services.modules.m2_1_subordination import m2_1_subordination as sub
from app.services.modules.m2_2_challenging import m2_2_challenging as ch
from app.services.modules.m3_economics import executive_fee, m3_economics
from app.services.modules.m4_practice import au_delta, au_independence_score, judge_delta
from app.tasks.recalculate import is_critical
from app.tasks.update_stats import delta_new


def test_m0():
    assert m0_filter("pledge")["passes"] and m0_filter("surety")["passes"]
```

```
    assert not m0_filter("penalty")["passes"] and not m0_filter("deposit")["passes"]
    assert m0_filter("supply") == {"passes": True, "queue": "third"}


def test_m1():
    d = date(2026, 1, 1)
    assert m1_queue(date(2026, 2, 1), d, "supply", False, False)["queue"] == "current"
    assert m1_queue(date(2025, 1, 1), d, "pledge", False, False) == {"queue": "secured", "alpha": 0.7}
    assert m1_queue(date(2025, 1, 1), d, "pledge", True, False)["alpha"] == 0.8
    assert m1_queue(date(2025, 1, 1), d, "loan", False, True)["queue"] == "third_subordinated"
    assert m1_queue(date(2025, 1, 1), d, "supply", False, False)["queue"] == "third"


@pytest.mark.parametrize("kw,expected", [
    (dict(status="kdl", crisis=True, rescue_form=True, unusual_conditions=True), 0.85),
    (dict(status="affiliated", crisis=True, rescue_form=True), 0.70),
    (dict(status="affiliated", crisis=True), 0.50),
    (dict(status="independent"), 0.10),
    (dict(status="kdl", crisis=True, rescue_form=True, exception_surety_before_crisis=True), 0.05),
    (dict(status="kdl", crisis=True, rescue_form=True, exception_non_voluntary=True), 0.05),
    (dict(status="kdl", crisis=True, rescue_form=True, exception_market_practice=True), 0.10),
    (dict(status="kdl", crisis=True, rescue_form=True, exception_majority_consent=True), 0.10),
    (dict(status="kdl", crisis=True, rescue_form=True, only_covenants=True), 0.15),
    (dict(status="kdl", crisis=True, rescue_form=True, only_covenants=True, beneficial_interest=True),
0.70),
])
def test_m2_1(kw, expected):
    assert sub(**kw)["p_sub"] == expected


def test_m2_2_windows():
    f = date(2026, 6, 1)
    big = dict(amount=5e6, total_assets=100e6)
    assert ch(None, f, **big)["p_ch"] == 0
    assert ch(date(2026, 5, 15), f, **big)["p_ch"] == 0.70
    assert ch(date(2026, 3, 1), f, **big)["p_ch"] == 0.50
    assert ch(date(2025, 10, 1), f, is_equivalent=False, **big)["p_ch"] == 0.60
    assert ch(date(2025, 10, 1), f, **big)["p_ch"] == 0.05
    assert ch(date(2024, 6, 1), f, is_gratuitous=True, **big)["p_ch"] == 0.40
    assert ch(date(2020, 6, 1), f, is_gratuitous=True, **big)["p_ch"] == 0.05


def test_m2_2_ordinary_activity():
    assert ch(date(2026, 5, 20), date(2026, 6, 1), amount=1e6, total_assets=100e6)["p_ch"] == 0.05
    assert ch(date(2026, 5, 20), date(2026, 6, 1), amount=1_000_001, total_assets=100e6)["p_ch"] == 0.70


def test_m3():
    r = m3_economics(150e6, 10e6, 0, 0, 0, [], None)
    assert r["M"] == 140e6 and r["p_zero"] == 0.50
    r = m3_economics(150e6, 10e6, 0, 0, 0, [{"C": 100e6, "L": 0.9}])
    assert r["total_collateral_net"] == pytest.approx(90e6) and r["p_zero"] == 0.20
    assert m3_economics(5, 10, 0, 0, 0, [])["p_zero"] == 0.85
    assert m3_economics(5, 10, 0, 0, 0, [])["M"] == 0


def test_executive_fee():
    assert executive_fee(1_000_000) == 120_000
    assert executive_fee(10_000) == 20_000
    assert executive_fee(10_000, is_citizen=True) == 2_000
    assert executive_fee(0) == 0


def test_m4_judge():
    assert not judge_delta({"n_cases": 9, "approval_rate": 0.9})["profile"]
    assert judge_delta({"n_cases": 45, "approval_rate": 0.70, "is_significant": True})["delta_ch"] ==
0.15
    assert judge_delta({"n_cases": 45, "approval_rate": 0.45, "is_significant": True})["delta_ch"] == 0
```

```
    assert judge_delta({"n_cases": 45, "approval_rate": 0.20, "is_significant": True})["delta_ch"] == pytest.approx(-0.138)
    assert judge_delta({"n_cases": 20, "approval_rate": 0.70, "is_significant": False})["delta_ch"] == 0.075
    assert judge_delta({"n_cases": 45, "approval_rate": 0.55, "is_significant": True})["delta_ch"] == pytest.approx(0.072)


def test_m4_au():
    assert not au_delta({"n_cases": 4})["profile"]
    assert au_delta({"n_cases": 10, "filed_on_your_deal": True})["delta_ch"] == 0.10
    assert au_delta({"n_cases": 10, "filed_similar": 2})["delta_ch"] == 0.06
    assert au_delta({"n_cases": 10, "filed_similar": 20})["delta_ch"] == 0.20      # потолок
    a = au_delta({"n_cases": 10, "passive_on_suspicious": True, "losses_recovered": 1, "rau_top20": True})
    assert a["delta_zero"] == pytest.approx(0.15)


def test_au_index():
    assert au_independence_score({})["score"] == 0
    r = au_independence_score({"removed": True, "disqualification_filed": True})
    assert r["score"] == 80 and r["critical"]
    assert au_independence_score({"removed": True})["critical"] is False   # 50 ≤ 60
    assert au_independence_score({k: True for k in ["removed", "disqualified", "sro_match"]})["score"] == 100


def test_mode_detector():
    assert mode_from_bundle({"found": False})["mode"] == "P1"
    assert mode_from_bundle({"found": False, "crisis_signs": True})["mode"] == "P2"
    m = lambda *t: {"found": True, "debtor": {"name": "x"}, "messages": [{"type": x} for x in t]}
    assert mode_from_bundle(m("case_opened"))["mode"] == "P3"
    assert mode_from_bundle(m("judge_appointed", "au_appointed"))["mode"] == "P4"
    assert mode_from_bundle(m("judge_appointed", "au_appointed", "registry_closed"))["mode"] == "P5"
    assert mode_from_bundle(m("registry_closed", "trading_started"))["mode"] == "P6"
    assert mode_from_bundle({"found": True, "messages": []})["mode"] == "P3"


def test_levels():
    assert detect_levels("P0", {})["M3"] == 0
    assert detect_levels("P4", {"has_assets": True})["M3"] == 3
    assert detect_levels("P5", {"has_assets": True, "has_collateral": True})["M3"] == 5
    assert detect_levels("P4", {"has_execution": True, "facts_confirmed": True, "has_payment_docs": True})["M2.2"] == 5


def test_recalc_rules():
    assert is_critical(0.60, 0.80)
    assert is_critical(0.69, 0.71)
    assert not is_critical(0.50, 0.55)
    assert delta_new(5, 8, 40, 100, 1) == 0
    assert delta_new(30, 45, 43, 100, 1) != 0
```

backend/.dockerignore (5 строк)

```
__pycache__
.pytest_cache
.coverage
*.db
storage
```

backend/.env.example (58 строк)

```
# ============================================================
# «Профиль кредитора» — конфигурация. Скопируйте в .env.
# Версия 1.0 → 2.0 отличается ТОЛЬКО значениями этих переменных.
# ============================================================
```

```
# --- Режим ---------------------------------------------------
PUBLIC_MODE=false          # true — публичный доступ (v2.0)
ENVIRONMENT=local          # local | prod. При local схема БД создаётся автоматически; в prod — alembic upgrade head

# --- Доступ --------------------------------------------------
PUBLIC_SIGNUP=false        # true — открыть /api/v1/auth/register
BILLING_ENABLED=false      # true — включить тарифы; false — всем tier="admin", /pricing показывает «Оплата скоро»
EMAIL_SENDING=false        # true — реально отправлять письма (нужен EMAIL_PROVIDER=smtp)
SSL_ENABLED=false          # true — HTTPS (nginx + certbot, см. README)

# --- Функции -------------------------------------------------
ANALYTICS_ENABLED=true     # внутренняя аналитика (таблица events)
METRICS_ENABLED=true       # Prometheus /metrics
RATE_LIMITING_ENABLED=true # лимит 100 запросов/мин на IP
RATE_LIMIT_PER_MINUTE=100
CIRCUIT_BREAKER_ENABLED=true
SHOW_COOKIE_BANNER=false   # показывать cookie-баннер (v2.0)

# --- Внешние сервисы -----------------------------------------
PARSERS_MODE=mock          # mock — демо-данные без сети; live — реальные ЕФРСБ и КАД Арбитр
EFRSB_API_URL=https://fedresurs.ru/api/v1
EFRSB_API_KEY=
KAD_BASE_URL=https://kad.arbitr.ru
PAYMENT_PROVIDER=mock      # mock | yookassa
EMAIL_PROVIDER=mock        # mock | smtp
# YOOKASSA_SHOP_ID=
# YOOKASSA_SECRET_KEY=
# SMTP_HOST=
# SMTP_USER=
# SMTP_PASSWORD=

# --- База данных и кэш ---------------------------------------
POSTGRES_PASSWORD=postgres # пароль PostgreSQL (docker-compose); в prod задайте свой
DATABASE_URL=postgresql+asyncpg://postgres:postgres@db:5432/creditor
REDIS_URL=redis://redis:6379/0

# --- Файлы ---------------------------------------------------
STORAGE_TYPE=local
STORAGE_PATH=/app/storage
MAX_FILE_SIZE_MB=20

# --- Безопасность --------------------------------------------
SECRET_KEY=change-me-in-production     # python -c "import secrets; print(secrets.token_urlsafe(48))"
ADMIN_PASSWORD=admin                    # пароль admin@local при первом создании; в v2.0 задайте свой
JWT_ALGORITHM=HS256
JWT_EXPIRE_MINUTES=1440
CORS_ORIGINS=http://localhost:5173,http://localhost:8080

# --- Публичный доступ (v2.0) ---------------------------------
DOMAIN=localhost

# --- Celery --------------------------------------------------
CELERY_TASK_ALWAYS_EAGER=true          # v1.0: фоновые задачи выполняются синхронно
```

backend/Dockerfile (12 строк)

```
FROM python:3.12-slim
WORKDIR /app
ENV PYTHONUNBUFFERED=1 PYTHONDONTWRITEBYTECODE=1
RUN apt-get update && apt-get install -y --no-install-recommends tesseract-ocr tesseract-ocr-rus curl \
    && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt \
    && playwright install --with-deps chromium
```

```
COPY . .
RUN mkdir -p /app/storage
EXPOSE 8000
CMD ["sh", "-c", "alembic upgrade head && uvicorn app.main:app --host 0.0.0.0 --port 8000"]
```

backend/alembic.ini (19 строк)

```
[alembic]
script_location = alembic
prepend_sys_path = .

[loggers]
keys = root
[handlers]
keys = console
[formatters]
keys = generic
[logger_root]
level = WARN
handlers = console
[handler_console]
class = StreamHandler
args = (sys.stderr,)
formatter = generic
[formatter_generic]
format = %(levelname)-5.5s [%(name)s] %(message)s
```

backend/pytest.ini (4 строк)

```
[pytest]
asyncio_mode = auto
testpaths = tests
filterwarnings = ignore::DeprecationWarning
```

backend/requirements-dev.txt (4 строк)

```
-r requirements.txt
pytest>=8
pytest-asyncio>=0.23
pytest-cov>=5
```

backend/requirements.txt (23 строк)

```
fastapi>=0.115
uvicorn[standard]>=0.30
sqlalchemy[asyncio]>=2.0
asyncpg>=0.29
aiosqlite>=0.20
alembic>=1.13
pydantic>=2.7
pydantic-settings>=2.3
email-validator>=2.1
python-multipart>=0.0.9
redis>=5.0
celery>=5.4
httpx>=0.27
playwright>=1.45
structlog>=24.1
prometheus-fastapi-instrumentator>=7.0
pdfplumber>=0.11
pytesseract>=0.3.10
python-docx>=1.1
pybreaker>=1.2
```

```
PyJWT>=2.8
bcrypt>=4.1
greenlet>=3.0
```

Frontend (React + TypeScript)

frontend/src/App.tsx (46 строк)

```tsx
import { Navigate, Outlet, Route, Routes } from "react-router-dom";
import { Layout } from "./components/Layout";
import { getToken } from "./services/api";
import Audit from "./pages/Audit";
import Cases from "./pages/Cases";
import Details from "./pages/Details";
import Documents, { DocumentsIndex } from "./pages/Documents";
import Home from "./pages/Home";
import Legal from "./pages/Legal";
import Login from "./pages/Login";
import Notifications from "./pages/Notifications";
import Pricing from "./pages/Pricing";
import Profile from "./pages/Profile";
import Registry from "./pages/Registry";
import Register from "./pages/Register";
import Summary from "./pages/Summary";

function Protected() {
  return getToken() ? <Layout><Outlet /></Layout> : <Navigate to="/login" replace />;
}

export default function App() {
  return (
    <Routes>
      <Route path="/login" element={<Login />} />
      <Route path="/register" element={<Register />} />
      <Route path="/terms" element={<Legal kind="terms" />} />
      <Route path="/privacy" element={<Legal kind="privacy" />} />
      <Route path="/offer" element={<Legal kind="offer" />} />
      <Route element={<Protected />}>
        <Route path="/" element={<Home />} />
        <Route path="/cases" element={<Cases />} />
        <Route path="/documents" element={<DocumentsIndex />} />
        <Route path="/notifications" element={<Notifications />} />
        <Route path="/profile" element={<Profile />} />
        <Route path="/pricing" element={<Pricing />} />
        <Route path="/analysis/:id" element={<Summary />} />
        <Route path="/analysis/:id/details" element={<Details />} />
        <Route path="/analysis/:id/audit" element={<Audit />} />
        <Route path="/analysis/:id/registry" element={<Registry />} />
        <Route path="/analysis/:id/documents" element={<Documents />} />
      </Route>
      <Route path="*" element={<Navigate to="/" replace />} />
    </Routes>
  );
}
```

frontend/src/components/AnalysisShell.tsx (25 строк)

```tsx
import { ArrowLeft } from "lucide-react";
import type { ReactNode } from "react";
import { Link, NavLink, useParams } from "react-router-dom";
import { cn } from "./ui";

const TABS = [["", "Сводка"], ["details", "Детали"], ["audit", "Аудит"], ["registry", "Реестр"],
["documents", "Документы"]];

export function AnalysisShell({ title, children }: { title: string; children: ReactNode }) {
  const { id } = useParams();
  return (
```

```
    <div>
      <Link to="/" className="mb-3 inline-flex items-center gap-1 text-sm text-muted hover:text-fg">
<ArrowLeft size={16} /> Назад</Link>
      <h1 className="mb-4 text-2xl font-semibold text-safe">{title}</h1>
      <nav className="-mx-4 mb-6 flex gap-1 overflow-x-auto px-4 md:mx-0 md:px-0" aria-label="Разделы
анализа">
        {TABS.map(([p, label]) => (
          <NavLink key={p} to={`/analysis/${id}${p ? "/" + p : ""}`} end
            className={({ isActive }) => cn("shrink-0 rounded-md px-3 py-2 text-sm", isActive ? "bg-
primary text-bg" : "text-muted hover:bg-surface2")}>
            {label}
          </NavLink>
        ))}
      </nav>
      {children}
    </div>
  );
}
```

frontend/src/components/CookieBanner.tsx (14 строк)

```
import { useState } from "react";
import { Button } from "./ui";

/** Показывается только при SHOW_COOKIE_BANNER=true (флаг приходит из /users/me). */
export function CookieBanner({ enabled }: { enabled: boolean }) {
  const [hidden, setHidden] = useState(() => { try { return !!localStorage.getItem("cp_cookie"); } catch
{ return false; } });
  if (!enabled || hidden) return null;
  return (
    <div role="dialog" aria-label="Cookie" className="fixed inset-x-0 bottom-16 z-40 mx-4 rounded-lg
border border-border bg-bg p-4 shadow-lg md:bottom-4 md:left-auto md:max-w-md">
      <p className="mb-3 text-sm">Мы используем cookie для работы сервиса и внутренней аналитики.</p>
      <Button onClick={() => { try { localStorage.setItem("cp_cookie", "1"); } catch { /* ignore */ }
setHidden(true); }}>Понятно</Button>
    </div>
  );
}
```

frontend/src/components/Disclaimer.tsx (19 строк)

```
const TEXT: Record<string, string> = {
  general: "Сервис носит информационно-аналитический характер. Не является юридической услугой. Прогнозы
вероятностны и не предсказывают исход конкретного дела. Не заменяет консультацию юриста.",
  practice: "Анализ судебной практики построен на опубликованных судебных актах. Не является оценкой
личности судьи или рекомендацией воздействия на суд.",
  au: "Признаки аффилированности АУ, требующие проверки: баллы отражают публичные факты и являются
аналитической гипотезой, а не утверждением о нарушении. Решение об отстранении принимает только суд.",
  pd: "Обрабатываются только общедоступные персональные данные (ст. 8 152-ФЗ). Вы вправе потребовать
уточнения или удаления (ст. 14).",
  fee: "Размер и очерёдность исполнительского сбора зависят от статуса основного долга. Расчёт носит
ориентировочный характер.",
  criminal: "Система выявляет публичные признаки рисков, но не квалифицирует действия как преступление.
Рекомендуется консультация профильного юриста.",
  ads: "Сервис соблюдает требования ст. 28.1 Закона «О рекламе». Материалы содержат обязательные указания
о негативных последствиях банкротства.",
};

/** Дисклеймеры раздела 17 ТЗ. Общий показывается всегда; остальные — по контексту экрана. */
export function Disclaimer({ also = [] }: { also?: (keyof typeof TEXT)[] }) {
  return (
    <aside className="mt-8 space-y-2 border-t border-border pt-4 text-xs text-muted text-safe" data-
testid="disclaimer">
      <p>{TEXT.general}</p>
      {also.map((k) => <p key={k}>{TEXT[k]}</p>)}
    </aside>
  );
}
```

frontend/src/components/Layout.tsx (84 строк)

```tsx
import { Bell, FileText, FolderOpen, Home, Menu, Settings, User, X, CreditCard } from "lucide-react";
import { useState, type ReactNode } from "react";
import { Link, NavLink, useNavigate } from "react-router-dom";
import { useMe } from "../hooks/useAuth";
import { setToken } from "../services/api";
import { cn } from "./ui";
import { CookieBanner } from "./CookieBanner";

const NAV = [
  { to: "/", label: "Главная", icon: Home, end: true },
  { to: "/cases", label: "Мои дела", icon: FolderOpen },
  { to: "/documents", label: "Документы", icon: FileText },
  { to: "/profile", label: "Настройки", icon: Settings },
  { to: "/pricing", label: "Подписка", icon: CreditCard },
];
const TABS = [
  { to: "/", label: "Главная", icon: Home, end: true },
  { to: "/cases", label: "Дела", icon: FolderOpen },
  { to: "/documents", label: "Документы", icon: FileText },
  { to: "/notifications", label: "Уведомл.", icon: Bell },
  { to: "/profile", label: "Профиль", icon: User },
];

const Logo = () => (
  <Link to="/" className="flex items-center gap-2 font-semibold text-fg">
    <img src="/icons/icon.svg" alt="" className="h-7 w-7" /> Профиль кредитора
  </Link>
);

export function Layout({ children }: { children: ReactNode }) {
  const { data: me } = useMe();
  const [open, setOpen] = useState(false);
  const nav = useNavigate();
  const logout = () => { setToken(null); nav("/login"); };
  const link = ({ isActive }: { isActive: boolean }) =>
    cn("flex items-center gap-3 rounded-md px-3 py-2 text-sm", isActive ? "bg-surface2 font-medium text-fg" : "text-muted hover:bg-surface2");

  return (
    <div className="min-h-screen lg:flex">
      {/* Десктоп: sidebar 280px */}
      <aside className="sticky top-0 hidden h-screen w-[280px] shrink-0 flex-col border-r border-border bg-surface p-4 lg:flex">
        <Logo />
        <nav className="mt-8 flex flex-1 flex-col gap-1">
          {NAV.map((n) => <NavLink key={n.to} to={n.to} end={n.end} className={link}><n.icon size={18} /> {n.label}</NavLink>)}
        </nav>
        <div className="border-t border-border pt-4 text-sm">
          <div className="text-safe font-medium">{me?.email}</div>
          <div className="text-muted">Тариф: {me?.tier}</div>
          <button onClick={logout} className="mt-2 text-info">Выйти</button>
        </div>
      </aside>

      <div className="min-w-0 flex-1">
        {/* Планшет: top bar + hamburger (скрыт на мобильном и десктопе) */}
        <header className="sticky top-0 z-30 hidden items-center justify-between border-b border-border bg-bg px-6 py-3 md:flex lg:hidden">
          <Logo />
          <button aria-label="Меню" onClick={() => setOpen(!open)} className="rounded-md p-2 hover:bg-surface2">
            {open ? <X /> : <Menu />}
          </button>
        </header>
        {open && (
```

```
          <nav className="hidden border-b border-border bg-surface p-4 md:grid md:grid-cols-2 lg:hidden">
            {NAV.map((n) => <NavLink key={n.to} to={n.to} end={n.end} onClick={() => setOpen(false)} className={link}><n.icon size={18} />{n.label}</NavLink>)}
            <button onClick={logout} className="px-3 py-2 text-left text-sm text-info">Выйти</button>
          </nav>
        )}
        <header className="flex items-center justify-between border-b border-border px-4 py-3 md:hidden"><Logo /></header>

        <main className="mx-auto w-full max-w-[1280px] px-4 pb-28 pt-6 md:px-6 md:pb-12 lg:px-8">{children}</main>
      </div>

      {/* Мобильный: bottom tab bar */}
      <nav aria-label="Основная навигация" className="fixed inset-x-0 bottom-0 z-30 grid grid-cols-5 border-t border-border bg-bg pb-[env(safe-area-inset-bottom)] md:hidden">
        {TABS.map((t) => (
          <NavLink key={t.to} to={t.to} end={t.end}
            className={({ isActive }) => cn("flex h-14 flex-col items-center justify-center gap-0.5 text-[11px]", isActive ? "text-fg font-medium" : "text-muted")}>
            <t.icon size={22} />{t.label}
          </NavLink>
        ))}
      </nav>
      <CookieBanner enabled={!!me?.flags.show_cookie_banner} />
    </div>
  );
}
```

frontend/src/components/Traffic.tsx (27 строк)

```
import { cn } from "./ui";
import { pct } from "../services/format";
import type { Risk } from "../services/api";

const DOT = { red: "bg-danger", yellow: "bg-warning", green: "bg-success" } as const;
const TXT = { red: "Высокий уровень", yellow: "Средний уровень", green: "Низкий уровень" } as const;

/** Светофор: цвет всегда дублируется текстом (не только цветом). */
export function Traffic({ risks }: { risks: Risk[] }) {
  return (
    <ul className="flex flex-col gap-3 md:flex-row md:gap-4" aria-label="Светофор рисков">
      {risks.map((r) => (
        <li key={r.key} className="flex flex-1 items-start gap-3 rounded-md border border-border bg-bg p-3 min-w-0">
          <span aria-hidden className={cn("mt-1.5 h-3 w-3 shrink-0 rounded-full", DOT[r.light])} />
          <div className="min-w-0">
            <div className="text-sm text-muted line-clamp-2">{r.title}</div>
            <div className="font-medium text-safe">
              {r.key === "mass" ? r.label : r.label}{" "}
              <span className="amount text-muted">({pct(r.p)})</span>
            </div>
            <span className="sr-only">{TXT[r.light]}</span>
          </div>
        </li>
      ))}
    </ul>
  );
}
```

frontend/src/components/ui.tsx (53 строк)

```
import clsx, { type ClassValue } from "clsx";
import { forwardRef, type ButtonHTMLAttributes, type HTMLAttributes, type InputHTMLAttributes, type ReactNode, type SelectHTMLAttributes } from "react";
import { twMerge } from "tailwind-merge";

```

```tsx
export const cn = (...c: ClassValue[]) => twMerge(clsx(c));

type BtnProps = ButtonHTMLAttributes<HTMLButtonElement> & { variant?: "primary" | "outline" | "ghost"; block?: boolean };
export const Button = forwardRef<HTMLButtonElement, BtnProps>(({ className, variant = "primary", block, ...p }, ref) => (
  <button ref={ref} {...p}
    className={cn("inline-flex items-center justify-center gap-2 rounded-md px-4 font-medium transition-colors",
      "h-11 md:h-10 disabled:opacity-50 disabled:pointer-events-none",
      block && "w-full md:w-auto",
      variant === "primary" && "bg-primary text-bg hover:bg-primary-hover",
      variant === "outline" && "border border-border bg-bg text-fg hover:bg-surface2",
      variant === "ghost" && "text-fg hover:bg-surface2", className)} />
));

export const Card = ({ className, ...p }: HTMLAttributes<HTMLDivElement>) => (
  <div {...p} className={cn("rounded-lg border border-border bg-surface p-4 md:p-6 shadow-sm min-w-0", className)} />
);

export const Input = forwardRef<HTMLInputElement, InputHTMLAttributes<HTMLInputElement>>(({ className, ...p }, ref) => (
  <input ref={ref} {...p} className={cn("h-11 md:h-10 w-full rounded-md border border-border bg-bg px-3 text-fg placeholder:text-muted", className)} />
));

export const Select = ({ className, ...p }: SelectHTMLAttributes<HTMLSelectElement>) => (
  <select {...p} className={cn("h-11 md:h-10 w-full rounded-md border border-border bg-bg px-3 text-fg", className)} />
);

/** Подпись + поле в одном <label>: поле программно связано с подписью (a11y). */
export const Field = ({ label, children, className }: { label: string; children: ReactNode; className?: string }) => (
  <label className={cn("block", className)}>
    <span className="mb-1 block text-sm text-muted">{label}</span>
    {children}
  </label>
);

export const Badge = ({ tone = "neutral", children }: { tone?: "neutral" | "success" | "warning" | "danger" | "info"; children: ReactNode }) => (
  <span className={cn("inline-block rounded-full px-2.5 py-0.5 text-xs font-medium",
    tone === "neutral" && "bg-surface2 text-muted", tone === "success" && "bg-success/15 text-success",
    tone === "warning" && "bg-warning/15 text-warning", tone === "danger" && "bg-danger/15 text-danger",
    tone === "info" && "bg-info/15 text-info")}>{children}</span>
);

export const Spinner = () => (
  <div role="status" aria-label="Загрузка" className="mx-auto my-12 h-8 w-8 animate-spin rounded-full border-2 border-border border-t-primary" />
);

export const ErrorBox = ({ error }: { error: unknown }) => (
  <Card role="alert" className="border-danger/40 text-danger">
    {error instanceof Error ? error.message : "Произошла ошибка"}
  </Card>
);
```

frontend/src/hooks/useAuth.ts (6 строк)

```tsx
import { useQuery } from "@tanstack/react-query";
import { api, getToken } from "../services/api";

export function useMe() {
  return useQuery({ queryKey: ["me"], queryFn: api.me, enabled: !!getToken(), staleTime: 60_000, retry: false });
```

```
}
```

frontend/src/hooks/useTheme.ts (13 строк)

```
import { useEffect, useState } from "react";

export type Theme = "light" | "dark";
const read = (): Theme => { try { return localStorage.getItem("cp_theme") === "dark" ? "dark" : "light"; } catch { return "light"; } };

export function useTheme() {
  const [theme, setTheme] = useState<Theme>(read);
  useEffect(() => {
    document.documentElement.dataset.theme = theme;
    try { localStorage.setItem("cp_theme", theme); } catch { /* ignore */ }
  }, [theme]);
  return { theme, setTheme };
}
```

frontend/src/main.tsx (20 строк)

```
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import { registerSW } from "virtual:pwa-register";
import App from "./App";
import "./styles/index.css";

try { document.documentElement.dataset.theme = localStorage.getItem("cp_theme") === "dark" ? "dark" : "light"; } catch { /* ignore */ }
registerSW({ immediate: true });

const client = new QueryClient({ defaultOptions: { queries: { retry: false, refetchOnWindowFocus: false } } });

ReactDOM.createRoot(document.getElementById("root")!).render(
  <React.StrictMode>
    <QueryClientProvider client={client}>
      <BrowserRouter><App /></BrowserRouter>
    </QueryClientProvider>
  </React.StrictMode>,
);
```

frontend/src/pages/Audit.tsx (67 строк)

```
import { useQuery } from "@tanstack/react-query";
import { useParams } from "react-router-dom";
import { AnalysisShell } from "../components/AnalysisShell";
import { Disclaimer } from "../components/Disclaimer";
import { Button, Card, ErrorBox, Spinner } from "../components/ui";
import { api } from "../services/api";
import { pct } from "../services/format";

function download(name: string, mime: string, content: string) {
  const a = document.createElement("a");
  a.href = URL.createObjectURL(new Blob([content], { type: mime }));
  a.download = name; a.click(); URL.revokeObjectURL(a.href);
}

export default function Audit() {
  const { id = "" } = useParams();
  const q = useQuery({ queryKey: ["audit", id], queryFn: () => api.audit(id) });
  if (q.isLoading) return <Spinner />;
  if (q.error || !q.data) return <ErrorBox error={q.error} />;
```

```
  const a = q.data;
  const p = a.probabilities;
  return (
    <AnalysisShell title="Аудит расчёта">
      <Card>
        <h2 className="mb-2 text-lg font-medium">Формулы</h2>
        <p className="mb-2 text-sm text-muted">Метод: {a.method === "probabilistic" ? "вероятностный" : a.method === "scenario" ? "сценарный" : "точный"} · {a.formula}</p>
        <pre className="overflow-x-auto whitespace-pre-wrap rounded-md bg-bg p-3 text-sm text-safe tabular-nums">{a.formulas.join("\n")}</pre>
      </Card>
      <Card className="mt-4">
        <h2 className="mb-2 text-lg font-medium">Вероятности</h2>
        <p className="text-sm text-muted">p_итог = clamp(p_база + Δ_практика + Δ_АУ, 0, 1)</p>
        <dl className="mt-2 grid grid-cols-2 gap-y-1 text-sm md:grid-cols-5">
          {[["p_sub", p.p_sub], ["p_ch", p.p_ch], ["p_zero", p.p_zero], ["Δ практика", p.delta_practice], ["Δ АУ", p.delta_au]].map(([k, v]) => (
            <div key={k as string}><dt className="text-muted">{k}</dt><dd className="amount font-medium">{(v as number).toFixed(2).replace(".", ",")}</dd></div>
          ))}
        </dl>
      </Card>
      <Card className="mt-4">
        <h2 className="mb-2 text-lg font-medium">Анализ судебной практики</h2>
        {a.judge ? (
          <ul className="space-y-1 text-sm">
            <li className="text-safe">{a.judge.name}</li>
            <li>N = <span className="amount">{a.judge.n_cases}</span> дел · доля удовлетворённых: <span className="amount">{pct(a.judge.approval_rate)}</span></li>
            {a.judge_delta.deviation != null && <li>Отклонение от средней по суду: <span className="amount">{(a.judge_delta.deviation * 100).toFixed(0)} п.п.</span></li>}
            <li className="text-muted">{a.judge_delta.note}</li>
          </ul>
        ) : <p className="text-sm text-muted">Судья не определён.</p>}
      </Card>
      <Card className="mt-4">
        <h2 className="mb-2 text-lg font-medium">Уровни полноты данных</h2>
        <div className="grid grid-cols-4 gap-2 text-sm md:grid-cols-7">
          {Object.entries(a.information_levels).map(([k, v]) => <div key={k} className="rounded-md bg-bg p-2 text-center"><div className="text-muted">{k}</div><div className="font-medium">{v}/5</div></div>)}
        </div>
      </Card>
      {a.assumptions.length > 0 && <Card className="mt-4"><h2 className="mb-2 text-lg font-medium">Допущения</h2><ul className="list-disc space-y-1 pl-5 text-sm text-safe">{a.assumptions.map((x) => <li key={x}>{x}</li>)}</ul></Card>}
      <Card className="mt-4">
        <h2 className="mb-2 text-lg font-medium">Источники</h2>
        <ul className="space-y-1 text-sm">{a.sources.map((s) => <li key={s.url}><a className="text-info" href={s.url} target="_blank" rel="noreferrer">{s.title}</a></li>)}</ul>
      </Card>
      <div className="mt-4 flex flex-col gap-2 md:flex-row">
        <Button block variant="outline" onClick={() => window.print()}>Экспорт PDF</Button>
        <Button block variant="outline" onClick={() => download(`analysis-${id}.json`, "application/json", JSON.stringify(a, null, 2))}>Экспорт JSON</Button>
      </div>
      <Disclaimer also={["practice", "au", "pd", "fee", "criminal"]} />
    </AnalysisShell>
  );
}
```

frontend/src/pages/Cases.tsx (30 строк)

```
import { useQuery } from "@tanstack/react-query";
import { Link } from "react-router-dom";
import { Disclaimer } from "../components/Disclaimer";
import { Badge, Card, ErrorBox, Spinner } from "../components/ui";
import { api } from "../services/api";
import { MODE_NAMES, pct, rub } from "../services/format";
```

```

export default function Cases() {
  const q = useQuery({ queryKey: ["analyses"], queryFn: api.analyses });
  if (q.isLoading) return <Spinner />;
  if (q.error) return <ErrorBox error={q.error} />;
  return (
    <div>
      <h1 className="mb-4 text-2xl font-semibold">Мои дела</h1>
      {q.data?.length === 0 && <p className="text-muted">Анализов пока нет. <Link className="text-info" to="/">Рассчитать</Link></p>}
      <div className="grid gap-3 md:grid-cols-2 lg:grid-cols-3">
        {q.data?.map((a) => (
          <Link key={a.id} to={`/analysis/${a.id}`}>
            <Card className="h-full hover:bg-surface2">
              <div className="line-clamp-1 font-medium">{a.case?.number ?? a.case?.debtor ?? `Анализ № ${a.id}`}</div>
              <div className="mt-1"><Badge tone="info">{MODE_NAMES[a.mode]}</Badge></div>
              <div className="amount mt-2 text-lg">{rub(a.expected_amount)} <span className="text-muted">({pct(a.expected_pct)})</span></div>
            </Card>
          </Link>
        ))}
      </div>
      <Disclaimer />
    </div>
  );
}
```

frontend/src/pages/Details.tsx (113 строк)

```
import { useQuery } from "@tanstack/react-query";
import { Calendar, CheckSquare, Square } from "lucide-react";
import { useState } from "react";
import { Bar, BarChart, Cell, ResponsiveContainer, Tooltip, XAxis, YAxis } from "recharts";
import { useParams } from "react-router-dom";
import { AnalysisShell } from "../components/AnalysisShell";
import { Disclaimer } from "../components/Disclaimer";
import { Badge, Card, ErrorBox, Spinner } from "../components/ui";
import { api } from "../services/api";
import { SCENARIO_COLORS, mln, pct, rub } from "../services/format";

export default function Details() {
  const { id = "" } = useParams();
  const q = useQuery({ queryKey: ["details", id], queryFn: () => api.details(id) });
  const [done, setDone] = useState<Record<string, boolean>>({});
  if (q.isLoading) return <Spinner />;
  if (q.error || !q.data) return <ErrorBox error={q.error} />;
  const d = q.data;
  const chart = d.scenarios.map((s) => ({ ...s, label: s.title, p: Math.round(s.probability * 100) }));

  return (
    <AnalysisShell title="Детали расчёта">
      <section className="space-y-3">
        <h2 className="text-lg font-medium">Сценарии</h2>
        <Card>
          <div className="h-56" aria-hidden>
            <ResponsiveContainer>
              <BarChart data={chart} layout="vertical" margin={{ left: 8, right: 16 }}>
                <XAxis type="number" domain={[0, 100]} unit="%" stroke="var(--color-text-muted)" fontSize={12} />
                <YAxis type="category" dataKey="label" width={120} stroke="var(--color-text-muted)" fontSize={11} />
                <Tooltip formatter={(v: number) => `${v}%`} />
                <Bar dataKey="p" radius={4}>{chart.map((s) => <Cell key={s.name} fill={SCENARIO_COLORS[s.name]} />)}</Bar>
              </BarChart>
            </ResponsiveContainer>
          </div>
```

```
          <ul className="mt-2 divide-y divide-border">
            {d.scenarios.map((s) => (
              <li key={s.name} className="flex items-center justify-between gap-3 py-2 text-sm">
                <span className="min-w-0 text-safe">{s.title}</span>
                <span className="amount shrink-0 text-muted">{pct(s.probability)}</span>
                <span className="amount w-24 shrink-0 text-right font-medium">{mln(s.payout_rub)}</span>
              </li>
            ))}
          </ul>
        </Card>
      </section>

      <section className="mt-8 space-y-3">
        <h2 className="text-lg font-medium">Приоритеты</h2>
        <ol className="list-decimal space-y-1 pl-5">{d.priorities.map((p) => <li key={p} className="text-safe">{p}</li>)}</ol>
      </section>

      <section className="mt-8 grid gap-4 md:grid-cols-2">
        <Card>
          <h2 className="mb-2 text-lg font-medium">Анализ судебной практики</h2>
          {d.judge ? (
            <div className="space-y-1 text-sm">
              <div className="font-medium text-safe">{d.judge.name}</div>
              <div className="text-muted">Дел: <span className="amount">{d.judge.n_cases}</span> · доля удовлетворённых: <span className="amount">{pct(d.judge.approval_rate)}</span></div>
              <Badge tone={d.judge.is_significant ? "success" : "warning"}>{d.judge.is_significant ? "статистически значимо" : "статистически незначимо"}</Badge>
            </div>
          ) : <p className="text-sm text-muted">Судья ещё не определён — используется статистика суда.</p>}
        </Card>
        <Card>
          <h2 className="mb-2 text-lg font-medium">Признаки аффилированности АУ, требующие проверки</h2>
          {d.au ? <div className="text-sm"><div className="font-medium text-safe">{d.au.name}, {d.au.sro}</div></div> : <p className="text-sm text-muted">Арбитражный управляющий не назначен.</p>}
          <div className="mt-2 text-sm text-muted">Баллы: <span className="amount font-medium text-fg">{d.au_index.score}</span> / 100 (флаг при &gt; 60)</div>
          <ul className="mt-1 list-disc pl-5 text-sm text-safe">{d.au_index.items.map((i) => <li key={i.label}>{i.label} (+{i.points})</li>)}</ul>
          {d.au_index.critical && <Badge tone="danger">Критический флаг — требуется судебная проверка</Badge>}
        </Card>
      </section>

      <section className="mt-8 space-y-3">
        <h2 className="text-lg font-medium">Стоимость действий</h2>
        <div className="grid gap-4 md:grid-cols-2">
          {d.actions.map((a) => (
            <Card key={a.key}>
              <div className="flex items-start justify-between gap-2"><div className="font-medium text-safe">{a.title}</div>
                {a.expected_gain_rub != null && <Badge tone={a.economically_justified ? "success" : "warning"}>{a.economically_justified ? "оправдано" : "не оправдано"}</Badge>}</div>
              <p className="mt-1 text-sm text-muted text-safe">{a.reason}</p>
              <dl className="mt-3 grid grid-cols-2 gap-y-1 text-sm">
                <dt className="text-muted">Госпошлина</dt><dd className="amount text-right">{rub(a.fee_rub)}</dd>
                <dt className="text-muted">Юристы</dt><dd className="amount text-right">{Math.round(a.lawyers_rub[0] / 1000)}–{Math.round(a.lawyers_rub[1] / 1000)} тыс. ₽</dd>
                <dt className="text-muted">Срок</dt><dd className="text-right">{a.months} мес.</dd>
                {a.expected_gain_rub != null && (<><dt className="text-muted">Прирост</dt><dd className="amount text-right font-medium">{rub(a.expected_gain_rub)}</dd></>)}
              </dl>
            </Card>
          ))}
        </div>
      </section>

      <section className="mt-8 grid gap-4 md:grid-cols-2">
```

```
        <Card>
          <h2 className="mb-2 text-lg font-medium">Чек-лист</h2>
          <ul className="space-y-2">
            {d.checklist.map((c) => (
              <li key={c}><button className="flex w-full items-start gap-2 text-left text-sm" onClick={() => setDone({ ...done, [c]: !done[c] })} aria-pressed={!!done[c]}>
                {done[c] ? <CheckSquare size={18} className="mt-0.5 shrink-0 text-success" /> : <Square size={18} className="mt-0.5 shrink-0 text-muted" />}
                <span className={done[c] ? "text-muted line-through text-safe" : "text-safe"}>{c}</span></button></li>
            ))}
          </ul>
        </Card>
        <Card>
          <h2 className="mb-2 text-lg font-medium">Календарь</h2>
          <ul className="space-y-2 text-sm">
            {d.calendar.map((c) => <li key={c.title} className="flex gap-2"><Calendar size={16} className="mt-0.5 shrink-0 text-muted" /><span className="text-safe">{c.date ? new Date(c.date).toLocaleDateString("ru-RU") + " — " : ""}{c.title}</span></li>)}
          </ul>
        </Card>
      </section>
      <Disclaimer also={["practice", "au", "fee", "criminal"]} />
    </AnalysisShell>
  );
}
```

frontend/src/pages/Documents.tsx (85 строк)

```
import { useMutation, useQuery, useQueryClient } from "@tanstack/react-query";
import { Upload } from "lucide-react";
import { useRef, useState } from "react";
import { Link, useNavigate, useParams } from "react-router-dom";
import { AnalysisShell } from "../components/AnalysisShell";
import { Disclaimer } from "../components/Disclaimer";
import { Badge, Button, Card, ErrorBox, Spinner } from "../components/ui";
import { api, type DocStatus } from "../services/api";

const TYPE_NAMES: Record<string, string> = { supply: "Договор поставки", loan: "Договор займа", pledge: "Договор залога",
  payment: "Платёжный документ", act: "Акт / накладная", claim_letter: "Претензия", other: "Документ" };

function DocCard({ id }: { id: number }) {
  const qc = useQueryClient();
  const nav = useNavigate();
  const q = useQuery({ queryKey: ["doc", id], queryFn: () => api.docStatus(id) });
  const confirm = useMutation({
    mutationFn: () => api.confirmFacts(id),
    onSuccess: (r) => { qc.invalidateQueries({ queryKey: ["doc", id] }); if (r.analysis_id) nav(`/analysis/${r.analysis_id}`); },
  });
  const d = q.data as DocStatus | undefined;
  if (!d) return null;
  const pending = d.facts.some((f) => !f.confirmed);
  return (
    <Card>
      <div className="flex items-center justify-between gap-2"><div className="font-medium">{TYPE_NAMES[d.document_type] ?? d.document_type}</div><Badge tone={d.status === "processed" ? "success" : "warning"}>{d.status}</Badge></div>
      {d.facts.length === 0 ? <p className="mt-2 text-sm text-muted">Факты не извлечены (возможно, нет текста или OCR недоступен).</p> : (
        <ul className="mt-3 divide-y divide-border text-sm">
          {d.facts.map((f) => (
            <li key={f.id} className="flex items-center justify-between gap-3 py-1.5">
              <span><span className="text-muted">{f.field}</span><span className="min-w-0 flex-1 text-right text-safe">{f.value}</span>
              <Badge tone={f.confirmed ? "success" : "warning"}>{f.label}</Badge>
            </li>))}
```

```
        </ul>
      )}
      {pending && <Button className="mt-3" block onClick={() => confirm.mutate()} disabled={confirm.isPending}>Подтверждаю факты</Button>}
      {pending && <p className="mt-2 text-xs text-muted">Пока факты не подтверждены, расчёт использует консервативные допущения.</p>}
    </Card>
  );
}

export default function Documents() {
  const { id = "" } = useParams();
  const qc = useQueryClient();
  const input = useRef<HTMLInputElement>(null);
  const [err, setErr] = useState<unknown>(null);
  const s = useQuery({ queryKey: ["summary", id], queryFn: () => api.summary(id) });
  const claimId = s.data?.claim_id;
  const list = useQuery({ queryKey: ["docs", claimId], queryFn: () => api.documents(claimId!), enabled: !!claimId });
  const up = useMutation({
    mutationFn: (f: File) => api.upload(claimId!, f),
    onSuccess: () => { setErr(null); qc.invalidateQueries({ queryKey: ["docs", claimId] }); },
    onError: setErr,
  });
  if (s.isLoading) return <Spinner />;
  if (s.error) return <ErrorBox error={s.error} />;
  return (
    <AnalysisShell title="Документы">
      <Card>
        <p className="mb-3 text-sm text-muted">PDF, DOCX, JPG, PNG до 20 МБ. Система извлечёт даты, суммы и условия; вы подтвердите факты вручную. Фактическая аффилированность и осведомлённость определяются только экспертно.</p>
        <input ref={input} type="file" hidden accept=".pdf,.docx,.jpg,.jpeg,.png" onChange={(e) => { const f = e.target.files?.[0]; if (f) up.mutate(f); e.target.value = ""; }} />
        <Button block onClick={() => input.current?.click()} disabled={up.isPending}><Upload size={16} /> {up.isPending ? "Обработка…" : "Загрузить документ"}</Button>
        {!!err && <div className="mt-3"><ErrorBox error={err} /></div>}
      </Card>
      <div className="mt-4 space-y-4">{list.data?.map((d) => <DocCard key={d.id} id={d.id} />)}</div>
      {list.data?.length === 0 && <p className="mt-4 text-sm text-muted">Документы пока не загружены.</p>}
      <Disclaimer also={["pd"]} />
    </AnalysisShell>
  );
}

/** /documents — выбор анализа, к которому относятся документы */
export function DocumentsIndex() {
  const q = useQuery({ queryKey: ["analyses"], queryFn: api.analyses });
  if (q.isLoading) return <Spinner />;
  return (
    <div>
      <h1 className="mb-4 text-2xl font-semibold">Документы</h1>
      <p className="mb-4 text-muted">Выберите анализ, к которому нужно загрузить документы.</p>
      <div className="space-y-2">{q.data?.map((a) => <Link key={a.id} to={`/analysis/${a.id}/documents`} className="block rounded-md border border-border bg-surface p-3 hover:bg-surface2 text-safe">Анализ № {a.id} — {a.case?.number ?? a.case?.debtor ?? "без дела"}</Link>)}</div>
      {q.data?.length === 0 && <p className="text-muted">Сначала выполните анализ на главной.</p>}
      <Disclaimer />
    </div>
  );
}
```

frontend/src/pages/Home.tsx (98 строк)

```
import { useMutation } from "@tanstack/react-query";
import { Check } from "lucide-react";
import { useState } from "react";
```

```
import { useNavigate } from "react-router-dom";
import { Disclaimer } from "../components/Disclaimer";
import { Button, Card, ErrorBox, Input, Field, Select } from "../components/ui";
import { api, type AnalyzeBody } from "../services/api";

const CONSTR = [["supply", "Поставка / услуги"], ["loan", "Заём"], ["pledge", "Залог"], ["surety", "Поручительство"],
  ["guarantee", "Независимая гарантия"], ["escrow", "Эскроу"], ["title", "Сохранение права собственности"],
  ["penalty", "Неустойка"], ["deposit", "Задаток"]];

export default function Home() {
  const nav = useNavigate();
  const [q, setQ] = useState("");
  const [more, setMore] = useState(false);
  const [caseOpen, setCaseOpen] = useState(false);
  const [c, setC] = useState({ filing: "", assets: "", expenses: "", sigma3: "", jN: "", jRate: "", jSig: false, colC: "", colL: "0.6" });
  const [f, setF] = useState({ amount: "", executed: "", execDate: "", construction: "supply", status: "independent", credit: false });
  const m = useMutation({
    mutationFn: (b: AnalyzeBody) => api.analyze(b),
    onSuccess: (s) => nav(`/analysis/${s.id}`),
  });
  const digits = q.replace(/\s/g, "");
  const valid = /^(\d{10}|\d{12})$/.test(digits) || /^[AA]\d{2}-\d+\/\d{4}$/i.test(q.trim());

  const submit = (e: React.FormEvent) => {
    e.preventDefault();
    const isInn = /^\d{10,12}$/.test(digits);
    const claim = {
      amount: f.amount ? Number(f.amount) : undefined,
      executed_amount: f.executed ? Number(f.executed) : undefined,
      execution_date: f.execDate || undefined,
      construction_type: f.construction, status: f.status,
    };
    const num = (v: string) => (v ? Number(v) : undefined);
    const manual = {
      filing_date: c.filing || undefined, total_assets: num(c.assets), current_expenses: num(c.expenses), sigma3: num(c.sigma3),
      judge: c.jN && c.jRate ? { n_cases: Number(c.jN), approval_rate: Number(c.jRate) / 100, is_significant: c.jSig } : undefined,
      collateral: c.colC ? [{ C: Number(c.colC), L: Number(c.colL) }] : undefined,
    };
    const hasManual = Object.values(manual).some((v) => v !== undefined);
    m.mutate({ ...(isInn ? { inn: digits } : { case_number: q.trim() }), claim, ...(hasManual ? { manual } : {}) });
  };

  return (
    <div className="mx-auto max-w-2xl">
      <h1 className="text-3xl font-semibold">Профиль кредитора</h1>
      <p className="mt-2 text-muted">Узнайте, сколько вы получите при банкротстве должника</p>
      <Card className="mt-6">
        <form onSubmit={submit} className="space-y-4">
          <Field label="ИНН должника или номер дела"><Input value={q} onChange={(e) => setQ(e.target.value)} placeholder="7701234567 или A40-12345/2025" inputMode="text" aria-label="ИНН или номер дела" autoFocus /></Field>
          <button type="button" className="text-sm text-info" onClick={() => setMore(!more)} aria-expanded={more}>
            {more ? "Скрыть параметры требования" : "Указать параметры вашего требования (точнее прогноз)"}
          </button>
          {more && (
            <div className="grid gap-4 md:grid-cols-2">
              <Field label="Сумма требования (неоплаченный остаток), ₽"><Input type="number" min="0" value={f.amount} onChange={(e) => setF({ ...f, amount: e.target.value })} placeholder="10000000" />
</Field>
```

```
              <Field label="Тип конструкции"><Select value={f.construction} onChange={(e) => setF({ ...f, construction: e.target.value })}>{CONSTR.map(([v, l]) => <option key={v} value={v}>{l}</option>)}</Select></Field>
              <Field label="Получено исполнение, ₽"><Input type="number" min="0" value={f.executed} onChange={(e) => setF({ ...f, executed: e.target.value })} placeholder="2000000" /></Field>
              <Field label="Дата исполнения"><Input type="date" value={f.execDate} onChange={(e) => setF({ ...f, execDate: e.target.value })} /></Field>
              <Field label="Связь с должником"><Select value={f.status} onChange={(e) => setF({ ...f, status: e.target.value })}>
                  <option value="independent">Независимый кредитор</option><option value="affiliated">Аффилированное лицо</option><option value="kdl">Контролирующее лицо (КДЛ)</option>
                </Select></Field>
            </div>
          )}
          <button type="button" className="block text-sm text-info" onClick={() => setCaseOpen(!caseOpen)} aria-expanded={caseOpen}>
            {caseOpen ? "Скрыть данные дела" : "Ввести данные дела вручную (если источники недоступны)"}
          </button>
          {caseOpen && (
            <div className="grid gap-4 md:grid-cols-2">
              <Field label="Дата принятия заявления"><Input type="date" value={c.filing} onChange={(e) => setC({ ...c, filing: e.target.value })} /></Field>
              <Field label="Активы должника, ₽"><Input type="number" min="0" value={c.assets} onChange={(e) => setC({ ...c, assets: e.target.value })} /></Field>
              <Field label="Текущие расходы, ₽"><Input type="number" min="0" value={c.expenses} onChange={(e) => setC({ ...c, expenses: e.target.value })} /></Field>
              <Field label="Требования третьей очереди (Σ3), ₽"><Input type="number" min="0" value={c.sigma3} onChange={(e) => setC({ ...c, sigma3: e.target.value })} /></Field>
              <Field label="Судья: число дел"><Input type="number" min="0" value={c.jN} onChange={(e) => setC({ ...c, jN: e.target.value })} /></Field>
              <Field label="Судья: доля удовлетворённых, %"><Input type="number" min="0" max="100" value={c.jRate} onChange={(e) => setC({ ...c, jRate: e.target.value })} /></Field>
              <Field label="Залог: стоимость, ₽"><Input type="number" min="0" value={c.colC} onChange={(e) => setC({ ...c, colC: e.target.value })} /></Field>
              <Field label="Залог: ликвидность (0–1)"><Input type="number" min="0.01" max="1" step="0.05" value={c.colL} onChange={(e) => setC({ ...c, colL: e.target.value })} /></Field>
            </div>
          )}
          {m.error && <ErrorBox error={m.error} />}
          <Button type="submit" block disabled={!valid || m.isPending}>{m.isPending ? "Считаем…" : "Рассчитать →"}</Button>
          {q && !valid && <p className="text-xs text-muted">Введите ИНН (10 или 12 цифр) или номер дела вида А40-12345/2025</p>}
        </form>
      </Card>
      <div className="mt-6">
        <div className="mb-2 font-medium">Что вы узнаете:</div>
        <ul className="space-y-1.5 text-muted">
          {["Очередь и приоритет", "Риски оспаривания и субординации", "Ожидаемую выплату в рублях", "Рекомендации и стоимость действий"].map((t) => (
            <li key={t} className="flex items-center gap-2"><Check size={16} className="text-success" /> {t}</li>
          ))}
        </ul>
      </div>
      <Disclaimer also={["ads"]} />
    </div>
  );
}
```

frontend/src/pages/Legal.tsx (19 строк)

```
import { Disclaimer } from "../components/Disclaimer";
import { Card } from "../components/ui";

const DOCS: Record<string, [string, string]> = {
  terms: ["Условия использования", "Заглушка. Полный текст пользовательского соглашения будет опубликован к запуску версии 2.0. Сервис носит информационно-аналитический характер и не является юридической услугой."],
```

```
  privacy: ["Политика конфиденциальности", "Заглушка. Сервис обрабатывает только общедоступные персональные данные (ст. 8 152-ФЗ). Вы вправе потребовать уточнения или удаления данных (ст. 14). Полная политика будет опубликована к версии 2.0."],
  offer: ["Публичная оферта", "Заглушка. Условия оказания платных услуг будут опубликованы при включении оплаты (BILLING_ENABLED=true)."],
};

export default function Legal({ kind }: { kind: "terms" | "privacy" | "offer" }) {
  const [title, text] = DOCS[kind];
  return (
    <div className="mx-auto max-w-2xl px-4 py-8 md:px-0">
      <h1 className="mb-4 text-2xl font-semibold">{title}</h1>
      <Card><p className="text-safe">{text}</p></Card>
      <Disclaimer also={["pd"]} />
    </div>
  );
}
```

frontend/src/pages/Login.tsx (40 строк)

```
import { useMutation, useQueryClient } from "@tanstack/react-query";
import { useEffect, useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import { Disclaimer } from "../components/Disclaimer";
import { Button, Card, ErrorBox, Input, Field } from "../components/ui";
import { api, setToken } from "../services/api";

export default function Login() {
  const nav = useNavigate();
  const qc = useQueryClient();
  const [email, setEmail] = useState("admin@local");
  const [password, setPassword] = useState("");
  const m = useMutation({
    mutationFn: () => api.login(email, password),
    onSuccess: (t) => { setToken(t.access_token); qc.invalidateQueries({ queryKey: ["me"] }); nav("/"); },
  });
  // Десктоп-версия: сервер разрешает вход без пароля — сразу заходим
  const [auto, setAuto] = useState(true);
  useEffect(() => {
    api.localLogin()
      .then((t) => { setToken(t.access_token); qc.invalidateQueries({ queryKey: ["me"] }); nav("/", { replace: true }); })
      .catch(() => setAuto(false));
  }, []); // eslint-disable-line react-hooks/exhaustive-deps
  if (auto) return null;
  return (
    <div className="mx-auto mt-12 max-w-sm px-4">
      <h1 className="mb-6 text-2xl font-semibold">Вход</h1>
      <Card>
        <form className="space-y-4" onSubmit={(e) => { e.preventDefault(); m.mutate(); }}>
          <Field label="Email"><Input value={email} onChange={(e) => setEmail(e.target.value)} autoComplete="username" /></Field>
          <Field label="Пароль"><Input type="password" value={password} onChange={(e) => setPassword(e.target.value)} autoComplete="current-password" /></Field>
          {m.error && <ErrorBox error={m.error} />}
          <Button type="submit" block disabled={m.isPending}>Войти</Button>
        </form>
        <p className="mt-4 text-sm text-muted">Нет аккаунта? <Link className="text-info" to="/register">Регистрация</Link></p>
      </Card>
      <Disclaimer />
    </div>
  );
}
```

frontend/src/pages/Notifications.tsx (11 строк)

```
import { Disclaimer } from "../components/Disclaimer";

export default function Notifications() {
  return (
    <div>
      <h1 className="mb-4 text-2xl font-semibold">Уведомления</h1>
      <p className="text-muted">Уведомлений пока нет. Критические изменения вероятностей (&gt; 0,15 или пересечение порога) появятся здесь; email-рассылка в версии 1.0 отключена (mock).</p>
      <Disclaimer />
    </div>
  );
}
```

frontend/src/pages/Pricing.tsx (29 строк)

```
import { useQuery } from "@tanstack/react-query";
import { Disclaimer } from "../components/Disclaimer";
import { Card } from "../components/ui";
import { api } from "../services/api";
import { useMe } from "../hooks/useAuth";

const PLANS = [
  ["Сводка", "Бесплатно", "Светофор, диапазон выплаты, одно действие"],
  ["Детали", "Подписка", "Сценарии, профиль АУ, пороги, стоимость действий"],
  ["Аудит", "Премиум", "Формулы, доверительные интервалы, ссылки на акты, выгрузка"],
];

export default function Pricing() {
  const { data: me } = useMe();
  const sub = useQuery({ queryKey: ["sub"], queryFn: api.subscription });
  const disabled = me ? !me.flags.billing_enabled : true;
  return (
    <div>
      <h1 className="mb-4 text-2xl font-semibold">Тарифы</h1>
      {disabled && <Card className="mb-4" role="status"><div className="text-lg font-medium">Оплата скоро</div><p className="text-sm text-muted">В версии 1.0 все функции доступны без оплаты (тариф «{sub.data?.tier ?? "admin"}»).</p></Card>}
      <div className="grid gap-4 md:grid-cols-3">
        {PLANS.map(([n, price, d]) => (
          <Card key={n}><div className="text-lg font-medium">{n}</div><div className="text-muted">{price}</div><p className="mt-2 text-sm text-safe">{d}</p></Card>
        ))}
      </div>
      <Disclaimer also={["ads"]} />
    </div>
  );
}
```

frontend/src/pages/Profile.tsx (43 строк)

```
import { useMutation } from "@tanstack/react-query";
import { useState } from "react";
import { Disclaimer } from "../components/Disclaimer";
import { Button, Card, ErrorBox, Input, Field } from "../components/ui";
import { useMe } from "../hooks/useAuth";
import { useTheme } from "../hooks/useTheme";
import { api } from "../services/api";

export default function Profile() {
  const { data: me } = useMe();
  const { theme, setTheme } = useTheme();
  const [oldP, setOld] = useState("");
  const [newP, setNew] = useState("");
```

```
  const pw = useMutation({ mutationFn: () => api.changePassword(oldP, newP), onSuccess: () => { setOld(""); setNew(""); } });
  return (
    <div className="max-w-xl space-y-4">
      <h1 className="text-2xl font-semibold">Профиль</h1>
      <Card>
        <div className="text-safe font-medium">{me?.email}</div>
        <div className="text-sm text-muted">Роль: {me?.role} · Тариф: {me?.tier}</div>
        {me && <div className="mt-1 text-xs text-muted">Источники данных: {me.flags.parsers_mode === "mock" ? "демо (mock)" : "КАД Арбитр и ЕФРСБ"}</div>}
      </Card>
      <Card>
        <h2 className="mb-2 font-medium">Тема</h2>
        <div className="flex gap-2">
          <Button variant={theme === "light" ? "primary" : "outline"} onClick={() => setTheme("light")}>Светлая</Button>
          <Button variant={theme === "dark" ? "primary" : "outline"} onClick={() => setTheme("dark")}>Тёмная</Button>
        </div>
      </Card>
      <Card>
        <h2 className="mb-2 font-medium">Смена пароля</h2>
        <form className="space-y-3" onSubmit={(e) => { e.preventDefault(); pw.mutate(); }}>
          <Field label="Текущий пароль"><Input type="password" value={oldP} onChange={(e) => setOld(e.target.value)} autoComplete="current-password" /></Field>
          <Field label="Новый пароль (от 6 символов)"><Input type="password" value={newP} onChange={(e) => setNew(e.target.value)} autoComplete="new-password" /></Field>
          {pw.error && <ErrorBox error={pw.error} />}
          {pw.isSuccess && <p className="text-sm text-success">Пароль изменён</p>}
          <Button type="submit" disabled={pw.isPending || newP.length < 6}>Сменить пароль</Button>
        </form>
      </Card>
      <Disclaimer also={["pd"]} />
    </div>
  );
}
```

frontend/src/pages/Register.tsx (32 строк)

```
import { useMutation } from "@tanstack/react-query";
import { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import { Disclaimer } from "../components/Disclaimer";
import { Button, Card, ErrorBox, Input, Field } from "../components/ui";
import { api, setToken } from "../services/api";

export default function Register() {
  const nav = useNavigate();
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const m = useMutation({
    mutationFn: () => api.register(email, password),
    onSuccess: (t) => { setToken(t.access_token); nav("/"); },
  });
  return (
    <div className="mx-auto mt-12 max-w-sm px-4">
      <h1 className="mb-6 text-2xl font-semibold">Регистрация</h1>
      <Card>
        {/* При PUBLIC_SIGNUP=false бэкенд ответит 403 «Регистрация временно недоступна» — показываем как есть */}
        <form className="space-y-4" onSubmit={(e) => { e.preventDefault(); m.mutate(); }}>
          <Field label="Email"><Input type="email" value={email} onChange={(e) => setEmail(e.target.value)} /></Field>
          <Field label="Пароль"><Input type="password" value={password} onChange={(e) => setPassword(e.target.value)} autoComplete="new-password" /></Field>
          {m.error && <ErrorBox error={m.error} />}
          <Button type="submit" block disabled={m.isPending}>Зарегистрироваться</Button>
        </form>
```

```
        <p className="mt-4 text-sm text-muted">Уже есть аккаунт? <Link className="text-info" to="/login">Войти</Link></p>
      </Card>
      <Disclaimer />
    </div>
  );
}
```

frontend/src/pages/Registry.tsx (40 строк)

```
import { useQuery } from "@tanstack/react-query";
import { useParams } from "react-router-dom";
import { AnalysisShell } from "../components/AnalysisShell";
import { Disclaimer } from "../components/Disclaimer";
import { Badge, Card, ErrorBox, Spinner } from "../components/ui";
import { api } from "../services/api";
import { pct, rub } from "../services/format";

export default function Registry() {
  const { id = "" } = useParams();
  const q = useQuery({ queryKey: ["registry", id], queryFn: () => api.registry(id) });
  if (q.isLoading) return <Spinner />;
  if (q.error || !q.data) return <ErrorBox error={q.error} />;
  const r = q.data;
  return (
    <AnalysisShell title="Реестр требований">
      {!r.available ? <Card className="text-muted">{r.note}</Card> : (
        <>
          {r.demo_data && <p className="mb-3 text-sm text-info">Демонстрационные данные: строки других кредиторов — оценка по Σ3.</p>}
          {/* Десктоп/планшет — таблица, мобильный — карточки */}
          <table className="hidden w-full table-fixed text-sm md:table">
            <thead><tr className="border-b border-border text-left text-muted">
              <th className="w-1/3 py-2">Кредитор</th><th className="text-right">Сумма</th><th className="text-right">Доля</th><th className="text-right">p_ch</th><th className="pl-4">Действие</th></tr></thead>
            <tbody>{r.rows.map((x) => (
              <tr key={x.name} className="border-b border-border">
                <td className="table-cell py-2 font-medium">{x.name}</td><td className="amount text-right">{rub(x.amount)}</td>
                <td className="amount text-right">{pct(x.share, 1)}</td><td className="amount text-right">{x.p_ch.toFixed(2).replace(".", ",")}</td>
                <td className="pl-4">{x.challenge_candidate ? <Badge tone="warning">П4: рассмотреть оспаривание</Badge> : x.own ? <Badge tone="info">ваше требование</Badge> : "—"}</td>
              </tr>))}</tbody>
          </table>
          <div className="space-y-3 md:hidden">{r.rows.map((x) => (
            <Card key={x.name}><div className="flex justify-between"><span className="font-medium">{x.name}</span><span className="amount">{rub(x.amount)}</span></div>
              <div className="mt-1 text-sm text-muted">Доля {pct(x.share, 1)} · p_ch {x.p_ch.toFixed(2).replace(".", ",")}</div>
              {x.challenge_candidate && <Badge tone="warning">П4: рассмотреть оспаривание</Badge>}
            </Card>))}</div>
        </>
      )}
      <Disclaimer />
    </AnalysisShell>
  );
}
```

frontend/src/pages/Summary.tsx (83 строк)

```
import { useMutation, useQuery, useQueryClient } from "@tanstack/react-query";
import { AlertTriangle } from "lucide-react";
import { Link, useNavigate, useParams } from "react-router-dom";
import { AnalysisShell } from "../components/AnalysisShell";
import { Disclaimer } from "../components/Disclaimer";
```

```tsx
import { Traffic } from "../components/Traffic";
import { Badge, Button, Card, ErrorBox, Select, Spinner } from "../components/ui";
import { api } from "../services/api";
import { MODE_NAMES, QUEUE_NAMES, pct, rub } from "../services/format";

export default function Summary() {
  const { id = "" } = useParams();
  const nav = useNavigate();
  const qc = useQueryClient();
  const q = useQuery({ queryKey: ["summary", id], queryFn: () => api.summary(id) });
  const setMode = useMutation({
    mutationFn: (mode: string) => api.setMode(q.data!.case_id!, mode),
    onSuccess: (s) => { qc.setQueryData(["summary", String(s.id)], s); nav(`/analysis/${s.id}`); },
  });
  if (q.isLoading) return <Spinner />;
  if (q.error || !q.data) return <ErrorBox error={q.error} />;
  const s = q.data;
  const title = s.case?.number ? `Дело ${s.case.number}` : s.case?.inn ? `Должник ИНН ${s.case.inn}` : "Результат анализа";

  return (
    <AnalysisShell title={title}>
      {s.demo_data && <Banner tone="info">Демонстрационные данные: источники КАД/ЕФРСБ работают в режиме mock. Для реального анализа включите PARSERS_MODE=live.</Banner>}
      {s.manual_data && <Banner tone="info">Часть данных введена вручную и не проверена системой.</Banner>}
      {s.status === "partial" && <Banner tone="warn">Часть источников недоступна{s.manual_input_required ? " — требуется ручной ввод" : ""}: {s.warnings.join("; ")}</Banner>}

      <div className="mb-4 flex flex-wrap items-center gap-2 text-sm">
        <Badge tone="info">{MODE_NAMES[s.mode]}</Badge>
        <Badge>{QUEUE_NAMES[s.queue] ?? s.queue}</Badge>
        <span className="amount text-muted">Ваше требование: {rub(s.claim_amount + s.executed_amount)}</span>
      </div>

      {/* Мобильный порядок: рекомендация → выплата → риски. Десктоп: выплата → риски → рекомендация */}
      <div className="flex flex-col gap-4">
        <Card className="order-2 md:order-1">
          <div className="text-sm text-muted">Ожидаемая выплата · {s.probabilistic_label}</div>
          <div className="amount mt-1 text-3xl font-semibold">{rub(s.expected_amount)} <span className="text-xl text-muted">({pct(s.expected_pct)})</span></div>
          <div className="mt-1 text-sm text-muted">Диапазон: {pct(s.range_low_pct)} — {pct(s.range_high_pct)} · {rub(s.range_low)} — {rub(s.range_high)}</div>
          {s.confidence === "low" && (
            <div className="mt-3 flex items-start gap-2 rounded-md bg-warning/10 p-3 text-sm text-warning">
              <AlertTriangle size={16} className="mt-0.5 shrink-0" />Вероятностный прогноз: данных мало, диапазон расширен (25–75%).
            </div>
          )}
        </Card>
        <div className="order-3 md:order-2"><div className="mb-2 font-medium">Риски</div><Traffic risks={s.risks} /></div>
        {s.recommendation && (
          <Card className="order-1 md:order-3 border-info/40">
            <div className="text-sm text-muted">Рекомендация</div>
            <div className="mt-1 text-lg font-medium text-safe">{s.recommendation.title}</div>
            <p className="mt-1 text-muted text-safe">{s.recommendation.reason}</p>
            <Link to={`/analysis/${id}/details`}><Button className="mt-4" block>Что делать →</Button></Link>
          </Card>
        )}
      </div>

      {s.assumptions.length > 0 && (
        <details className="mt-6 text-sm text-muted">
          <summary className="cursor-pointer">Допущения расчёта ({s.assumptions.length})</summary>
          <ul className="mt-2 list-disc space-y-1 pl-5">{s.assumptions.map((a) => <li key={a} className="text-safe">{a}</li>)}</ul>
```

```
        </details>
      )}

      {s.case_id && (
        <div className="mt-6 max-w-sm">
          <label className="mb-1 block text-sm text-muted" htmlFor="mode">Режим (определён автоматически: {MODE_NAMES[s.auto_mode.mode]})</label>
          <Select id="mode" value={s.mode} onChange={(e) => setMode.mutate(e.target.value)} disabled={setMode.isPending}>
            {Object.entries(MODE_NAMES).map(([k, v]) => <option key={k} value={k}>{v}</option>)}
          </Select>
          {(s.mode_manual || setMode.data) && <p className="mt-2 text-xs text-warning">Вы выбрали режим вручную. Автоматический расчёт может отличаться. Убедитесь, что данные актуальны.</p>}
        </div>
      )}
      <Disclaimer also={["fee", "ads"]} />
    </AnalysisShell>
  );
}

function Banner({ tone, children }: { tone: "info" | "warn"; children: React.ReactNode }) {
  return <div role="note" className={`mb-4 rounded-md p-3 text-sm text-safe ${tone === "info" ? "bg-info/10 text-info" : "bg-warning/10 text-warning"}`}>{children}</div>;
}
```

frontend/src/services/api.ts (104 строк)

```
// Клиент API. Токен хранится в localStorage; при 401 — выход на /login.
export class ApiError extends Error {
  constructor(public status: number, message: string, public requestId?: string) { super(message); }
}

const TOKEN_KEY = "cp_token";
export const getToken = () => localStorage.getItem(TOKEN_KEY);
export const setToken = (t: string | null) => (t ? localStorage.setItem(TOKEN_KEY, t) : localStorage.removeItem(TOKEN_KEY));

async function request<T>(path: string, init: RequestInit = {}): Promise<T> {
  const headers = new Headers(init.headers);
  const t = getToken();
  if (t) headers.set("Authorization", `Bearer ${t}`);
  if (init.body && !(init.body instanceof FormData)) headers.set("Content-Type", "application/json");
  const r = await fetch(`/api/v1${path}`, { ...init, headers });
  if (r.status === 401 && !path.startsWith("/auth/")) {
    setToken(null);
    if (location.pathname !== "/login") location.assign("/login");
  }
  const data = await r.json().catch(() => ({}));
  if (!r.ok) throw new ApiError(r.status, data.error || "Ошибка запроса", data.request_id);
  return data as T;
}

export const api = {
  login: (email: string, password: string) =>
    request<{ access_token: string }>("/auth/login", { method: "POST", body: JSON.stringify({ email, password }) }),
  localLogin: () => request<{ access_token: string }>("/auth/local", { method: "POST" }),
  register: (email: string, password: string) =>
    request<{ access_token: string }>("/auth/register", { method: "POST", body: JSON.stringify({ email, password }) }),
  me: () => request<Me>("/users/me"),
  changePassword: (old_password: string, new_password: string) =>
    request("/users/me/password", { method: "POST", body: JSON.stringify({ old_password, new_password }) }),
  subscription: () => request<{ status: string; tier: string; message?: string }>("/users/me/subscription"),
  analyze: (body: AnalyzeBody) => request<Summary>("/analyze", { method: "POST", body: JSON.stringify(body) }),
```

```
  analyses: () => request<AnalysisRow[]>("/analyses"),
  summary: (id: string) => request<Summary>(`/analysis/${id}`),
  details: (id: string) => request<Details>(`/analysis/${id}/details`),
  audit: (id: string) => request<Audit>(`/analysis/${id}/audit`),
  registry: (id: string) => request<RegistryResp>(`/analysis/${id}/registry`),
  setMode: (caseId: number, mode: string) =>
    request<Summary & { warning: string }>(`/cases/${caseId}/set-mode`, { method: "POST", body: JSON.stringify({ mode }) }),
  documents: (claimId: number) => request<Doc[]>(`/claims/${claimId}/documents`),
  upload: (claimId: number, file: File) => {
    const fd = new FormData(); fd.append("file", file);
    return request<UploadResp>(`/claims/${claimId}/documents`, { method: "POST", body: fd });
  },
  docStatus: (id: number) => request<DocStatus>(`/documents/${id}/status`),
  confirmFacts: (id: number) =>
    request<{ confirmed: number; total: number; analysis_id: number | null }>(`/documents/${id}/confirm-facts`, { method: "POST", body: "{}" }),
};

export type Me = { id: number; email: string; role: string; tier: string;
  flags: { billing_enabled: boolean; public_signup: boolean; public_mode: boolean; show_cookie_banner: boolean; parsers_mode: string } };
export type ClaimBody = { amount?: number; construction_type?: string; claim_date?: string; execution_date?: string;
  executed_amount?: number; status?: string; crisis?: boolean; rescue_form?: boolean; unusual_conditions?: boolean };
export type ManualBody = { filing_date?: string; total_assets?: number; current_expenses?: number; sigma3?: number;
  judge?: { n_cases: number; approval_rate: number; is_significant: boolean };
  collateral?: { C: number; L: number }[] };
export type AnalyzeBody = { inn?: string; case_number?: string; claim?: ClaimBody; manual?: ManualBody; mode?: string };
export type Risk = { key: string; title: string; p: number; light: "red" | "yellow" | "green"; label: string };
export type Action = { key: string; title: string; reason: string; fee_rub: number; months: string; lawyers_rub: [number, number];
  expected_gain_rub: number | null; economically_justified: boolean };
export type Summary = {
  id: number; claim_id: number; case_id: number | null; mode: string; queue: string;
  case: { number?: string; court?: string; debtor?: string; inn?: string } | null;
  claim_amount: number; executed_amount: number; expected_amount: number; expected_pct: number;
  range_low_pct: number; range_high_pct: number; range_low: number; range_high: number;
  confidence: "low" | "medium" | "high"; probabilistic_label: string; risks: Risk[];
  recommendation: { title: string; reason: string } | null; assumptions: string[]; warnings: string[];
  status: string; manual_input_required: boolean; mode_manual: boolean; demo_data: boolean; manual_data: boolean;
  auto_mode: { mode: string; reason: string };
  disclaimers: { general: string; ads: string };
};
export type Scenario = { name: string; title: string; probability: number; payout_rub: number; payout_pct: number };
export type Details = {
  scenarios: Scenario[]; actions: Action[]; priorities: string[]; checklist: string[];
  calendar: { date: string | null; title: string }[];
  au: { name: string; sro: string; n_cases: number } | null;
  au_index: { score: number; critical: boolean; items: { label: string; points: number }[] };
  judge: { name: string; n_cases: number; approval_rate: number; is_significant: boolean } | null;
  information_levels: Record<string, number>;
  disclaimers: Record<string, string>;
};
export type Audit = {
  formulas: string[]; formula: string; method: string;
  probabilities: { p_sub: number; p_ch: number; p_zero: number; delta_practice: number; delta_au: number };
  judge: { name: string; n_cases: number; approval_rate: number; is_significant: boolean } | null;
  judge_delta: { deviation?: number; note?: string; profile?: boolean };
  au_index: { score: number; critical: boolean; items: { label: string; points: number }[] };
  modules: Record<string, Record<string, unknown>>;
  information_levels: Record<string, number>; assumptions: string[];
  sources: { title: string; url: string }[]; disclaimers: Record<string, string>;
```

```
  audit: Record<string, unknown>;
};
export type RegistryResp = { available: boolean; note: string | null; demo_data: boolean;
  rows: { name: string; amount: number; share: number; p_ch: number; challenge_candidate: boolean; own: boolean }[] };
export type AnalysisRow = { id: number; mode: string; case: { number?: string; debtor?: string } | null;
  expected_amount: number; expected_pct: number; calculated_at: string };
export type Doc = { id: number; document_type: string; mime_type: string; file_size: number; status: string };
export type UploadResp = { id: number; status: string; document_type: string; needs_confirmation: boolean;
  facts: { field: string; value: string; confidence: number }[] };
export type DocStatus = { id: number; status: string; document_type: string;
  facts: { id: number; field: string; value: string; confidence: number; confirmed: boolean; label: string }[] };
```

frontend/src/services/format.ts (18 строк)

```
const nf = new Intl.NumberFormat("ru-RU", { maximumFractionDigits: 0 });
export const rub = (n: number | null | undefined) => (n == null ? "—" : `${nf.format(Math.round(n))} ₽`);
export const pct = (p: number | null | undefined, digits = 0) =>
  p == null ? "—" : `${(p * 100).toFixed(digits).replace(".", ",")}%`;
export const mln = (n: number) => `${(n / 1e6).toFixed(1).replace(".", ",")} млн`;

export const MODE_NAMES: Record<string, string> = {
  P0: "P0 · До сделки", P1: "P1 · Сделка заключена", P2: "P2 · Признаки кризиса",
  P3: "P3 · Заявление подано", P4: "P4 · Судья и АУ известны", P5: "P5 · Реестр закрыт", P6: "P6 · Торги",
};
export const QUEUE_NAMES: Record<string, string> = {
  current: "Текущие платежи (вне очереди)", secured: "Залоговый кредитор", third: "Третья очередь",
  third_subordinated: "Третья очередь, риск субординации",
};
export const SCENARIO_COLORS: Record<string, string> = {
  base: "var(--color-success)", challenged: "var(--color-warning)", subordinated: "var(--color-danger)",
  subordinated_after_ch: "#9333ea", zero: "var(--color-text-muted)",
};
```

frontend/src/styles/index.css (20 строк)

```
@import "./tokens.css";
@tailwind base;
@tailwind components;
@tailwind utilities;

html { font-family: var(--font-sans); }
body { background: var(--color-bg); color: var(--color-text); font-size: var(--text-base); line-height: var(--leading-base); -webkit-font-smoothing: antialiased; }

/* Правила устойчивости вёрстки (ТЗ 12.1) */
.text-safe { overflow-wrap: anywhere; word-break: break-word; }
.line-clamp-1 { display: -webkit-box; -webkit-line-clamp: 1; -webkit-box-orient: vertical; overflow: hidden; }
.line-clamp-2 { display: -webkit-box; -webkit-line-clamp: 2; -webkit-box-orient: vertical; overflow: hidden; }
.line-clamp-3 { display: -webkit-box; -webkit-line-clamp: 3; -webkit-box-orient: vertical; overflow: hidden; }
.amount { white-space: nowrap; font-variant-numeric: tabular-nums; }
.table-cell { min-width: 0; overflow: hidden; text-overflow: ellipsis; white-space: nowrap; }
h1, h2, h3 { text-wrap: balance; }
p { text-wrap: pretty; }

:focus-visible { outline: 2px solid var(--color-info); outline-offset: 2px; }
@media (prefers-reduced-motion: reduce) { * { transition: none !important; animation: none !important; } }
```

frontend/src/styles/tokens.css (51 строк)

```
:root {
  --color-primary: #1e293b;
  --color-primary-hover: #334155;
  --color-bg: #ffffff;
  --color-surface: #f8fafc;
  --color-surface-2: #f1f5f9;
  --color-border: #e2e8f0;
  --color-text: #0f172a;
  --color-text-muted: #64748b;
  --color-success: #16a34a;
  --color-warning: #d97706;
  --color-danger: #dc2626;
  --color-info: #2563eb;

  --font-sans: 'Inter', system-ui, sans-serif;
  --text-xs: 12px; --leading-xs: 16px;
  --text-sm: 14px; --leading-sm: 20px;
  --text-base: 16px; --leading-base: 24px;
  --text-lg: 18px; --leading-lg: 28px;
  --text-xl: 20px; --leading-xl: 28px;
  --text-2xl: 24px; --leading-2xl: 32px;
  --text-3xl: 30px; --leading-3xl: 36px;

  --space-1: 4px; --space-2: 8px; --space-3: 12px;
  --space-4: 16px; --space-5: 20px; --space-6: 24px;
  --space-8: 32px; --space-10: 40px; --space-12: 48px;

  --radius-sm: 6px; --radius-md: 10px; --radius-lg: 16px;

  --shadow-sm: 0 1px 2px rgba(15, 23, 42, 0.06);
  --shadow-md: 0 4px 12px rgba(15, 23, 42, 0.08);
  --shadow-lg: 0 12px 32px rgba(15, 23, 42, 0.12);

  --transition-fast: 150ms ease;
  --transition-base: 200ms ease;
}

/* Тёмная тема: переключается в профиле, акцент — светлый слейт */
[data-theme="dark"] {
  --color-primary: #e2e8f0;
  --color-primary-hover: #f8fafc;
  --color-bg: #0b1220;
  --color-surface: #111a2e;
  --color-surface-2: #1a2540;
  --color-border: #263354;
  --color-text: #f1f5f9;
  --color-text-muted: #94a3b8;
  --shadow-sm: 0 1px 2px rgba(0, 0, 0, 0.4);
  --shadow-md: 0 4px 12px rgba(0, 0, 0, 0.45);
  --shadow-lg: 0 12px 32px rgba(0, 0, 0, 0.55);
}
```

frontend/.dockerignore (3 строк)

```
node_modules
dist
test-results
```

frontend/Dockerfile (19 строк)

```
# --- dev: Vite с HMR (docker-compose.yml) ---
FROM node:20-alpine AS dev
WORKDIR /app
```

```
COPY package*.json ./
RUN npm install --no-audit --no-fund
COPY . .
ENV VITE_API_PROXY=http://backend:8000
EXPOSE 5173
CMD ["npm", "run", "dev"]

# --- build: статическая сборка ---
FROM dev AS build
RUN npm run build

# --- prod: статика под nginx (docker-compose.prod.yml) ---
FROM nginx:1.27-alpine AS prod
COPY --from=build /app/dist /usr/share/nginx/html
RUN printf 'server { listen 80; root /usr/share/nginx/html; location / { try_files $uri /index.html; } }\n' > /etc/nginx/conf.d/default.conf
EXPOSE 80
```

frontend/e2e/smoke.spec.ts (37 строк)

```
import { expect, test } from "@playwright/test";

test("главная → ввод ИНН → сводка → детали → аудит", async ({ page }) => {
  await page.goto("/");
  await expect(page).toHaveURL(/\/login/);
  await page.getByLabel("Пароль").fill("admin");
  await page.getByRole("button", { name: "Войти" }).click();

  await expect(page.getByRole("heading", { name: "Профиль кредитора" })).toBeVisible();
  await expect(page.getByTestId("disclaimer").first()).toContainText("информационно-аналитический");
  await page.getByLabel("ИНН или номер дела").fill("7701234567");
  await page.getByRole("button", { name: /Рассчитать/ }).click();

  await expect(page.getByText("Ожидаемая выплата")).toBeVisible();
  await expect(page.getByLabel("Светофор рисков")).toBeVisible();
  await expect(page.getByText("Вероятностный прогноз").first()).toBeVisible();
  await expect(page.getByTestId("disclaimer")).toBeVisible();

  await page.getByRole("link", { name: "Детали" }).first().click();
  await expect(page.getByRole("heading", { name: "Сценарии" })).toBeVisible();
  await expect(page.getByRole("heading", { name: "Стоимость действий" })).toBeVisible();

  await page.getByRole("link", { name: "Аудит" }).first().click();
  await expect(page.getByRole("heading", { name: "Формулы" })).toBeVisible();
});

test("служебные страницы существуют и содержат дисклеймер", async ({ page }) => {
  for (const path of ["/terms", "/privacy", "/offer", "/register"]) {
    await page.goto(path);
    await expect(page.getByTestId("disclaimer")).toBeVisible();
  }
  await page.goto("/register");
  await page.getByLabel("Email").fill("x@y.ru");
  await page.getByLabel("Пароль").fill("secret1");
  await page.getByRole("button", { name: "Зарегистрироваться" }).click();
  await expect(page.getByRole("alert")).toContainText("Регистрация временно недоступна");
});
```

frontend/index.html (15 строк)

```
<!doctype html>
<html lang="ru">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0, viewport-fit=cover" />
    <meta name="theme-color" content="#1e293b" />
```

```
    <link rel="icon" href="/icons/icon.svg" type="image/svg+xml" />
    <link rel="apple-touch-icon" href="/icons/icon-192.png" />
    <title>Профиль кредитора</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

frontend/package.json (35 строк)

```
{
  "name": "creditor-profile-frontend",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite --host 0.0.0.0",
    "build": "tsc --noEmit && vite build",
    "preview": "vite preview --host 0.0.0.0",
    "typecheck": "tsc --noEmit",
    "e2e": "playwright test --trace off"
  },
  "dependencies": {
    "@tanstack/react-query": "^5.51.0",
    "clsx": "^2.1.1",
    "lucide-react": "^0.424.0",
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "react-router-dom": "^6.26.0",
    "recharts": "^2.12.7",
    "tailwind-merge": "^2.4.0"
  },
  "devDependencies": {
    "@playwright/test": "^1.45.0",
    "@types/react": "^18.3.3",
    "@types/react-dom": "^18.3.0",
    "@vitejs/plugin-react": "^4.3.1",
    "autoprefixer": "^10.4.19",
    "postcss": "^8.4.40",
    "tailwindcss": "^3.4.7",
    "typescript": "^5.5.4",
    "vite": "^5.4.0",
    "vite-plugin-pwa": "^0.20.1"
  }
}
```

frontend/playwright.config.ts (16 строк)

```
import { defineConfig } from "@playwright/test";

// Перед запуском: бэкенд на :8000 и `npm run build && npm run preview` (:4173) либо `npm run dev` (:5173)
export default defineConfig({
  testDir: "./e2e",
  timeout: 30_000,
  use: {
    baseURL: process.env.E2E_BASE_URL || "http://localhost:4173", trace: "off",
    // PW_CHROMIUM=/путь/к/chromium — если браузер установлен отдельно от версии Playwright
    launchOptions: process.env.PW_CHROMIUM ? { executablePath: process.env.PW_CHROMIUM } : {},
  },
  projects: [
    { name: "desktop", use: { viewport: { width: 1440, height: 900 } } },
    { name: "mobile", use: { viewport: { width: 390, height: 844 }, isMobile: true, hasTouch: true } },
  ],
```

```
});
```

frontend/postcss.config.js (1 строк)

```
export default { plugins: { tailwindcss: {}, autoprefixer: {} } };
```

frontend/public/icons/icon.svg (1 строк)

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 512 512"><rect width="512" height="512" rx="96" fill="#1e293b"/><path d="M256 96l128 48v96c0 88-56 142-128 176-72-34-128-88-128-176v-96z" fill="none" stroke="#fff" stroke-width="28" stroke-linejoin="round"/><path d="M200 256l44 44 76-88" fill="none" stroke="#fff" stroke-width="28" stroke-linecap="round" stroke-linejoin="round"/></svg>
```

frontend/public/manifest.json (14 строк)

```
{
  "name": "Профиль кредитора",
  "short_name": "Кредитор",
  "description": "Прогноз выплаты и риски кредитора в банкротстве",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#1e293b",
  "lang": "ru",
  "icons": [
    { "src": "/icons/icon-192.png", "sizes": "192x192", "type": "image/png", "purpose": "any maskable" },
    { "src": "/icons/icon-512.png", "sizes": "512x512", "type": "image/png", "purpose": "any maskable" }
  ]
}
```

frontend/tailwind.config.js (20 строк)

```
/** @type {import('tailwindcss').Config} */
export default {
  content: ["./index.html", "./src/**/*.{ts,tsx}"],
  darkMode: ["class", '[data-theme="dark"]'],
  theme: {
    screens: { md: "768px", lg: "1440px" },
    extend: {
      colors: {
        primary: "var(--color-primary)", "primary-hover": "var(--color-primary-hover)",
        bg: "var(--color-bg)", surface: "var(--color-surface)", surface2: "var(--color-surface-2)",
        border: "var(--color-border)", fg: "var(--color-text)", muted: "var(--color-text-muted)",
        success: "var(--color-success)", warning: "var(--color-warning)", danger: "var(--color-danger)",
info: "var(--color-info)",
      },
      fontFamily: { sans: ["Inter", "system-ui", "sans-serif"] },
      borderRadius: { sm: "var(--radius-sm)", md: "var(--radius-md)", lg: "var(--radius-lg)" },
      boxShadow: { sm: "var(--shadow-sm)", md: "var(--shadow-md)", lg: "var(--shadow-lg)" },
    },
  },
  plugins: [],
};
```

frontend/tsconfig.json (8 строк)

```
{
  "compilerOptions": {
    "target": "ES2020", "lib": ["ES2020", "DOM", "DOM.Iterable"], "module": "ESNext",
    "moduleResolution": "bundler", "jsx": "react-jsx", "strict": true, "skipLibCheck": true,
    "noEmit": true, "isolatedModules": true, "resolveJsonModule": true, "types": ["vite/client", "vite-
plugin-pwa/client"]
  },
```

```
  "include": ["src"]
}
```

frontend/vite.config.ts (25 строк)

```
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { VitePWA } from "vite-plugin-pwa";

export default defineConfig({
  plugins: [
    react(),
    VitePWA({
      registerType: "autoUpdate",
      manifest: false, // используем public/manifest.json
      workbox: {
        globPatterns: ["**/*.{js,css,html,svg,png}"],
        navigateFallback: "/index.html",
        navigateFallbackDenylist: [/^\/api/, /^\/health/, /^\/metrics/, /^\/docs/],
      },
    }),
  ],
  server: {
    port: 5173,
    proxy: {
      "/api": process.env.VITE_API_PROXY || "http://localhost:8000",
      "/health": process.env.VITE_API_PROXY || "http://localhost:8000",
    },
  },
});
```